

Manual de supervivencia a Comunicaciones

Facultad de Ciencias Exactas, Ingeniería y Agrimensura
Escuela de Ciencias Exactas y Naturales

Mateo Bacci, Juan Figueredo

2 de mayo de 2024

Índice

1. Introducción	4
1.1. Hardware de Red	4
1.2. Software de Red	6
1.3. Modelo de Referencia OSI	8
1.4. Modelo TCP/IP	10
2. Capa Física	13
2.1. Análisis Teórico	13
2.2. Medios de Transmisión Guiados	15
2.3. Medios de Transmisión Inalámbricos	18
2.4. Topologías de redes	21
3. Capa de Enlace	23
3.1. Subcapa LLC	23
3.2. Subcapa MAC	26
3.3. Tipos de trama en el IEEE 802.11	29
4. Capa de Red	31
4.1. ARP	31
4.2. Protocolo IP	34
4.3. IPv4	38
4.4. IPv6	46
4.5. ICMP	57
4.6. ICMPv6	59
5. Capa de Transporte	63
5.1. Protocolo de Datagrama de Usuario (UDP)	63
5.2. Protocolo de Control de Transmisión (TCP)	66
6. Capa de Aplicación	81
6.1. Domain Name Server (DNS)	81
6.2. Firewall	95

7. Preguntas Frecuentes	102
7.1. Preguntas	102
7.2. Respuestas	104
8. Glosario	107
9. Bibliografía	114

Prefacio

Durante el cursado de la materia, se recorren numerosos temas que suelen habitar en largas y complicadas bibliografías (Tanenbaum, Comer, etc). Este resumen/manual está dedicado a poder combinar todas estas fuentes en un único archivo para, de algún modo, aumentar las probabilidades de no solo entender mejor la materia, sino también de sobrevivir a ella.

Recomendamos usar este apunte para tener una primera idea de la materia o para usar de repaso una vez leídos los libros recomendados. Aunque intentamos hacerlo lo más completo posible, no recomendamos usarlo como única fuente de estudio antes de rendir.

La mayoría (por no decir toda) la información la sacamos del [Tanenbaum](#) y del [Comer](#). Al final de cada sección aclaramos los capítulos usados, y en algunos casos, las slides de la cátedra.

El apunte también cuenta con un [glosario](#) con conceptos claves a la materia y sus definiciones.

Mateo y Juan.

PD: Las palabras en [rosa](#) son links a páginas web externas, como: [Música para sobrevivir a Comunicaciones](#)

1. Introducción

Esta unidad trata de introducir al lector en el mundo de las redes, brindando un acercamiento a los tipos de redes, y presenta el famoso Modelo OSI, cuyas capas son el eje principal de la materia.

1.1. Hardware de Red

Al hablar de *hardware de red* hablamos de las cuestiones técnicas implicadas en el diseño e implementación de dichas redes. Existen muchos tipos de redes y muchas variaciones de cada tipo, por lo que encontrar categorías en las cuales caigan absolutamente TODAS las redes es casi imposible. De este modo nos concentraremos en solamente dos aspectos: la tecnología de transmisión y la escala.

1.1.1. Difusión vs Punto a Punto

En lo que a la tecnología de transmisión respecta, una red puede pertenecer a dos clases: Red de **difusión** (Broadcasting) o red de **Punto a Punto**.

En las redes Punto a Punto, las máquinas están conectadas en pares individuales que forman una especie de red. De esta manera, si un paquete se envía de una máquina a otra, probablemente deba pasar entre varias máquinas intermedias hasta llegar a su destino. Esto permite que puedan existir caminos de distinta longitud entre dos máquinas, por lo que lo que es importante encontrar la ruta correcta a la hora de transmitir Punto a Punto. Las redes Punto a Punto con un solo emisor y un solo receptor se conocen como redes de *Unidifusión* (Unicast).

Las redes de Difusión son aquellas en las que todas las máquinas involucradas comparten el mismo canal de comunicación, de modo que cualquier paquete que envíe una máquina, será recibido por todas las máquinas de la red. Los paquetes cuentan con un campo de dirección que especifica a la máquina a la que se dirige. Cuando se recibe un paquete se verifica la dirección del mismo, si este coincide con la dirección de la máquina receptora, el paquete se procesa. Si no coincide, simplemente se ignora.

1.1.2. Escala de Redes

El otro aspecto importante es el que refiere a la escala de la red, ya que a medida que la escala de una red va creciendo, las tecnologías involucradas cambian. Podemos distinguir 4 escalas de red:

- PAN

Las redes de área personal (**PAN**, de *Personal Area Network*) en las que los dispositivos se deben comunicar entre sí en el rango de una persona. Un claro ejemplo puede ser la red entre una computadora y sus periféricos.

■ LAN

Las redes LAN, o de área local (*Local Area Network*) poseen un rango de un edificio (una casa, una oficina, una empresa). Normalmente se utiliza para conectar computadoras personales con electrodomésticos e intercambiar información. Muchas redes LAN alámbricas están basadas en los enlaces Punto a Punto y utilizan comunmente el protocolo Ethernet.

Muchas veces, dentro de un mismo organismo, se utilizan redes LAN para cada uno de los departamentos que los integran, sin embargo, puede ocurrir que esta división lógica no se pueda lograr de manera física (ambos departamentos están en la misma oficina, por ejemplo). Para esto, surge el concepto de **VLAN** (o LAN Virtual) en donde se divide “virtualmente” una red en varias redes. Se utiliza un Switch que se encarga de reenviar paquetes de una maquina en una red virtual a otra en la misma red virtual. De modo que se simula la existencia de varias redes LAN distintas.

■ MAN

Una red de área metropolitana (MAN, *Metropolitan Area Network*) cubre una ciudad entera. El ejemplo más común es el de las redes de televisión por cable.

■ WAN

Una red WAN es una red de área amplia (*Wide Area Network*) y abarca una extensa área geográfica, como un país o un continente. Contamos con redes WAN tanto alámbricas como inalámbricas. Dentro de una red WAN podemos distinguir dos cosas, los **hosts**, es decir, las máquinas y la **subred**¹, cuya tarea es la de transportar los mensajes entre hosts.

Si analizamos esta subred, descubrimos dos componentes distintos: Las líneas de transmisión, que mueven bits entre máquinas (cables de cobre, fibra óptica, enlaces de radio, etc) y se suelen alquilar a una compañía de telecomunicaciones. Por otro lado encontramos a los elementos de conmutación, que operan como switches a gran escala. A estos elementos se los llama **enrutadores** o **routers**.

A simple vista, una red WAN es muy similar a una red LAN (sin considerar la escala, obviamente), aunque podemos seguir encontrando algunas diferencias fundamentales. Primero, en las redes WAN, los hosts y la subred suelen pertenecer a distintas personas. Segundo, los enrutadores, suelen conectar distintas tecnologías, es decir, las subredes pueden trabajar un tipo de tecnología completamente distinto

¹No confundir con el significado de subred en la capa de red

al que trabajan los hosts. Y por último, los hosts no siempre son computadoras individuales (como en redes LAN), sino que podrían ser redes LAN completas.

Las redes WAN pueden presentar variaciones. Nos concentraremos en dos posibles variaciones. La primera se da cuando en vez de utilizar líneas de transmisión dedicadas, una empresa conecta sus oficinas a internet. Esto les permite hacer conexiones entre las oficinas como enlaces virtuales. Este arreglo se conoce como **VPN** (Red Privada Virtual). Con respecto al arreglo común, una VPN tiene la ventaja de la virtualización, que se aprecia cuando se quiere agregar un host nuevo, por ejemplo. Sin embargo se pierde control sobre los recursos, ya que la línea de transmisión está delegada.

La otra variación ocurre cuando una empresa distinta opera la subred. A este operador de lo conoce como **proveedor de servicios de red**. Este operador usualmente también puede conectarse otras redes que formen parte del internet, y en estos casos se los conoce como **ISP** (Internet Service Provider).

1.2. Software de Red

En un intento por reducir la complejidad de su diseño, las redes suelen dividirse en capas o niveles. La cantidad de capas y sus funciones difieren de una red a otra, pero el propósito de cada capa es el mismo, brindar servicios a las capas superiores, mientras que les oculta los detalles relacionados a su implementación.

Cuando la capa n de una máquina se quiere comunicar con la capa n de otra máquina, lo hace a través de reglas y convenciones previamente establecidas. Este conjunto de especificaciones se conoce un **protocolo** (en particular, sería el protocolo de la capa n). En pocas palabras, un protocolo es un acuerdo entre las partes que se comunican, para establecer la forma en la que se llevará a cabo la comunicación.

La comunicación entre capas, aunque para cada capa parezca que se hace directamente hacia la misma capa de la otra máquina, se realiza mediante un medio físico (Cable de cobre, fibra óptica, etc), que sería la capa 1. De esta manera, cuando una capa quiere enviar un mensaje, este “baja” hacia las capas inferiores hasta llegar a la capa física, donde se realiza la transmisión hacia la máquina receptora. De ahí, sube nuevamente por las capas hasta llegar a la capa destino.

Como dijimos, la comunicación entre capas se realiza mediante servicios que se prestan, que están definidos por la **interfaz**. A la hora de diseñar una red, definir las interfaces para cada capa es una de las consideraciones más importantes. Tener interfaces bien definidas permite abstraerse completamente de la implementación y la tecnología que cada capa este utilizando, de modo que no importa cual sea el protocolo o la implementación de una capa, mientras ofrezca los servicios correctos.

Cuando hablamos de capas y protocolos hablamos de **arquitectura de red**.

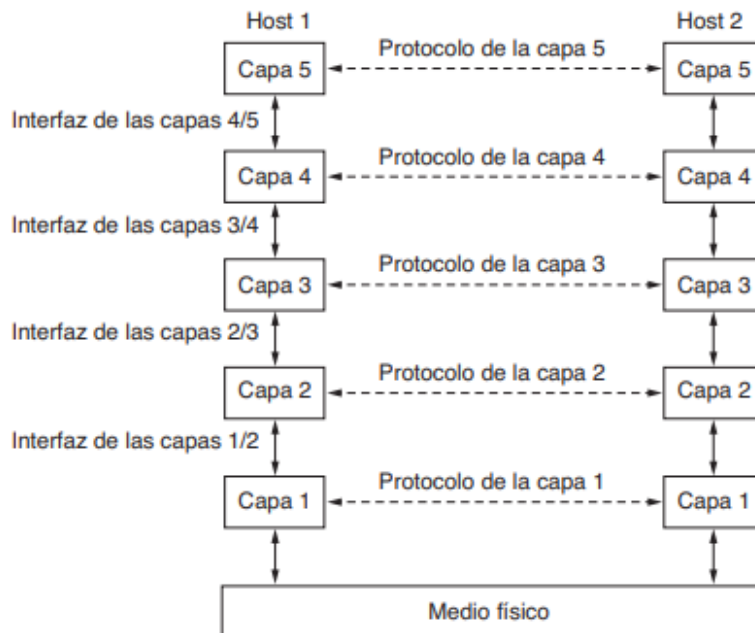


Figura 1: Capas, protocolos e interfaces - [Tanenbaum](#).

Ahora, hablemos un poco sobre los **servicios**. Una capa puede ofrecer dos tipos de servicios a sus capas superiores:

- **Servicio orientado a la conexión:** modelado a partir del sistema telefónico. Una computadora que desea comunicarse con otra, primero establece una conexión (llama a la otra computadora, y la otra atiende), la utiliza (habla) y luego termina la conexión (cuelga). Encontramos dos variaciones de los servicios orientados a la conexión: las secuencias de mensajes y el flujo de bytes. En la primera, se conservan los límites de cada mensaje, por ejemplo, si se envían dos mensajes de 1024 bytes, llegan dos mensajes de 1024 bytes, y nunca uno de 2048. La segunda variante es simplemente una conexión sin límites en donde los bytes se transportan libremente y no hay manera de saber cómo fueron enviados originalmente.
- **Servicio sin conexión:** similar al sistema postal. Una computadora manda un paquete (carta) con la dirección de la computadora de destino. Este mensaje es enrutado hacia nodos intermedios dentro del sistema, que reenvían el mensaje al siguiente nodo. Normalmente, llamamos a este tipo como servicio de **datagramas** (en analogía a los telegramas) cuando no se requiere una confirmación de recepción, y servicio de **datagramas con confirmación de recepción** cuando sí se requiere.

Podemos caracterizar cada servicio según su **confiabilidad**. Decimos que un servicio es **confiable** si se espera una confirmación de recepción del mensaje, para que el emisor esté seguro de que sus mensajes han llegado. Es claro que la confirmación de recepción implica una mayor carga y un retardo que según la situación pueden valer la pena (como en una transferencia de archivos) o no ser deseables (como una transmisión de voz). Es importante

	Servicio	Ejemplo
Orientado a conexión	Flujo de mensajes confiable.	Secuencia de páginas.
	Flujo de bytes confiable.	Descarga de películas.
	Conexión no confiable.	Voz sobre IP.
Sin conexión	Datagrama no confiable.	Correo electrónico basura.
	Datagrama confirmación de recepción.	Mensajería de texto.
	Solicitud-respuesta.	Consulta en una base de datos.

Figura 2: Tipos de servicios - [Tanenbaum](#).

comprender que los servicios y los protocolos no dependen uno del otro. Los servicios se relacionan con las interfaces entre capas, definen qué operaciones puede realizar la capa, pero no dicen nada de su implementación. Mientras que los protocolos se relacionan con los paquetes que se envían entre las entidades pares de distintas máquinas.

1.3. Modelo de Referencia OSI

El Modelo de Referencia OSI (Open System Interconnection) fue el primer intento de generar una estandarización en los protocolos utilizados en las distintas capas. Aunque sus protocolos no se suelen utilizar, su modelo es bastante general y sigue siendo válido. El modelo cuenta con siete capas (Veremos algunas más adelante) y son el eje principal de la materia. Para lograr determinar estas capas, se tuvieron en cuenta ciertos principios:

1. Se debe crear una capa donde se requiera un nivel diferente de abstracción.
2. Cada capa debe realizar una función bien definida.
3. La función de cada capa se debe elegir teniendo en cuenta a los protocolos estandarizados.
4. Se deben elegir límites entre las capas de modo que se minimice el flujo de datos entre las interfaces.

5. La cantidad de capas debe ser suficiente como para no tener capas que hagan muchas funciones distintas, pero lo bastante pequeña como para que no sea inmanejable.

Veamos ahora, de manera rápida y resumida, de que se tratan las distintas capas del modelo OSI.

1. Capa física

Relacionada con la transmisión pura de bits a través de un canal de transmisión. Encargada de asegurarse que los bits que se envíen lleguen de manera correcta. En esta capa se toman las decisiones de las distintas interfaces mecánicas y eléctricas.

2. Capa de enlace

Encargada de transformar el medio de transmisión puro a una línea que esté libre de errores. Enmascara los errores reales de manera que la capa de red no los vea. Divide los datos de entrada en **tramas de datos** y los transmite de forma secuencial. En las redes de difusión, esta capa normalmente está subdividida en dos: la **capa LLC** (*Logical Link Control*) y la **capa de acceso al medio MAC** (*Medium Access Control*). La tarea de la subcapa MAC es controlar que los transmisores no se pisen cuando quieran comunicarse.

3. Capa de red

Controla la operación de la subred. Los paquetes que se envíen deben ir direccionándose desde el origen hasta llegar al destino. Usualmente se logra mediante tablas de direccionamiento estáticas o dinámicas. Es responsabilidad de la capa de red ocuparse de problemas como cuellos de botellas, incompatibilidad de protocolos entre redes o problemas de tamaños de paquetes.

4. Capa de transporte

Encargada de procesar la información a transportar, dividiéndola en partes más pequeñas para la capa de red, y asegurar que la información llegue correctamente al destino, manteniendo la eficiencia. Esta capa trabaja de una manera **Extremo a Extremo**² ya que su comunicación es directamente con la máquina de destino (a través de encabezados y mensajes de control), y no entre la máquina y sus dispositivos vecinos, como en las capas inferiores.

5. Capa de sesión

Permite a los usuarios de distintas máquinas establecer **sesiones** entre ellos. Estas sesiones brindan servicios como control de diálogo, manejo de tokens y sincronización.

²No confundir con una conexión Punto a Punto

6. Capa de presentación

Se enfoca, principalmente, en la sintaxis y la semántica de la información que se transmite. Maneja estructuras de datos abstractas y permite el intercambio de estructuras de datos de mayor nivel entre máquinas.

7. **Capa de aplicación** Esta capa contiene variedades de protocolos que los usuarios utilizan con frecuencia para la transferencia de archivos, correos electrónicos, etc. Algunos ejemplos son: **HTTP** (*HyperText Transfer Protocol*), **SFTP** (*Secure File Transfer Protocol*) o **SSH** (*Secure Shell*), entre otros.

En la figura 3 se puede apreciar que las capas 1-3 son capas que trabajan Punto a Punto, y las capas 4-7 trabajan de Extremo a Extremo.

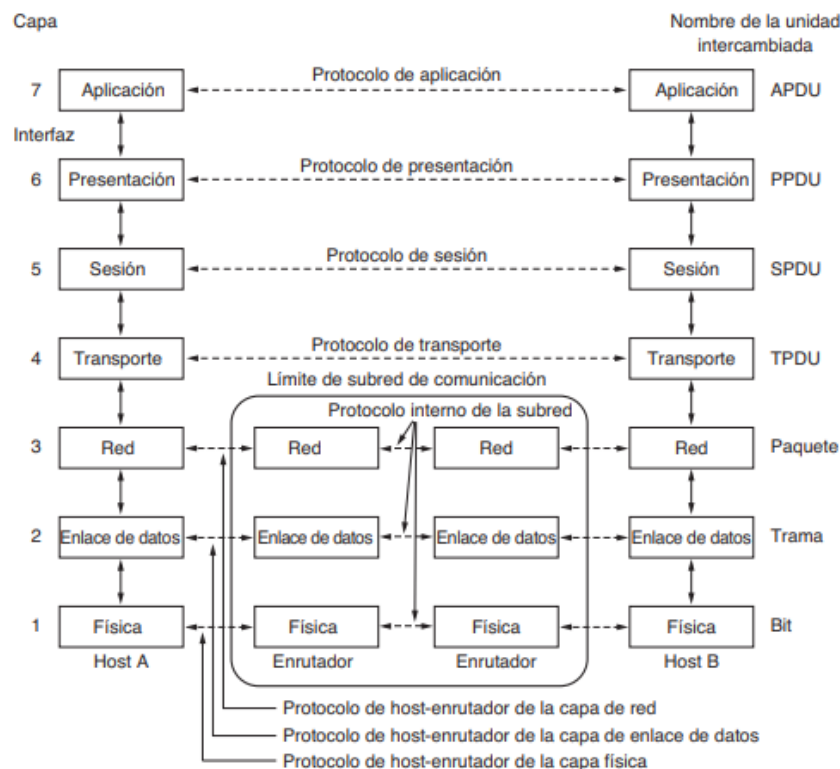


Figura 3: Modelo de referencia OSI - Tanenbaum.

1.4. Modelo TCP/IP

El modelo de referencia TCP/IP nació con el propósito de permitir la conexión de varias redes sin problemas. Otro de los objetivos era poder modelar redes que puedan sobrevivir a la pérdida de hardware de la subred, es decir que se mantenga la conexión siempre y cuando las máquinas de origen y destino estuvieran funcionando. Estos requerimientos

llevaron a la elección de una red de conmutación de paquetes basada en una capa (capa de enlace) sin conexión que opera a través de distintas redes.

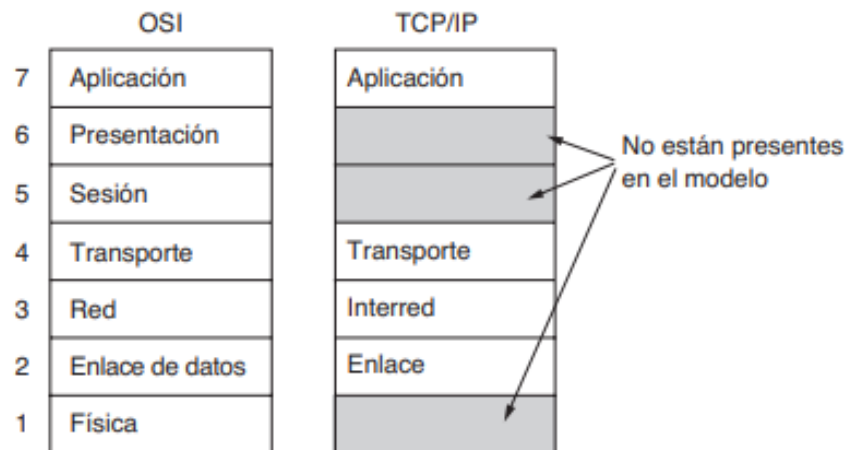


Figura 4: OSI vs. TCP/IP - Tanenbaum.

En contraste con el modelo de referencia OSI, este modelo cuenta con solamente cuatro capas:

1. Capa de enlace

Es la capa mas baja del modelo TCP/IP. Describe qué enlaces se deben llevar a cabo para cumplir con las necesidades esta capa de interred sin conexión. Es más una interfaz entre los hosts y los enlaces de transmisión, que una capa propiamente dicha.

2. Ccapa de interred

Esta es la capa más importante de la arquitectura, ya que la mantiene unida. Se puede comparar con la capa de red de OSI y su trabajo es permitir a los hosts que envíen paquetes en cualquier red y que estos viajen de manera independiente. No es responsabilidad de esta capa (sino que de las capas superiores) que estos paquetes lleguen en el mismo orden en el cual se los envió. Las unidades de información con los que trabaja son paquetes y el protocolo oficial se conoce como **IP** (*Internet Protocol*). También cuenta con un protocolo complementario que se encarga de transmitir mensajes de error, conocido como protocolo **ICMP** (*Internet Control Message Protocol*), y un protocolo que mapea direcciones IP en direcciones físicas, conocido como **ARP** (*Adress Resolution Protocol*). La principal tarea de la capa es enviar los paquetes IP a donde deben ir, mediante el ruteo de paquetes.

3. Capa de transporte

Diseñada para permitir que las entidades pares lleven a cabo una correcta comunica-

ción. Esto implica utilizar protocolos de extremo a extremo. Existen dos protocolos ampliamente utilizados para esta tarea:

- **TCP** (*Transmission Control Protocol*): es un protocolo que ofrece un servicio de entrega de flujo confiable y orientado a la conexión. Permite que un flujo de bytes sea entregado sin errores desde una máquina fuente a cualquier máquina destino de la interred. El protocolo suele fragmentar dicho flujo en mensajes discretos que se envían por la red de redes y se reensablan a la hora de recibirlos. Además cuenta con un control de flujo para evitar la inundación de un receptor lento con más mensajes de los que pueda manejar.
- **UDP** (*User Datagram Protocol*): es un protocolo de entrega de paquetes sin conexión y no confiable. Se suele utilizar para consultas petición-respuesta y en las aplicaciones en donde la entrega oportuna es mas valiosa que una entrega precisa, como en transmisiones de voz o vídeo.

4. Capa de aplicación

Este modelo, a diferencia del modelo OSI no cuenta con capas de sesión o de presentación, las aplicaciones simplemente incluyen las funciones de estas capas que consideren necesarias. La capa de aplicación contiene todos los protocolos de alto nivel, como los de transferencia de archivos (FTP) y correo electrónico (SMTP).

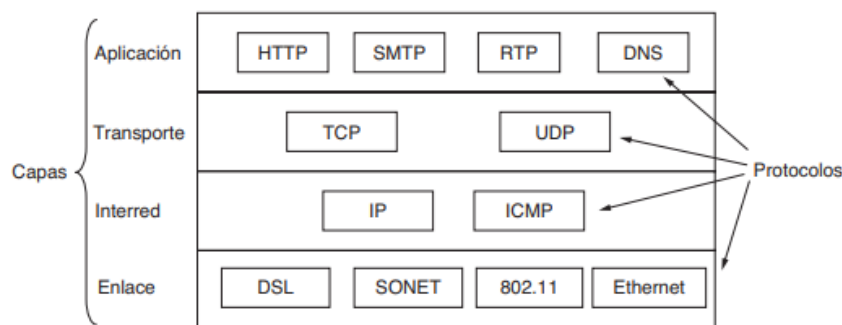


Figura 5: Protocolos y Capas del Modelo TCP/IP - [Tanenbaum](#)

Bibliografía usada: Capítulo 1 de [Tanenbaum](#) y capítulos 1 a 3 de [Comer](#).

2. Capa Física

En esta unidad nos centraremos en la capa mas baja del modelo OSI, la capa física. Recorreremos distintos temas pertinentes a esta capa y sus protocolos, como el análisis teórico detrás de la transmisión de información y los distintos medios físicos que contamos para lograr esta tarea.

2.1. Análisis Teórico

Mediante la variación de alguna propiedad física, como el voltaje o la corriente, es posible transmitir información a través de cables. Este valor puede representarse como una función $f(t)$ que permite modelar el comportamiento de la señal y analizarlo matemáticamente.

2.1.1. Análisis de Fourier

El matemático Jean-Baptise Fourier demostró que cualquier función periódica $g(t)$ con un período T puede construirse como la suma (posiblemente infinita) de senos y cosenos. Tal que:

$$g(t) = \frac{1}{2}c + \sum_{n=1}^{\infty} a_n \text{sen}(2\pi nft) + \sum_{n=1}^{\infty} b_n \text{cos}(2\pi nft) \quad (1)$$

en donde $f = 1/T$ es la frecuencia fundamental, a_n y b_n son las amplitudes del seno y coseno del n -ésimo armónico y c es una constante. A esta descomposición se la conoce como **serie de Fourier**. Lo importante de esto es que si se conoce el periodo T y las amplitudes, se puede encontrar la función original del tiempo.

¿Cómo se relaciona esto con la comunicación?

Al enviar información por algún medio, se arriesga a que la señal transmitida sufra pérdidas de potencia en el proceso. Si todos los componentes de la serie de Fourier disminuyeran en la misma proporción, no habría problema, solamente habría una pérdida de amplitud. Sin embargo, los distintos medios utilizados reducen los componentes en distintas medidas, provocando **distorsión**.

No todos los canales son iguales, cuentan con distintas propiedades. Una de ellas refiere a la capacidad de transmitir información sin ninguna atenuación relevante, es decir, un rango de frecuencias en el cual la comunicación será más segura. Esta propiedad se conoce como **Ancho de banda** (analógico) y es una **propiedad física**. Normalmente se mide desde el cero hasta la frecuencia en la cual la potencia que se recibe se reduce a la mitad. Usualmente se utilizan filtros para limitar el ancho de banda de una señal, esto permite que varias señales compartan una región dada del espectro. Por ejemplo, una señal que deba ser transmitida por un cable que pueden utilizar 20MHz deberá filtrar el ancho de

banda de la señal con ese tamaño. Las señales que van desde cero hasta una frecuencia máxima se conocen como señales de **banda base**, y las que se desplazan para ocupar un rango de frecuencias mas altas se conocen como señales de **pasa-banda**.

Existe otra definición de **ancho de banda** que es el ancho de banda digital, este refiere a la tasa de bits máxima que puede transmitirse por un medio, y se mide en bits/segundo. Esta definición es interesante, vimos que el ancho de banda analógico refiere a una propiedad física del medio y esto es lo que limita el rango de frecuencias sin atenuación, pero entonces, ¿qué nos limita la tasa máxima de datos en un canal? Es decir, ¿Qué define al ancho de banda digital?

2.1.2. Teorema de Nyquist y Teorema de Shannon

El ingeniero Henry Nyquist descubrió que incluso un canal perfecto tiene una capacidad de transmisión finita. Esta tasa de datos máxima para un canal sin ruido con un ancho de banda finito puede expresarse en una ecuación. Nyquist demostró que si se pasa una señal a través de un filtro pasa-bajas con ancho de banda B , hacen falta solamente $2B$ muestreos por segundo de la señal para reconstruir la señal original. Si la señal consiste en V niveles discretos, entonces:

$$\text{Tasa de datos Máxima} = 2B \log_2 V \text{ bits/seg} \quad (2)$$

Esta ecuación solamente sirve en un canal ideal en el cual no hay ruido presente, pero cuando hay ruido (siempre lo hay), la situación empeora con rapidez. Normalmente el ruido presente es el ruido térmico, que se genera debido al movimiento de las moléculas del sistema. Este ruido puede medirse en base a la relación entre la potencia de la señal y la potencia del ruido. A este coeficiente se lo conoce como **SNR** (*Signal-to-Noise Ratio*). Si la potencia de la señal es S , y la potencia del ruido es N , luego el coeficiente SNR , que se mide en decibeles (**dB**), se expresa en una escala de logaritmo como:

$$SNR = 10 \log_{10}(S/N) \text{ dB}.$$

Claude Shannon amplió el trabajo de Nyquist y agregó la noción del ruido aleatorio a su ecuación, tal que la tasa máxima de datos de un canal ruidoso, cuyo ancho de banda es B Hz y la relación señal a ruido es S/N , luego:

$$\text{Número máximo de bits/seg} = B \log_2(1 + S/N) \quad (3)$$

2.2. Medios de Transmisión Guiados

Dijimos que el propósito de la capa física era transportar los bits en su manera mas pura de una máquina a otra. Existen obviamente varios medios físicos, cada uno con su propio ancho de banda, retardo, costo, etc. Los medios en sí se dividen en medios guiados y medios no guiados. Ahora nos concentremos en los **medios guiados**.

Algunos términos generales que se comparten en la mayoría de los medios y refieren a los enlaces son los de **Full-dúplex**, **Half-dúplex** y **Simplex**. Estos hablan de las direcciones en las cuales se puede transmitir dichos enlaces, y son en ambos sentidos al mismo tiempo, en ambos sentidos pero una sola dirección a la vez y en una única dirección, respectivamente.

2.2.1. Médios magnéticos

Este medio se separa del resto ya que consiste en la manera mas básica de transmitir información: Almacenarlo en una cinta magnética (o disco, pen-drive, etc). Es claro ver que estos dispositivos suelen tener un bajo costo con respecto a las grandes cantidades de información que pueden llevar, por lo que nadie puede negar que el ancho de banda de este medio es enorme, y su relación de costo por bit transmitido es espectacular. Sin embargo, para llevar un pen a otra máquina, usualmente el tiempo se mide en horas o minutos, resultando una velocidad muy pobre.

2.2.2. Par trenzado

Este medio es uno de los más antiguos y más comunes. Consiste en un par de cables de cobre aislados que se trenzan de forma helicoidal. El trenzado se realiza ya que dos cables en paralelo forman una antena simple, y al trenzarlos las ondas se cancelan y el cable irradia menos. Esto mejora la inmunidad del medio al ruido externo.

Se pueden extender por varios kilómetros sin necesidad de ampliación, pero cuando la señal se atenúa demasiado se requieren repetidores. El ancho de banda depende del grosor del cable y la distancia que recorre (llegando a varios megabits/segundo. Los pares trenzados se utilizan mucho debido a su adecuado desempeño y bajo costo.

2.2.3. Cable coaxial

El cable coaxial (coax) es otro medio de transmisión común. Cuenta con mayor blindaje y mayor ancho de banda que los pares trenzados, por lo que abarca mayores distancias a velocidades mas altas. Se puede utilizar tanto para transmisión digital como analógica. Se constituye de un alambre de cobre rígido cubierto por un material aislante. Esta protección

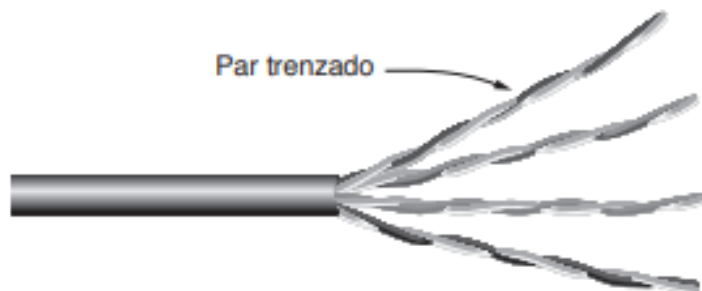


Figura 6: Cable con 4 pares trenzados - Tanenbaum

le permite tener una buena inmunidad al ruido. Los cables coaxiales suelen utilizarse muy seguido para las señales de televisión por cable y las MAN.

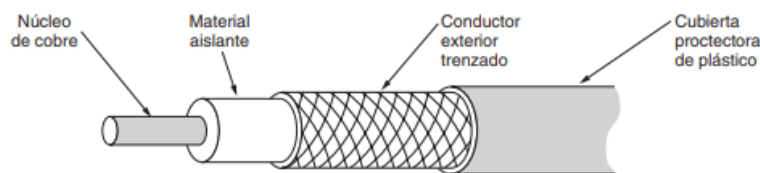


Figura 7: Cable coaxial - Tanenbaum

2.2.4. Líneas eléctricas

Este medio no es tan común y todavía está en desarrollo. La idea es aprovecharse de los cableados eléctricos dentro de las casas (diseñados para transportar energía) y utilizarlas para enviar señales de datos. Las propiedades de los cables suelen cambiar de una instalación a otra y los electrodomésticos conectados suelen crear ruido eléctrico cuando se encienden, causando dificultades a la hora de utilizar este medio. Sin embargo, se es práctico enviar por lo menos 100 Mbps por este medio.

2.2.5. Fibra óptica

La fibra óptica es la *crème de la crème* de los medios guiados, se utiliza para la transmisión de larga distancias en las redes troncales, las redes LAN para el hogar y el acceso a Internet de alta velocidad. En términos teóricos, el ancho de banda que se puede lograr con fibra óptica mayor a los 50 TBps (Sí, 50 **Terabytes** por segundo), aunque realmente el límite práctico está más cerca de los 100 Gbps. Esto se debe mayormente a la incapacidad de convertir señales ópticas a señales eléctricas (y viceversa) con rapidez.

Un sistema óptico está compuesto principalmente por 3 componentes clave: una fuente de

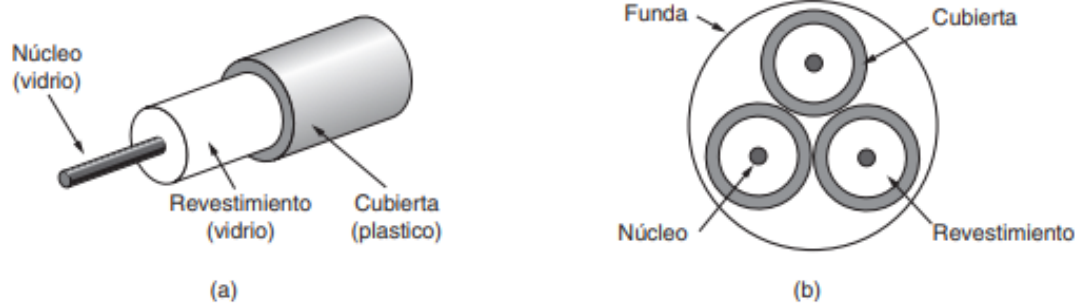


Figura 8: Cable de fibra óptica (a), Funda con tres fibras (b) - Tanenbaum

luz, un medio de transmisión y un detector. Para la fuente de luz se suelen utilizar LEDs y por convención, un pulso de luz significa un 1 y la ausencia de luz un 0. En cuanto al medio de transmisión se utiliza justamente una fibra de vidrio ultra delgada. El detector genera un pulso eléctrico cuando la luz incide en él.

Los rayos de luz que viaja por una fibra óptica se emiten tal que incidan en las paredes de la fibra con un cierto ángulo, produciendo que al reflejarse se mantengan dentro de la fibra. Esto evita que hayan fugas de luz y por lo tanto pérdida de información. El ángulo con el que inciden los rayos se conoce como **modo**, una fibra con rayos de distintos modos se conoce como **fibra multimodal**. Si se reduce el diámetro de la fibra a unas cuantas longitudes de onda se obtiene una especie de “guía” para las onda y los rayos se propagan en línea recta, esta fibra se conoce como **fibra monomodo**, y son las más costosas (se utilizan para largas distancias).

Los cables de fibra óptica se componen del núcleo (la fibra per se), un revestimiento de vidrio y una cubierta de plástico. Se suelen agrupar en fundas. El problema principal con las fibras ópticas viene al momento de empalmar dos cables. Como se necesita que la luz se transmita con la menor pérdida posible, estos empalmes deben ser lo más precisos posibles. Hay tres maneras de empalmar dos cables. La primera es utilizar conectores en los extremos de los cables y encastrar ambos cables mediante estos dispositivos. Este método produce entre un 10 % y un 20 % de pérdida de luz, pero agiliza el proceso. El segundo método se trata de un empalme mecánico, simplemente se ubican ambos cables de la manera más alineada posible, se pierde un aproximado de 10 % de luz. El tercer y último método consiste en fusionar dos piezas de fibra, fundiéndolas y uniéndolas. Es casi tan bueno como una sola fibra.

Por último, analizaremos las diferencias entre la fibra óptica y los demás medios de cobre. De entrada, la fibra puede manejar anchos de banda mucho mayores, como tiene una baja atenuación, necesita repetidores cada 50km en contraste con el cobre que los necesita cada 5km aproximadamente. Al ser un haz de luz, no le afectan las sobrecargas de

energía, las interferencias electromagnéticas ni los cortes de energía. Suele ser muchísimo mas liviano que el cobre, y al no tener fugas de luz y ser difíciles de intervenir, son bastante seguras contra el espionaje. Sin embargo, son bastante frágiles y como la comunicación es unidireccional normalmente, para una comunicación en dos sentidos se deben utilizar por lo menos dos fibras.

2.3. Medios de Transmisión Inalámbricos

2.3.1. Protocolo IEEE 802.11

Las redes inalámbricas WLAN (*Wireless Local Area Network*) actuales están generalmente regidas por el protocolo **IEEE 802.11**, o mejor conocido como **WI-FI**. Este protocolo define el uso de las capas físicas y de enlace, especificando las normas de funcionamiento. El modelo estandar del protocolo data del año 1997. Cuando comenzó, el **802.11** manejaba velocidades de transmisión de entre 1 y 2 Mbps como mucho y trabajaba en la banda de frecuencia de 2,4 Ghz. Claramente el protocolo evolucionó con el paso de los años y mutó en distintas versiones, como las versiones **802.11a** y **802.11b**. En 2009 se llegó a la versión **802.11n**, que amplió la velocidad hasta los 54Mbps. Hoy en día, se trabaja con dispositivos basados en el protocolo **802.11ac**, que no solo permite velocidades más altas sino que también es menos sensible a ruidos y obstáculos físicos. Esto lo veremos en mas detalle un poco más adelante.

2.3.2. Arquitectura de una red WLAN 802.11

Una red local inalámbrica que sigue el protocolo 802.11 se basa en una **arquitectura celular**, donde el sistema se subdivide en celdas, llamadas **BSS** (*Basic Service Set*) y cada una de ellas se controla mediante una estación base **AP** (*Access Point*). Una red celular puede estar compuesta por una única celda, con un único AP, o componerse de varias celdas con varios AP conectados entre sí mediante un enlace troncal (comunmente Ethernet). Este sistema en su conjunto se conoce como un **Sistema de Distribución** (DS, *Distribution System*).

2.3.3. Estándar 802.11ac MU-MIMO

Como dijimos antes, actualmente se utiliza la versión del protocolo estándar **802.11ac**. Este nuevo estándar opera sobre el espectro de 5 GHz, en contraste con sus predecesores que trabajaban con 2,4 GHz. ¿Qué cambia esto? Básicamente este espectro otorga menos sensibilidad frente a los obstáculos físicos, permitiendo una mejor transmisión de la señal en ámbitos locales. Además, la mayoría de los electrodomésticos domésticos, como

teléfonos inalámbricos o microondas operan en el rango de 2,4 GHz, causando ruido y degradando la señal en este rango. Es claro que las señales del protocolo **802.11ac** no se verán afectadas por esto. Sin embargo, todo lo bueno tiene algo malo (algo medio Yin-Yang) y en este caso, las señales del rango 5 GHz suelen estar más restringidas en la distancia de transmisión. Además, no son compatibles con la banda de 2,4 GHz.

Perfecto, pero ¿Qué es MU-MIMO?

2.3.4. MU-MIMO

MU-MIMO, o *Multi User - Multiple Input Multiple Output* es una técnica que permite aprovechar las múltiples antenas de transmisión y recepción de los AP para generar subcanales equivalentes, paralelos e independientes por los cuales se logra enviar o recibir múltiples cadenas de datos simultáneamente. MIMO se vale de dos técnicas para reducir la relación señal ruido de la transmisión. Estas son: BeamForming, y Multiplexación espacial con aprovechamiento del dominio espacial. La primera permite manejar la señal de radiofrecuencia de un punto de acceso que utiliza múltiples antenas para transmitir la misma señal y analizar el feedback de los clientes. De esta manera se logra ajustar las señales enviadas y determinar el mejor camino. La segunda nos permite obtener altas tasas de transferencia.

2.3.5. Técnicas de ensanchamiento de espectro

El estándar **IEEE 802.11** también se vale de algunas estrategias para “ensanchar” el espectro de una señal. Es decir, esparcir la señal a transmitir por un ancho de banda más grande que el ancho de banda mínimo requerido para la transmisión de esa señal. La razón de esto es poder reducir la interferencia de otras señales. Las estrategias que utiliza el protocolo son **DSSS** y **FHSS** (son siglas de nombres, no onomatopeyas).

2.3.6. Frequency-Hopping Spread Spectrum (FHSS)

Este método consiste en transmitir una parte de la información por determinada frecuencia durante un periodo de tiempo arbitrario (conocido como **dwell time**). Una vez pase este intervalo, se cambia la frecuencia de emisión y se transmite por otra frecuencia. Cabe destacar que el orden de los saltos está dado por una secuencia pseudo aleatoria que el emisor y el receptor acuerdan previamente.

Un detalle es que aunque se da un cambio de frecuencia, o sea, de canal físico, a nivel lógico, el canal se mantiene (ya que el enlace es único).

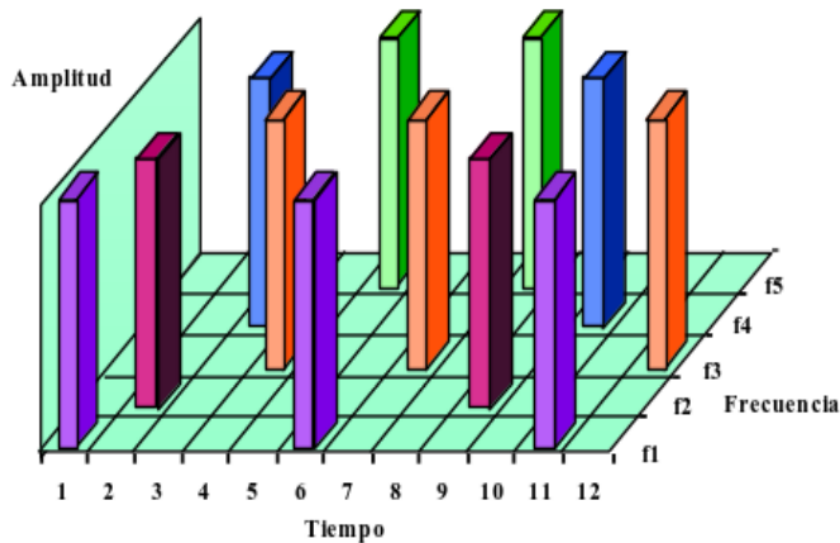


Figura 9: Espectro Disperso por Salto de Frecuencia (FHSS) - [Apunte Redes inalámbricas](#)

2.3.7. Direct-Sequence Spread Spectrum (DSS)

Esta técnica consiste en modular la señal a transmitir con una secuencia de bits de alta velocidad conocida como **secuencia de Baker**. Estos bits se llaman “chips” y se suelen utilizar entre 11 y 100 en una modulación. La ventaja que trae este método no es un aumento de potencia o ancho de banda, sino una mayor resistencia al ruido, ya que la interferencia solo afectará a unos pocos bits de la secuencia original.

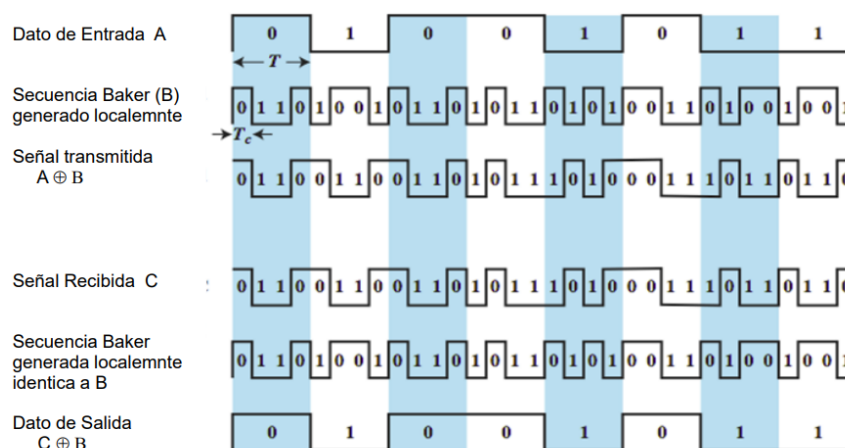


Figura 10: Dispersión de espectro con secuencia directa (DSS), Modulación y Recuperación de datos - [Apunte Redes Inalámbricas](#)

2.4. Topologías de redes

Como último detalle sobre esta unidad, con Mateo queríamos hablar sobre topologías de redes. Este tema se refiere a la manera en la que se pueden disponer los distintos hosts y los enlaces entre ellos. Uno pensaría que esto es meramente estético o no le daría gran importancia, sin embargo, una mala configuración de red puede llevar a problemas de congestión, costos y hasta fallos en la comunicación.

La mayoría de redes se pueden expresar como grafos, donde los vértices representarían a los distintos hosts y una arista conectaría a dos hosts si estos están conectados físicamente. Analizaremos 4 formas de distribuir redes y veremos las ventajas y desventajas que estas presentan.

■ Topología de Malla

En este tipo de configuración, todos los hosts de la red están conectados físicamente entre sí. Como grafo, esto se puede pensar como un grafo completo. Si tenemos una red malla con n hosts, entonces no es difícil ver que tendremos $\frac{n(n-1)}{2}$ enlaces en total. La ventaja que provee esta distribución es que en el caso que un host se caiga o un enlace falle, una trama de datos siempre podrá encontrar un camino, ya sea directamente o a través de algún host intermedio. Sin embargo, agregar un nodo en esta topología se vuelve muy costoso ya que implica agregar n enlaces físicos nuevos. Por lo general, en la práctica, la malla es un anillo de subredes donde cada una tiene (probablemente) forma de malla completamente conectada.

■ Topología de Estrella

Esta distribución se vale de un host central el cual está enlazado a todos los otros hosts, formando una especie de estrella. Es claro que una red estrella de n hosts cuenta con $n - 1$ enlaces totales. Esta topología permite agregar o quitar hosts fácilmente (menos el host central), ya que cada uno cuenta con un solo enlace. Sin embargo, la dependencia de un host central para la comunicación entre hosts causa que si por alguna razón ese host falla, la red se caerá en su totalidad. Sin ir a un ejemplo tan extremo, el solo hecho de que todas las comunicaciones pasen por el nodo central causa que si hay mucho tráfico, la red se congestione fácilmente.

■ Topología de Anillo

La topología de Anillo plantea que los hosts formen un ciclo de enlaces, contando solamente con dos enlaces cada uno. Esta configuración implica que en un red de n hosts haya n enlaces. Estos enlaces suelen ser bidireccionales. Si algún host o enlace se cae, la bidireccionalidad permite mantener la conexión entre los hosts. La tarea de agregar un host a una red anillo requiere cortar un enlace momentáneamente para agregar el nuevo nodo y conectarlo a los hosts que fueron desconectados.

■ Topología de Bus

Esta forma de disponer los hosts se basa en la existencia de un enlace troncal (bus) al que todos los hosts se conectan, y los paquetes fluyen por este enlace hacia su destino. Se dice que la red tiene solo un enlace. Fácilmente, se puede deducir que la presencia de un único enlace transportador combinado con un alto tráfico de datos puede generar congestión en la red de una manera muy fácil. Otra observación puede ser que si este enlace se llegara a cortar, se corta la comunicación entre los hosts. Sin embargo, se generan subredes en las que los hosts conectados a una de las porciones que quedan del enlace troncal pueden seguir comunicándose entre sí.

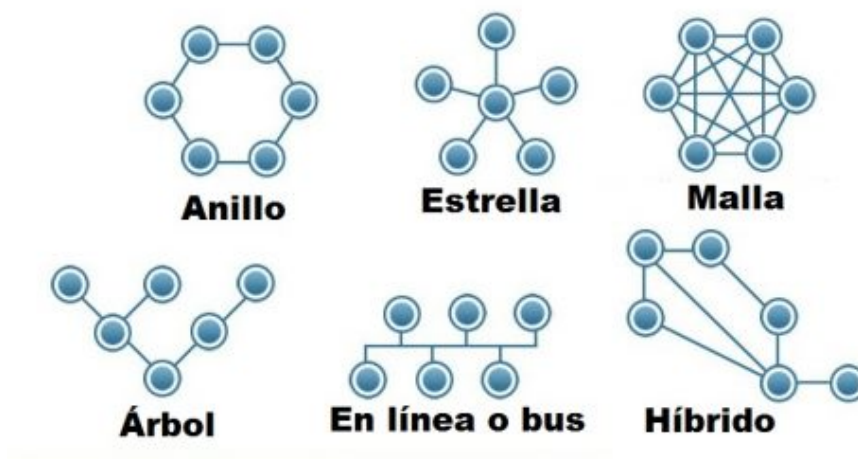


Figura 11: Distintas topologías de redes.

Bibliografía usada: Capítulo 2 de [Tanenbaum](#) y [Apunte Redes Inalámbricas](#).

3. Capa de Enlace

Esta capa, como se mencionó anteriormente, se encarga de procesar los mensajes que recibe de su capa superior (la capa de red) en unidades (tramas) preparadas para ser transmitidas por la capa física. Este proceso no es para nada trivial, ya que se debe tener en cuenta que los enlaces y los canales de transmisión no son perfectos y se cometen errores. Además, la tasa de datos es finita y existe un retardo entre que se envía un bit y el momento en el que se recibe. Otro trabajo de la capa de enlace es controlar el acceso al medio y regular el flujo de datos. Podemos dividir la capa de enlace en dos subcapas, una encima de la otra. La superior es la subcapa **LLC** (*Link Logical Control*), que actúa como una interfaz entre la capa de red y la subcapa inferior, conocida como la subcapa **MAC** (*Medium Access Control*).

3.1. Subcapa LLC

La subcapa LLC debe brindar, a la capa de red, un medio de comunicación seguro, y para esto le brinda servicios de transferencia de datos. La capa puede diseñarse para ofrecer distintos servicios, los más comunes son:

- **Servicio sin conexión ni confirmación de recepción:** Consiste en hacer que la máquina de origen envíe tramas independientes a la máquina destino, sin que esta confirme la recepción. Si se pierde alguna trama por ruido en la línea, la capa de enlace no hace ningún esfuerzo por detectar o recuperar la pérdida. Este servicio es apropiado cuando la tasa de error es muy baja.
- **Servicio sin conexión con confirmación de recepción:** En este servicio se debe confirmar de manera individual la recepción de cada trama recibida. De este modo el emisor sabe si la trama llegó o se perdió en el camino. Este servicio es útil en canales no confiables.
- **Servicio orientado a la conexión con confirmación de recepción:** Este es el servicio más sofisticado que la capa de enlace ofrece a la capa de red. Con este servicio, las máquinas de origen y de destino establecen una conexión antes de transferir datos. Cada trama enviada a través de la conexión está numerada y la capa de enlace garantiza que cada trama llegará a su destino exactamente una sola vez y en el orden correcto. Es apropiado para enlaces largos y poco confiables.

3.1.1. Entramado

La comunicación se hace, en última instancia, mediante la capa física, que envía bits puros de una máquina a la otra. En un mundo ideal, los únicos bits que se envían son los bits del mensaje a transmitir. Sin embargo, no vivimos en ese mundo ideal, claramente, y los canales de comunicación son susceptibles al ruido, que puede cambiar bits (de 0's a 1's o viceversa) o directamente eliminarlos, lo que hace que el mensaje llegue alterado. La capa de enlace divide el flujo de bits en unidades llamadas **tramas**, y les agrega una **suma de verificación**, que se utiliza para que cuando la trama llegue, el receptor calcule la suma (mediante algún algoritmo en particular) y compare con el número recibido. Si la suma no coincide, la capa de enlace sabe que ha ocurrido un error y toma las medidas necesarias para manejarlo.

Ahora, la dificultad se encuentra en la correcta división del flujo de datos en distintas tramas. La idea está en poder balancear la facilidad del receptor en encontrar el inicio de las tramas y utilizar la menor cantidad de ancho de banda posible. Existen muchos métodos, nosotros nos concentraremos en 4:

1. Conteo de bytes.

Este método se basa en agregar un encabezado que especifique el número de bytes en la trama. De modo que se dedica un byte para esta información. Este método consigue utilizar muy poco ancho de banda adicional, sin embargo, puede suceder que el ruido en el canal afecte este byte, cambiándolo de valor, y por lo tanto, afectando la manera en la que el receptor interpretará las tramas.

2. Bytes bandera con relleno de bytes.

Este método se vale de un byte delimitador para lograr que el receptor se vuelva a sincronizar con el emisor luego de algún error. Este byte se conoce como **byte bandera** y se utiliza tanto al principio como al final de una trama. De este modo, si el receptor perdió la sincronización, sólo debe esperar a leer dos bytes banderas (el final de una trama y el principio de la siguiente) para ponerse en orden. Sin embargo, puede ocurrir que el propio byte bandera esté incluido en los datos, causando confusiones. Esto se resuelve agregando un **byte de escape** justo antes de cada byte bandera en los datos. Cuando el receptor recibe la trama, simplemente quita el byte de escape. Esta técnica se denomina **relleno de bytes**.

3. Bits bandera con relleno de bits.

De la misma manera que el relleno de bytes, el **relleno de bits** agrega una serie de bits delimitadores al principio y al final de cada trama. Normalmente se utiliza el patrón 01111110 o 0x7E. Cada vez que la capa encuentra cinco bits 1 consecutivos, agrega un bit 0 para evitar las ambigüedades. Este método resuelve la desventaja

del relleno de bytes, que es tener que utilizar longitudes de tramas en múltiplos de 8 bits.

4. Violaciones de codificación de la capa física.

Esta forma se aprovecha del hecho de que la capa física suele agregar bits redundantes para ayudar al receptor. Esto significa que algunas señales no ocurrirán en los datos regulares. Se pueden aprovechar estas señales reservadas para indicar el inicio y el fin de cada trama. Esto se conoce como **violaciones de código**, y tiene como ventaja que es bastante fácil encontrar el inicio y el fin de cada trama, ya que tienen sus señales reservadas, y por lo tanto no hay necesidad de rellenar los datos.

3.1.2. Control de errores

¡Listo! Ya nos aseguramos de como definir correctamente las tramas a partir del flujo de datos, nuestro trabajo esta terminado... ¿No?. No, todavía nos queda poder asegurar que las tramas lleguen en el orden apropiado a la capa de red de destino. Es verdad que para un servicio sin conexión ni confirmación este problema no tiene mucha relevancia, pero en servicios orientados a la conexión es de suma importancia.

Una primer idea podría ser que el receptor provea una retroalimentación al emisor, informando sobre lo que pasa del otro lado de la linea. Este protocolo exige que el receptor devuelva un mensaje de confirmación, conocidos como **tramas de control**, tanto positivos como negativos de cada trama que llegue. Si el emisor recibe una recepción positiva, sabe que todo va bien. Si recibe una negativa, hubo algún problema y retransmite la trama.

La complicación se encuentra en la posibilidad de que problemas en el hardware causen la desaparición de una trama en su totalidad. En este caso, el receptor nunca recibiría la trama y por lo tanto nunca enviaría una trama de control, por lo que el emisor quedará esperando indefinidamente. Del mismo modo, puede desaparecer la trama de control en sí, causando el mismo problema. La solución a esto es introducir temporizadores en la capa de enlace, tal que cuando un emisor envía una trama también inicia un temporizador, que se expirará luego de un intervalo lo suficientemente grande como para que el emisor pueda recibir la trama de control. En el caso de que no llegue trama de control y el temporizador expire, el emisor se alerta de un posible problema y reenvía la trama. Sin embargo, esto tampoco es completamente correcto ya que puede suceder que el receptor acepte dos veces (o más) la misma trama y la pase a la capa de red más de una vez. Esto normalmente se evita asignándole a cada trama un numero de secuencia, tal que se pueda distinguir entre las tramas originales y las retransmitidas.

3.1.3. Control de flujo

¡Ahora sí! Ya cubrimos todos los puntos débiles, ¿ya podemos terminar el tema? (No quiero escribir más)... ¡Casi! Sólo nos queda un detallito, qué hacer cuando un emisor desea transmitir tramas a velocidades mucho mayores de las que nuestro receptor puede recibirlas. Normalmente, si esto no se controla, el receptor se satura y deja de poder controlar todas las tramas que recibe, y por ende, se perderá información. El concepto de manejar estas situaciones se conoce como **control de flujo**. Se utilizan dos métodos comunes de control de flujo: el **control de flujo basado en retroalimentación**, donde el receptor envía información al emisor para autorizarle que envíe más datos, y el **control de flujo basado en tasa**, donde el protocolo tiene un mecanismo que limita la tasa en la que el emisor puede enviar datos. Este último método suele ser implementado en la capa de transporte, mientras que el control basado en retroalimentación se puede implementar en esta capa.

3.2. Subcapa MAC

Ahora nos adentramos más profundo en la capa de enlace de datos y llegamos a la última subcapa, la subcapa de control de acceso al medio (MAC, *Medium Access Control*). En las redes de difusión, el problema está en decidir qué máquina puede utilizar el canal en cada momento. Dejar este problema desatendido puede provocar errores en la comunicación, como **colisiones** entre las tramas, invalidándolas y por lo tanto perdiendo información. Usualmente, vemos este problema en las redes inalámbricas como las redes de radio, ya que el canal es de difusión por su propia naturaleza. Esta subcapa contiene todos los protocolos necesarios para lograr esta coordinación entre los hosts.

3.2.1. El problema de la estación oculta

Un problema que suele surgir en redes de difusión inalámbricas es el conocido problema de la estación oculta. Este problema surge cuando en una red, dos hosts están lo suficientemente alejados entre ellos como para no poder identificarse el uno al otro. En estos casos, cuando alguno de estos hosts escuche el medio, no podrá escuchar (ni siquiera si existe) al otro host fuera de rango. Esto puede provocar que al enviar una trama al **access point**, se produzca una colisión con alguna otra trama enviada por alguna estación oculta.

Para solucionar esto, se pensó en la técnica **RTS/CTS**, que mediante una serie de convenciones y tramas especiales (tramas RTS (*Request To Send*), tramas CTS (*Clear To Send*)) se logra una coordinación entre todas las estaciones de la red.

¿Como funciona la técnica?

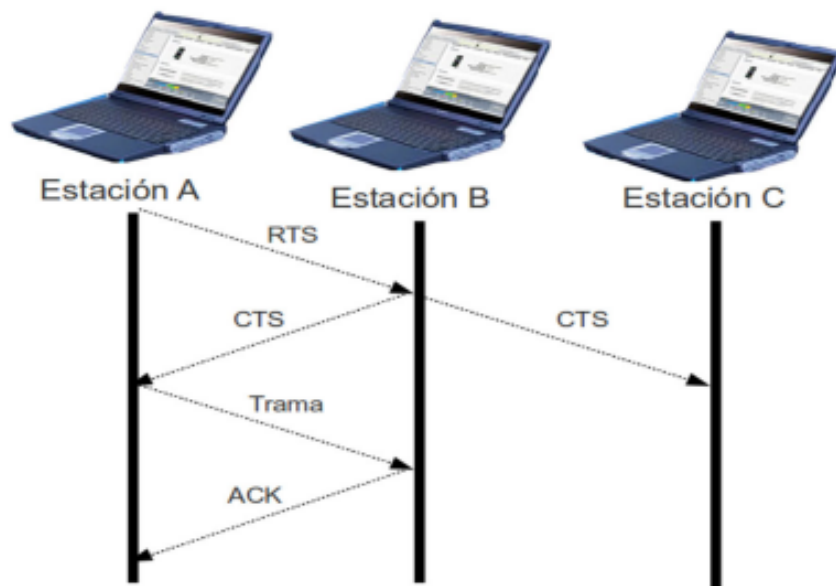


Figura 12: Solución al problema - [Apunte redes inalámbricas](#).

Supongamos que existen 3 estaciones: A, B (que es el *access point*) y C. Además, A y C se encuentran una fuera del rango de la otra, por lo que no podrán identificarse. Ahora, si A quiere comunicarse con B, antes de enviar su mensaje, envía una trama RTS, o sea, le avisa al *access point* que le quiere mandar un mensaje. Si B recibe este RTS, envía una trama CTS a todas las estaciones de la red (en este caso A Y C). Ahora tenemos dos casos distintos: Por un lado, A envió un RTS y recibió un CTS, que le habilita la comunicación, y acto seguido, envía su mensaje. Por otro lado, C recibió una trama CTS pero nunca envió su respectiva trama RTS, y por lo tanto se detiene y no transmite durante el tiempo que dure la transmisión. Este tiempo es un tiempo convenido por el **NAV** (*Network Asignation Vector*), que es un temporizador que indica la cantidad de tiempo que el medio será reservado. El tiempo del NAV estará especificado en la trama RTS, y se basa en el tamaño de la trama a transmitir. Luego de recibir la trama de datos, la estación B envía una trama de control ACK para avisarle a la estación A que el mensaje llegó correctamente.

Esta técnica soluciona correctamente este problema, pero genera una mayor sobrecarga en información de control.

Si observamos la figura 13 podemos entender cómo se realiza un esquema del proceso.

3.2.2. Espaciamiento intertrama

Las redes inalámbricas, en particular las que están implementadas sobre el protocolo estándar **IEEE 802.11** utilizan una técnica de espaciado entre las tramas enviadas para

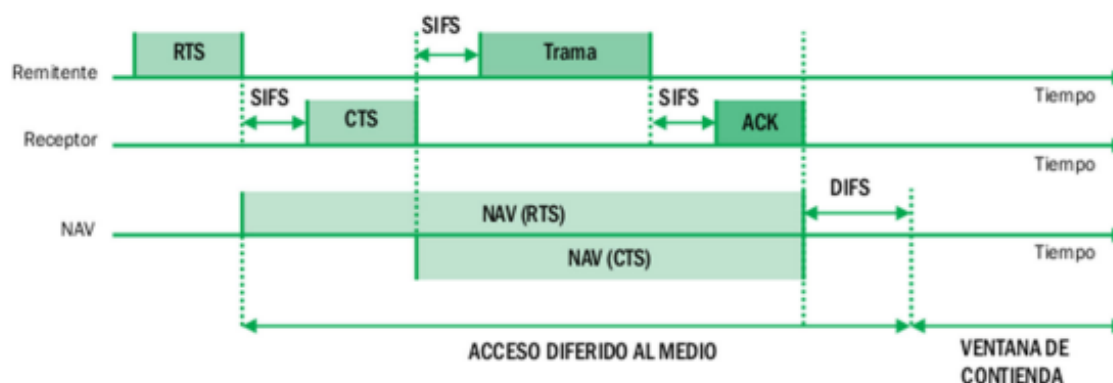


Figura 13: Esquema RTS/CTS - [Apunte redes inalámbricas](#).

poder distinguir las prioridades de cada trama. Si el lector prestó la suficiente atención, pudo notar que en la figura 13 hay algunas siglas que no llegamos a explicar (SIFS y DIFS). Estas siglas refieren justamente al espaciado intertrama. Ahora ¿Cuales son los distintos tiempos de intertramas? Los siguientes:

- **SIFS** (*Short InterFrame Space*): Se utiliza entre tramas de alta prioridad, como las tramas RTS/CTS y ACK.
- **PIFS** (*PFC InterFrame Space*): Es usado por la PFC (En unos minutos (o segundos según como leas) vas a saber qué es) durante una operación libre de contienda.
- **DIFS** (*DCF InterFrame Space*): Es el tiempo mínimo para servicios basados en contienda (como DCF, que también está explicado mas adelante) en el cual el medio tiene que estar libre para que una estación pueda acceder. Si el medio ha estado libre por mas tiempo que el DIFS, entonces la estación puede acceder inmediatamente.
- **EIFS** (*Extended InterFrame Space*): Es usado únicamente cuando ha ocurrido un error con alguna trama.

Ahora que sabemos esto, ¿Cómo interpretamos la figura 13?

No es difícil comprender que antes de cada trama RTS/CTS hay un espaciado SIFS, ya que son tramas de alta prioridad. Cuando la comunicación termina, hay un espaciado DIFS, marcando que el medio ahora está libre y las estaciones pueden competir por él.

3.2.3. Funciones de coordinación DCF y PCF

Para lograr su cometido, la subcapa MAC cuenta con dos funciones de coordinación: la **DCF** (*Distributed Coordination Function*) y la **PCF** (*Punctual Coordination Function*). La DCF es el mecanismo de acceso básico del estándar **CSMA/CA** (*Carrier Sense*

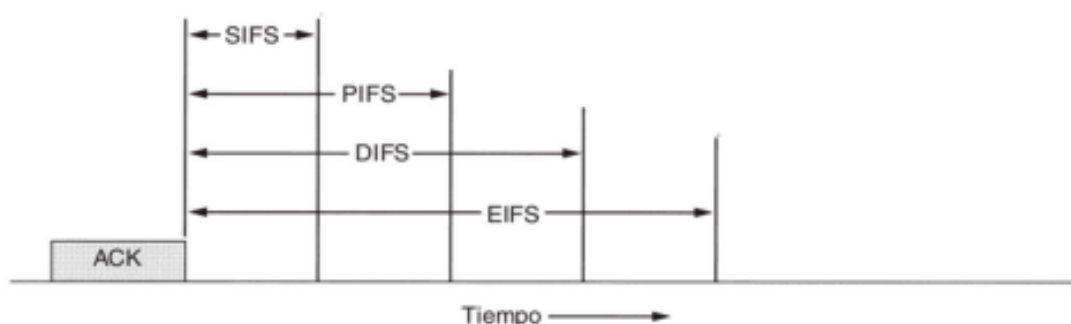


Figura 14: Espaciado Intertrama 802.11 - [Apunte redes inalámbricas](#)

Multiple Access with Collision Detection) donde primero se verifica que el enlace de radio se encuentre limpio para transmitir. En ese momento, para evitar colisiones se retrasa la emisión de tramas y se escucha una vez más el canal para poder transmitir. La DCF puede utilizar la técnica de **RTS/CTS** para reducir la posibilidad de colisiones. Se dice que este tipo de acceso es aleatorio, y por lo tanto es un sistema de contienda, donde los hosts compiten por el canal.

Por otro lado, la PCF está asociada a las transmisiones libres de contienda, donde se utilizan técnicas de acceso deterministas para decidir qué host puede acceder al medio. Se suele utilizar en servicios de tipo síncrono que no soportan un retraso aleatorio en la emisión de las tramas.

3.3. Tipos de trama en el IEEE 802.11

Para cerrar de la mejor manera y no terminar con un mal sabor de boca, ¡vamos a hablar de los tipos de trama y su estructura en el protocolo 802.11! Existen 3 tipos distintos de tramas con las que trabaja la capa de enlace de datos:

- **Trama de datos:** Es la unidad de datos y es usada para la transmisión de datos (duh).
- **Trama de control:** Son las tramas dedicadas a mantener el control de acceso al medio. Son, por ejemplo, las tramas RTS/CTS y las ACK.
- **Tramas de gestión:** Son transmitidas de la misma manera que las tramas de datos, pero solamente llevan información de administración y por lo tanto, no son transmitidas a capas superiores.

Una trama de datos 802.11 se ve, mas o menos, como en la figura 15.

Podemos distinguir algunos campos en particular: El control de trama, el de duración,

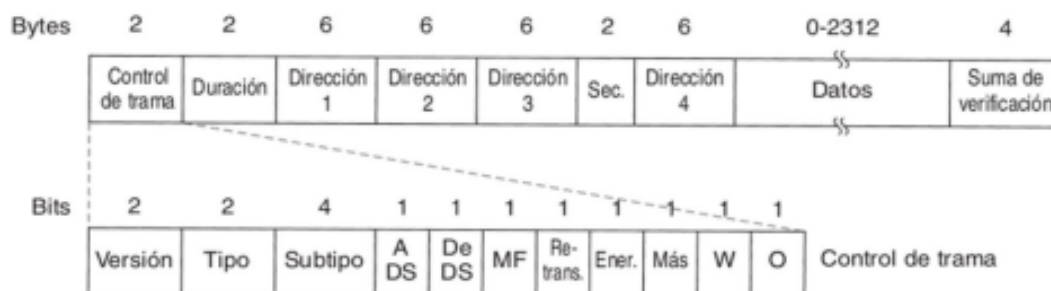


Figura 15: Trama de datos 802.11 - [Apunte Redes Inalámbricas](#)

el de direcciones, el control de secuencias y la suma de verificación. La figura también amplía sobre el campo de control de trama, el cual se compone de 2 bytes y guarda información importante para la trama. Contiene información de la versión del protocolo, el tipo de trama, el subtipo (RTS sería un subtipo del tipo trama de control), y mas información específica. El campo de duración provee información sobre el tiempo estimado de transmisión de la trama, es clave para el NAV. Para más detalles se puede referir directamente al [Apunte de redes inalámbricas](#).

Bibliografía usada: Capítulo 3 y 4 de [Tanenbaum](#) y [Apunte Redes Inalámbricas](#)

4. Capa de Red

La capa de red, como mencionamos, es la encargada de mantener una integridad en cuanto a direcciones y rutas de los paquetes que viajan por las redes. Se utilizan dos protocolos para lograr esta organización: El protocolo **IP** y el protocolo **ICMP**. Estos son dos caras de la misma moneda y se complementan el uno al otro, permitiendo la correcta comunicación entre hosts. El origen de estos protocolos se remonta a los años 80, lo que significa que fueron mutando y adaptándose a medida que el tiempo pasaba. En particular, veremos 2 versiones del protocolo IP, el protocolo **IPv4** y el protocolo **IPv6** (Como los números lo indican, la versión 6 es más moderna que la versión 4).

4.1. ARP

4.1.1. Direcciones Físicas

Todos escuchamos alguna vez hablar de direcciones IP esto, direcciones IP aquello. Es claro que las direcciones IP nos permiten encontrar una máquina a través del enorme océano del internet. Pero las direcciones IP son parte del protocolo IP, esto quiere decir que solamente viven dentro de la capa de red del modelo de referencia. ¿Qué pasa cuando analizamos una transmisión desde una capa inferior? Claramente, estas direcciones dejan de existir.

A nivel físico, la comunicación, o mejor dicho, la identificación de un host se indica mediante una **dirección física** (conocida como **dirección mac**) única para cada dispositivo de red.

Entonces, ¿Para que sirven las direcciones IP? Simplemente brindan una capa de abstracción sobre las direcciones físicas, desvinculando el ruteo de paquetes del dispositivo físico en sí. Esto quiere decir que al trabajar con redes nos concentraremos solamente en las direcciones que le asignemos a cada host considerando el protocolo IP y nada más.

4.1.2. Problema de resolución de direcciones

Pero... Si tenemos una dirección de red que no se corresponde con la dirección física del host, ¿cómo hacemos para decirle a la capa física a qué dirección física debe enviar la trama? Es claro que debemos, desde la capa de red, conseguir la dirección física del host destino desde la dirección IP. Esto se conoce como **mapeo de direcciones** (*adress mapping*). Este mapeo se debe hacer en todos los pasos del camino entre el host emisor y el receptor, ya que todos los dispositivos involucrados (hosts, y routers) son dispositivos físicos y tienen su propia dirección física.

El problema del mapeo de direcciones de alto nivel a direcciones físicas se conoce como **problema de resolución de direcciones** (*address resolution problem*) y tiene varias formas de ser solucionado.

Primero y principal, nos interesan las direcciones físicas (obviamente). Existen dos tipos de direcciones físicas: Las direcciones Ethernet, que son de 48 bits y las direcciones proNET, que son mas chicas. Lo único que nos interesa de esto es ver distintas formas de resolver las direcciones. Si una dirección es lo suficientemente chica, como la de proNET, es posible embeber la dirección en la misma dirección IP. Como las direcciones IPv4 tienen solamente 32 bits, es claro que no podemos embeber una dirección Ethernet (en la versión IPv6, que cuenta con 128 bits, sí es posible), por lo que se implementa un **mapeo dinámico**.

4.1.3. Resolución mediante mapeo dinámico

El mapeo dinámico permite hacer una completa separación entre la dirección IP y la dirección física, y mantiene una relación entre ambas mediante un sistema de mensajes en el cual los dispositivos estarán constantemente intercambiando información sobre su dirección física. Esto es útil cuando, por ejemplo, se debe reemplazar un dispositivo, permitiendo que, aunque la dirección física cambie, la dirección IP se mantenga.

Este método se vale de la capacidad de *broadcast* para permitir a los hosts y routers conseguir las direcciones físicas de los otros dispositivos en la red sin tener que mantener una tabla o depender de una base de datos centralizada. El protocolo de bajo nivel que se encarga de este trabajo se conoce como **ARP** (*Address Resolution Protocol*).

4.1.4. ARP

¿Como resuelve el problema el protocolo ARP?

Propone un mecanismo tal que, cuando un host A quiera conseguir la dirección física de un host B (Siempre que quiera enviar un paquete a B. Bueno, no siempre, pero eso lo vemos mas adelante) deberá hacer uso del broadcast para difundir en la red un paquete ARP. Este paquete se encapsula en una trama física y contiene información sobre las direcciones físicas e IP del emisor y la dirección IP del host destino. Como es una difusión, todos los host de la red recibirán el paquete, pero solo uno (B) tendrá la misma dirección IP que la especificada en el paquete ARP. El host coincidente entonces responde al host A con un paquete que contenga su dirección física. Recién ahora el host A puede enviar un paquete al host B.

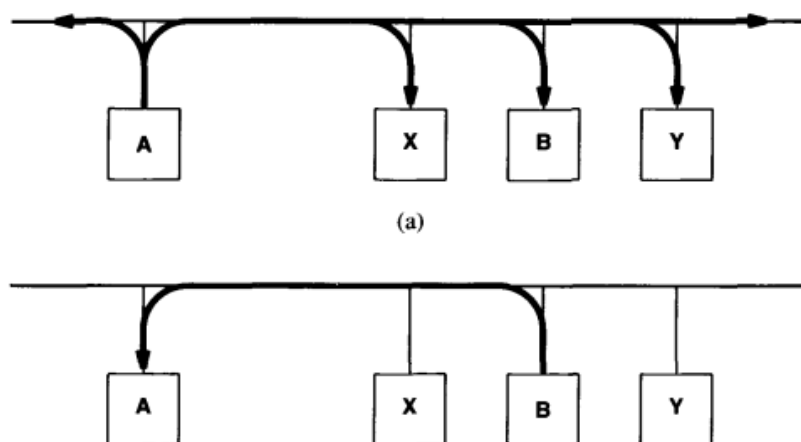


Figura 16: Mecanismo del protocolo ARP - Comer

4.1.5. Caché de ARP

En la práctica, realizar un broadcast es una tarea demasiado costosa para realizarla cada vez que se necesite enviar un paquete de un host a otro, por lo que para evitar esto se implementó un sistema de **caché**. Cada host mantiene una especie de tabla que mapea las direcciones IP con sus correspondientes direcciones físicas resueltas previamente. De este modo, solo se debe encontrar la dirección física de un host para el envío del primer paquete. Todos los paquetes siguientes pueden encontrar la dirección física de destino en la cache del host.

Antes dimos el ejemplo respecto la separación entre dirección IP con dirección física que decía el caso en el cual un dispositivo se cambiaba y su dirección física cambiaba pero su dirección IP no. Claramente la persona que está escribiendo esto ahora lo hizo con el propósito de usarlo nuevamente. Si pensamos cómo esto afectaría la caché no es difícil ver que nos daría una referencia errónea, ya que nos provee de una dirección física desactualizada. Se dice que la caché de ARP es un ejemplo de *soft state* donde el estado de la misma se puede volver inválido sin advertencia. Para evitar esto se implementan temporizadores en cada entrada de la caché. Cuando un temporizador se vence, la entrada se debe renovar (mediante el mecanismo ARP).

Utilizar la técnica de *soft state* en ARP trae tanto ventajas como desventaja. El lado bueno es que se consigue una autonomía de los hosts, donde cada uno es responsable de la información que tienen sobre las direcciones. El lado malo es que si el timer es de N segundos, la caché deberá esperar estrictamente N segundos para actualizarse. Si algún host se cae durante ese intervalo, deberemos esperar a que terminen los N segundos para detectar la caída.

4.1.6. Optimizaciones de ARP

Si el lector prestó la suficiente atención (espero) se pudo haber dado cuenta de que un paquete ARP contiene toda la información sobre las direcciones del host emisor. Esto significa que cuando el host destino recibe el paquete ARP, además de responder con su dirección física, puede almacenar la información del host emisor en su caché. De esta manera, no necesita realizar un mecanismo ARP para conseguir su dirección.

Si el lector prestó todavía más atención, se dio cuenta de que cuando el host emisor envía un paquete ARP hace un *broadcast*. Es decir, todos los hosts de la red reciben el paquete. Esto implica que reciben la información sobre las direcciones del emisor y aprovechan para almacenar esta información en su caché por si en algún momento tienen que comunicarse con él.

4.1.7. Encapsulado de un paquete ARP

Cuando pensamos sobre el protocolo ARP, pensamos en un protocolo de bajo nivel, que nos permite hacer un puente entre la capa física (de enlace, en realidad) y la capa de red. Siguiendo este pensamiento podemos tomar como que ARP es mas parte del sistema de red física que parte de los protocolos de internet.

Dicho sea esto, un paquete ARP se encapsula directamente en una trama física, siendo identificado mediante unas banderas en la trama misma. El encapsulado se puede apreciar en la figura 17.

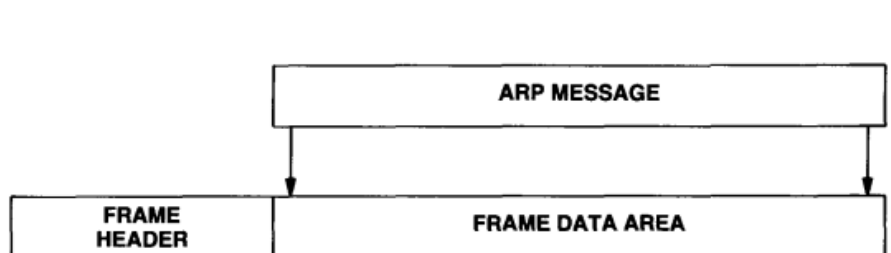


Figura 17: Paquete ARP siendo encapsulada en una trama física - Comer.

4.2. Protocolo IP

El **protocolo de internet**, o **IP** (*Internet Protocol*) a partir de ahora, define el mecanismo de entrega sin conexión y no confiable que utilizan los protocolos de nivel más alto (TCP y UDP). IP proporciona tres definiciones fundamentales:

1. La unidad básica para la transferencia de datos: **datagrama**. Se envía en una red de redes TCP/IP.

2. El software TCP encargado del **ruteo**, seleccionando la ruta por la que se envían los datos.
3. Un conjunto de reglas para la **entrega no confiable**, que caracterizan la forma en que se procesan los paquetes, cómo y cuándo generar errores y las condiciones bajo las cuales los mensajes pueden ser descartados.

Sin importar la versión de IP, todo datagrama IP se puede resumir a una sección denominada *encabezado*, *cabecera* o *header*, y una sección de datos. La sección de datos tiene el mensaje en sí que se desea transmitir, mientras que la cabecera cuenta con la información de a dónde y cómo enviar el datagrama. Si bien cada versión IP tiene sus propios campos en la cabecera, tanto IPv4 como IPv6 tienen la dirección IP de origen y la dirección IP de destino. En particular, la dirección de destino es indispensable para que el mensaje pueda viajar correctamente en una red de redes. Veamos ahora cómo es que se logra esto, es decir, cómo se *rutea* un datagrama IP.

4.2.1. Ruteo de un datagrama IP

En un sistema de conmutación de paquetes, el **ruteo** es el proceso de selección de caminos sobre el que viajan los paquetes, y el ruteador o **router** es la máquina encargada de esta selección.

El proceso de ruteo se divide en dos partes:

- **Entrega directa:** Transmisión de un datagrama entre máquinas conectadas por una sola red física. Es la base de toda comunicación en una red de redes. En esta transmisión no se involucran routers, sino que el transmisor mapea la dirección de destino en su dirección física (lo que vimos en ARP) y envía el datagrama encapsulado en una trama, que viaja directamente al destino. Cabe aclarar que el transmisor primero debe verificar que el destino esté en la misma red. Para eso, compara el *netid* de su propia dirección IP con el *netid* de la dirección IP del destino (para más información sobre direcciones IP consulte [4.3.4](#)).
- **Entrega indirecta:** Transmisión de un datagrama a entre máquinas que no pertenecen a la misma red física, por lo que el transmisor debe pasar el datagrama a un router. Luego, el datagrama pasa de router en router hasta llegar al de la red de destino. Ese último paso hasta el anfitrión de esa red es una entrega directa.

Veamos un poco más en profundidad cómo un datagrama viaja entre routers y llega a su destino correctamente.

4.2.2. Ruteo por tabla

El algoritmo de ruteo IP utiliza **tablas de ruteo IP**. Cada máquina de la red tiene su propia tabla de ruteo en la que almacena información de posibles destinos y cómo alcanzarlos. Para eso, almacena pares (N, R), donde N es la dirección IP de una *red* de destino y R es la dirección IP del siguiente router en el camino hasta la red N. Al router R se lo llama *salto siguiente* (*next hop*).

El hecho de que N sea la dirección de toda una red y no de un anfitrión en particular permite que las tablas se mantengan reducidas y que no tengan que actualizarse si una red cambia la dirección IP de sus anfitriones. También es importante aclarar que cada registro de una tabla apunta a un router al cual se puede alcanzar a través de una sola red física. En la figura 18 se muestra un ejemplo.

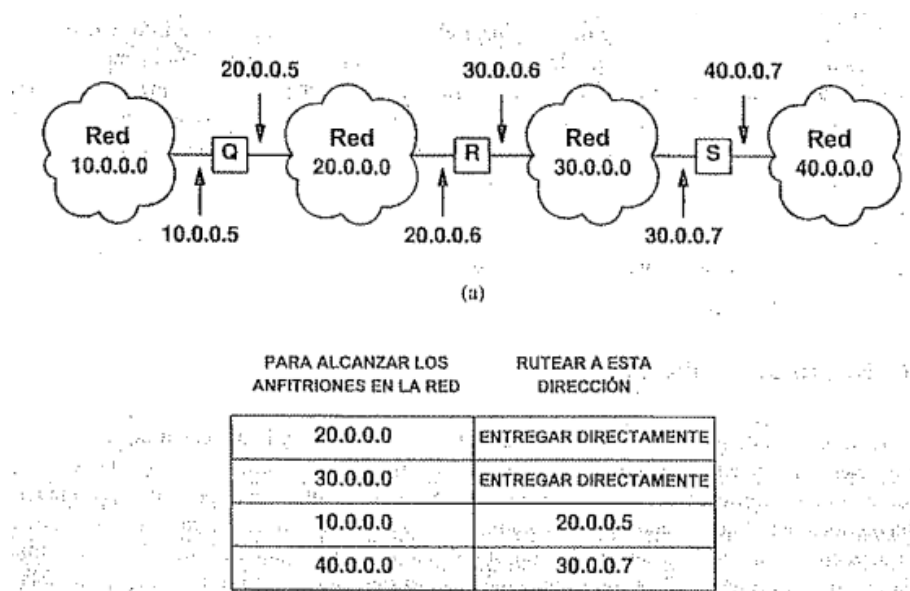


Figura 18: (a) Ejemplo de una red de 4 redes y 3 routers. (b) Tabla de ruteo de R

Otra técnica utilizada para mantener reducido el tamaño de las tablas de ruteo es asociar muchos registros a un **router asignado por omisión**. La idea es que el software IP busca una ruta hacia la red de destino y si no encuentra ninguna, envía el datagrama a ese router asignado por omisión. Por otro lado, el software de ruteo IP permite asignar rutas por un **anfitrión específico** como caso especial. Este tipo de ruteo sirve, más que nada, para depurar conexiones.

Juntando todo esto, el algoritmo de ruteo IP es el de la figura 19.

A modo de resumen, enumeremos los pasos que realiza IP para entregar un datagrama de una máquina a otra en una red de redes. Llamemos A a la máquina origen y B a la máquina destino.

Algoritmo:
RutaDatagrama (Datagrama, Tabla de Ruteo)

Extraer la dirección IP de destino, D, del datagrama
 y computar el prefijo de red, N;
 si N corresponde a cualquier dirección de red directamente
 conectada entregar el datagrama al destino D sobre dicha
 red. (Esto comprende la transformación de D en una
 dirección física, encapsulando el datagrama y enviando
 la trama.)
 De otra forma, si la tabla contiene una ruta con anfitrión
 específico para D, enviar el datagrama al salto siguiente
 especificado en la tabla;
 de otra forma, si la tabla contiene una ruta para la red N,
 enviar el datagrama al salto siguiente especificado en la
 tabla;
 de otra forma, si la tabla contiene una ruta asignada por
 omisión, enviar el datagrama al ruteador asignado
 por omisión especificado en la tabla;
 de otra forma, declarar un error de ruteo;

Figura 19: Algoritmo de ruteo en un pseudocódigo extraño. - Comer.

1. La máquina A prepara un datagrama IP completando todos los campos de la cabecera (luego vamos a nombrar cada uno y su uso) con los datos necesarios, lo encapsula en una trama, y como la dirección de la red de B no coincide con la dirección de su red, lo envía al router.
2. El router recibe la trama, extrae el datagrama y lo pasa al software IP. El software IP mira la dirección de destino, y como su *netid* no coincide con el de su dirección, busca en la tabla de ruteo a qué router enviarlo. Una vez que encuentra una coincidencia, empaqueta el datagrama en una trama de red y lo envía a esa dirección. En realidad hay un paso intermedio que consiste en traducir la dirección IP del router de paso siguiente a su dirección física, pero como de eso se encarga el ARP, no lo mencionamos.
3. Este proceso se repite de router en router. En algunos casos, quizás, tenga que usarse la dirección del router asignado por omisión para continuar con la comunicación hasta que llegue al router de la red de B.
4. Una vez que el datagrama llega al router de la red de B, éste lo extrae de la trama de red en la que viajó, mira la dirección de destino, y como es la dirección de una máquina a la que está conectado por una red física, lo encapsula en una trama de red y se lo envía para que, finalmente, llegue a B.

A continuación veremos las características de cada versión de IP, que si bien comparten un

mismo propósito y su funcionalidad es similar, cada una presenta distintas características, desde el formato de sus datagramas hasta su forma de transmitir los mensajes.

4.3. IPv4

4.3.1. Formato de un datagrama

El formato de un datagrama IPv4 es el de la figura 20.

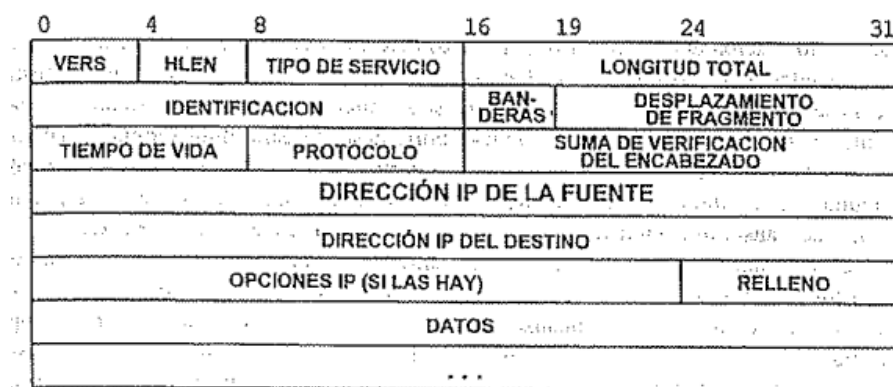


Figura 20: Formato de un datagrama IPv4. - Comer.

Analicemos los campos de la cabecera:

- **VERS:** versión de IP. En este caso es siempre 4.
- **HLEN:** longitud del encabezado. Como puede tener opciones, su tamaño es variable. Está expresado en palabras de 32 bits, el mínimo es 5, el máximo es 15.
- **TIPO DE SERVICIO:** campo de 8 bits que especifica cómo debe manejarse el datagrama. 3 bits para la prioridad (0-7), 2 que no se usan y 3 que se usan como banderas. En orden: retardos cortos (D), alto desempeño (T) y alta confiabilidad (R). Estos bits son sólo de ayuda para los algoritmos de ruteo, no se garantiza que se respeten.
- **LONGITUD TOTAL:** longitud del datagrama medido en bytes, incluyendo header y datos.
- **IDENTIFICACION:** entero único que identifica al datagrama. Se utiliza para identificar distintos fragmentos de un mismo datagrama fragmentado.
- **BANDERAS:** son 3: no fragmentación (NF), más fragmentos (MF) y uno sin uso (X). Si el bit NF está en 1, significa que el datagrama no puede fragmentarse, en

caso de no poder enviarse sin fragmentar se notifica con un error al origen. Si el bit MF está encendido significa que ese datagrama es un fragmento de otro mayor y que no es el último. Sirve para que el software IP sepa si llegó el último fragmento de un datagrama fragmentado.

- **DESPLAZAMIENTO DEL FRAGMENTO:** medido en unidades de 8 bytes. Como los fragmentos de un datagrama pueden llegar en desorden, este campo sirve para reensamblarlo.
- **TIEMPO DE VIDA:** duración, en segundos, del tiempo que un datagrama tiene permitido permanecer en el sistema de red de redes. Por cada router que pasa se va decrementando, una vez que llega a 0 se descarta.
- **PROTOCOLO:** protocolo de alto nivel utilizado para crear el mensaje que se transporta en la sección de datos (TCP, UDP, etc.).
- **SUMA DE VERIFICACIÓN:** utiliza únicamente los datos de la cabecera. Asegura la integridad de sus valores.
- **DIRECCIÓN IP DE LA FUENTE:** necesaria para saber a dónde enviar respuestas o mensajes de errores, en caso de ser necesario.
- **DIRECCIÓN IP DEL DESTINO:** a pesar de que el datagrama sea dirigido a través de varios routers intermedios, este campo y el anterior no se cambian en todo el trayecto.
- **OPCIONES:** su longitud depende de qué opción se selecciona. Todas las opciones tienen por lo menos un byte dividido en tres campos: COPY (1 bit), OPTION CLASS (2 bits), y OPTION NUMBER (5 bits). El bit COPY indica si la opción se debe copiar al fragmentar el datagrama, y los campos OPTION CLASS y OPTION NUMBER especifican la clase general de opción y establecen una opción específica en esta clase. Además, algunas opciones tienen más campos por lo que su longitud es variable. Luego vamos a ver las más usadas.

Una vez visto el formato del header de un datagrama IPv4, vamos a tratar de entender un poco cómo funciona un envío de mensajes en este protocolo y porqué son necesarios todos esos campos.

4.3.2. Envío de un datagrama

Sabemos que existen distintas versiones de IP, en particular la 4 y la 6 son las únicas que se utilizan a día de hoy, es por eso que el campo VERS tiene relevancia. Cada vez que un

software IP procesa un datagrama, lo primero que debe hacer es verificar que ambos sean de la misma versión, en caso contrario el datagrama se descarta para evitar que sea mal interpretado. También sabemos que cada datagrama lleva en su encabezado un listado de 0 o más opciones, que a su vez son de longitud variable entre sí, por lo que el campo HLEN indica el tamaño únicamente del encabezado, sin contar la sección de datos. El campo que sí incluye los datos es el de LONGITUD TOTAL. Notemos que tiene 16 bits por lo que el tamaño máximo posible de un datagrama es de $2^{16} = 65535$ bytes.

El campo TIPO DE SERVICIO, conocido como *Type Of Service (TOS)*, tiene información que puede llegar a utilizar el algoritmo de ruteo para elegir una entre varias rutas hacia un destino. Sin embargo puede no llegar a utilizarse dependiendo de la red y su tráfico.

Cada datagrama IP viaja **encapsulado** en una trama de red. Es importante entender que gracias a la estratificación (división por capas), el hardware nunca se entera si una trama de red tiene dentro un datagrama IP (todo el datagrama, incluido el encabezado, va dentro del área de datos de la trama), de hecho ni siquiera entiende de direcciones IP. El protocolo ARP es el encargado de tomar la dirección IP del datagrama y decirle a la trama de red a qué dirección física corresponde.

Para que un datagrama no quede dando vueltas en una red de redes eternamente (por ejemplo si los routers erróneamente crearon un ciclo con sus tablas de ruteo), se utiliza el campo TIEMPO DE VIDA, el cual contiene un entero positivo que se va decrementando a medida que el datagrama es procesado por routers.

Por último, el campo OPCIONES, como vimos antes tiene una longitud variable dependiendo de qué opciones se utilicen para enviar el datagrama, veamos las de ruteo y sello de hora que pueden llegar a ser las más interesantes, ya que proporcionan una manera de controlar cómo la red de redes maneja las rutas.

- **Opción de registro de ruta:** permite a la fuente crear una lista de direcciones IP y arreglarla para que cada router que maneje el datagrama añada su propia dirección IP a la lista. Cuando el datagrama llega a destino, la máquina puede extraer y procesar esta lista para saber cómo fue la ruta que tomó.
- **Opción de ruta fuente:** permite a la fuente dar un listado de direcciones IP por las que el datagrama debe pasar si o si. Es útil cuando se quiere testear el funcionamiento de una ruta o un/os router/s en particular. Existen dos formas de ruteo de fuente: el *ruteo estricto* que requiere que se especifiquen todas las direcciones desde la fuente al destino, de modo que tiene que existir una red física entre cada par de direcciones contiguas; y el *ruteo no estricto* que también incluye una lista de direcciones por las que debe pasar el datagrama, pero no son necesariamente todas las de la ruta.
- **Opción de sello de hora:** funciona similar a la opción de registro de ruta. Esta

opción contiene una lista inicial vacía y cada router por el que pasa el datagrama escribe sus datos en la lista. Estos datos son su dirección IP y la fecha y hora en la que pasó el datagrama. Esta funcionalidad sirve para dejar un registro más exacto de la ruta del datagrama para así saber, por ejemplo, si un router está congestionado.

El último campo RELLENO se llena con ceros y sirve para que el campo OPCIONES tenga un tamaño que sea múltiplo de 32.

4.3.3. Fragmentación

En un caso ideal, un datagrama completo viaja dentro de una sola trama física y llega a destino sin fragmentarse en el camino. Sin embargo esto no sucede siempre. Tengamos en cuenta que no se puede establecer un tamaño máximo de datagrama de manera que entre completo en una trama y pueda transportarse por cualquier red sin fragmentarse y aprovechando al máximo su ancho de banda. Esto es debido a la variedad de redes que existen, que, según la tecnología física que usen para sus enlaces, cada una con tendrá propia velocidad de transmisión y su propio tamaño MTU (*Maximum Transfer Unit*). Para solucionar esto existe un proceso llamado (aunque ya lo mencionamos) **fragmentación**, el cual consiste en tomar un datagrama, partirlo en varios pedazos de igual tamaño (excepto el último, probablemente, que será menor al resto) y enviar cada uno por separado. Cabe destacar que los tamaños de los fragmentos deben ser múltiplos de 8 bytes (Esto es por que el desplazamiento en la cabecera esta explicitado en múltiplos de 8 bytes). Supongamos que tenemos un datagrama de tamaño 1400 (indicado en el campo TAMAÑO TOTAL) y la red en la que se intenta transmitir tiene un tamaño de MTU de 620. Como el datagrama no puede viajar en una sola trama, se procede a fragmentarlo en tres partes. Dos serán de un tamaño cercano a 620, digamos 600, y la tercera de lo que sobre, es decir, 20. Este escenario está representado en la figura 21.

Todos los fragmentos tienen en el campo IDENTIFICACIÓN el mismo número que tenía el datagrama original, de esta manera, cuando lleguen al destino, el software IP sabrá que corresponden al mismo datagrama. Además, como pueden llegar desordenados, también es necesario el campo DESPLAZAMIENTO DEL FRAGMENTO para saber cuál va primero y cuál va después. En este caso el primer fragmento tiene un desplazamiento 0, el segundo 600 y el tercero 1200. Por último, en el ejemplo, el campo BANDERAS del primer y segundo fragmento tiene el bit *más fragmentos* (MF) en 1 y el tercero lo tiene en 0, indicando que es el último fragmento. Todos los campos del encabezado de cada fragmento son copiados del datagrama original, excepto por los campos DESPLAZAMIENTO DEL FRAGMENTO, BANDERAS, HLEN y TAMAÑO TOTAL.

Es importante saber que el ensamblado de estos fragmentos se realiza solamente en el destino, y no en cada router. Esto implica que los fragmentos se deben conservar hasta

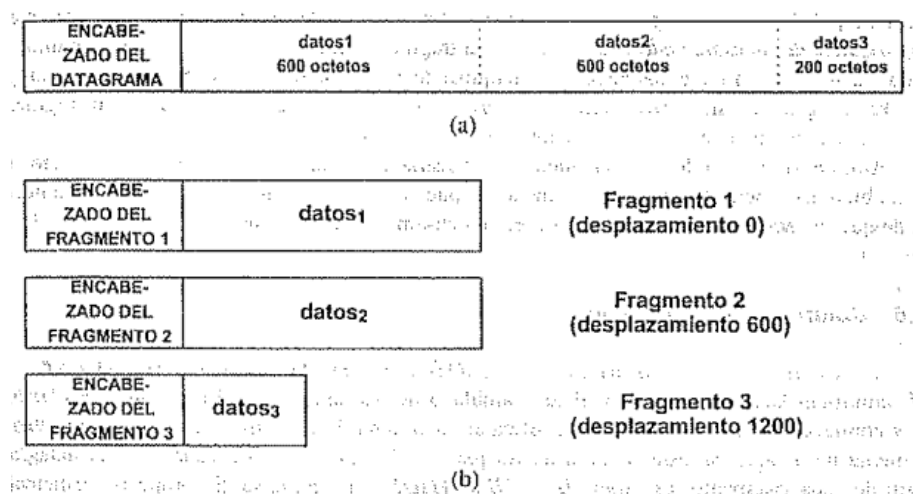


Figura 21: (a) Datagrama original de tamaño 1400. (b) Tres fragmentos para la red con MTU de 620. - [Comer](#).

llegar a su destino final. Esto trae algunas desventajas: Primero, puede ocurrir que el paquete fue dividido en fragmentos pequeños para pasar a través de una red y, como se mantienen en este tamaño, aunque la siguiente red tenga un MTU mayor, solo pasarán fragmentos pequeños. Esto es claramente ineficiente. Segundo, si el envío de alguno de los fragmentos falla, el datagrama no podrá completarse. Cuando se recibe un fragmento se suele iniciar un temporizador de reconstrucción, de modo que si este expira (es decir, no se pudo reconstruir el paquete), el datagrama se descarta. Esto lleva a que la probabilidad de pérdidas de datagramas se incremente, ya que con perder solamente un fragmento se descarta un datagrama entero.

4.3.4. Direcciones IP

Lo que nos queda por ver ahora es cómo son las direcciones IPv4. Una **dirección IPv4** está formada por 32 bits que se dividen en 4 octetos o bytes para que sea más fácil de leer. La notación entonces es **x.x.x.x** donde cada x es un byte. Cada dirección se divide en dos partes: la **netid** y la **hostid**. Los bits de la *netid* indican el número de identificación de la red, y los de la *hostid* identifican a las conexiones dentro de dicha red. Existen cinco clases de direcciones en IPv4, de las cuales sólo tres se usan en la práctica. Estas clases se muestran en la figura 22.

Es importante hacer hincapié en que las direcciones especifican la conexión a una red, no un equipo en particular. Esto es importante ya que una misma computadora o router puede tener más de una dirección IP asignada si es que está conectado a más de una red. Por lo tanto, un router que conecta n redes, tiene n direcciones distintas asignados a él. Por convención, una dirección que tiene su *hostid* con todos ceros, hace referencia a la

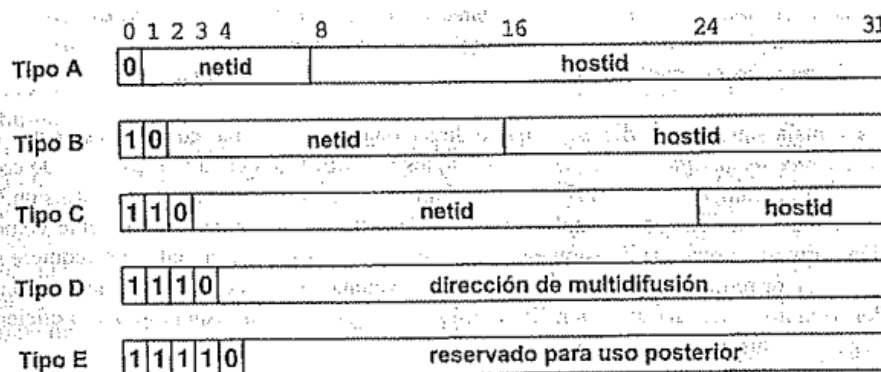


Figura 22: Las cinco clases de direcciones IPv4. En la práctica se utilizan la A, B y C. - [Comer](#).

red en sí misma, no a un anfitrión. Por otro lado, si todos los bits de su *hostid* son unos, está indicando la dirección de difusión o *broadcast* de la red, tampoco es un anfitrión.

Volvamos a los tipos de direcciones y analicemos un poco qué implica reservar bits para distinguir el tipo de cada una. Según vimos en la figura anterior, una dirección de tipo A tiene 7 bits para su *netid* y 24 campos para su *hostid*, por lo tanto esta clase sirve para redes enormes que tienen muchos anfitriones (pueden asignar más de 16 millones de direcciones distintas). Luego siguen las de clase B que se utilizan para redes medianas ya que tiene 14 bits para el *netid* y 16 para el *hostid*. Por último, están las de clase C, que tienen 21 bits para el *netid* y 8 bits para el *hostid*, por lo que se usan para redes pequeñas. Gracias a que las direcciones están divididas de esta manera, el ruteo de datagramas IP es eficiente, ya que sólo con ver el *netid* de una dirección, el router ya puede saber si corresponde a una dirección de la red local o no.

En la figura 23 se muestra el rango de valores de las direcciones IPv4 según cada clase. Notemos que no todas las posibles direcciones fueron asignadas a un tipo. Por ejemplo, la dirección 127.0.0.0 debería estar en las direcciones de clase A, pero no está ya que se utiliza para *loopback*. Esto es, que se utiliza para pruebas en la máquina local. Si un datagrama IP tiene como dirección de destino el *loopback*, éste nunca aparece en ninguna red, sino que regresa a la máquina de origen sin generar ningún tráfico de red.

Tipo	Dirección más baja	Dirección más alta
A	0.0.0.0	126.0.0.0
B	128.0.0.0	191.255.0.0
C	192.0.0.0	223.255.255.0
D	224.0.0.0	239.255.255.255
E	240.0.0.0	247.255.255.255

Figura 23: Rango según cada tipo de direcciones IPv4. Algunos valores están reservados para propósitos especiales. - [Comer](#).

4.3.5. Direcciones IP privadas vs públicas

Los diseñadores de TCP/IP se dieron cuenta que si seguían repartiendo direcciones sin medida, iban a terminar agotándose. A partir de este problema surge la idea de reservar ciertos rangos de direcciones para que se puedan utilizar en todas las redes que no tengan contacto con el exterior. A este tipo de direcciones se las llamó **direcciones privadas**. La idea detrás de esto consiste en darle una dirección IP pública al lado del router de una red conectado a Internet (para que los mensajes de otras redes puedan llegar hasta él), y que todos los anfitriones de una red que no están conectados directamente con Internet, tengan una dirección privada. Este tipo de direcciones es útil ya que todas las redes del mundo (no conectadas al exterior) podrían tener la misma dirección IP.

En la figura 24 se muestra un cuadro con las direcciones privadas de cada clase y en la figura 25, un diagrama representativo del escenario descrito.

Clase	Rango de direcciones reservadas de redes
A	10.0.0.0
B	172.16.0.0 - 172.31.0.0
C	192.168.0.0 - 192.168.255.0

Figura 24: Rango de direcciones IPv4 privadas según su clase. - [Slides de la cátedra](#).



Figura 25: Diagrama representativo del uso de las direcciones privadas. - [Slides de la cátedra](#).

4.3.6. Direccionamiento de subred (subneteo)

El último aspecto por ver de direcciones IPv4 es el **direccionamiento de subred** (subneteo para los amigos). Creado, al igual que las direcciones IP privadas, ante la necesidad de aumentar el número de direcciones. El subneteo consiste en que varias redes utilicen

la misma dirección. Esto es, utilizar bits del *hostid* para crear un nuevo campo llamado ***subnetid***. En la figura 26 se muestra una representación conceptual del esquema original de dirección IP, y debajo el esquema de subred. Para entenderlo mejor, sigamos el ejemplo

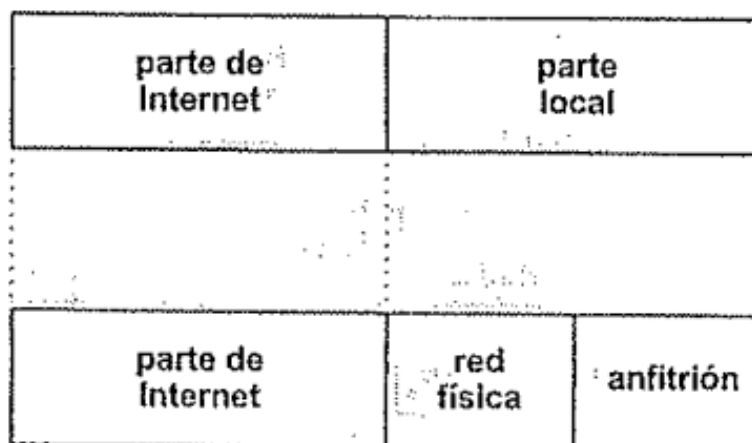


Figura 26: Representación conceptual de un esquema tradicional y uno con subneteo para una dirección IPv4. - Comer.

de la figura 27.

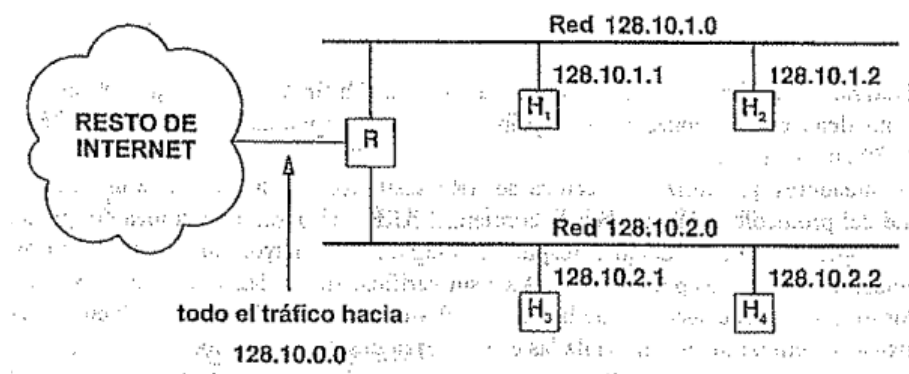


Figura 27: Ejemplo de dos redes que utilizan el concepto de subred. - Comer.

Este escenario consiste en dos redes: llamémoslas Red 1 a la 128.10.1.0 y Red 2 a la 128.10.2.0. Con lo que sabíamos hasta ahora, la dirección 128.10.0.0 corresponde a una de clase B, es decir que su máscara de red es 255.255.0.0, y sus dos bytes inferiores corresponden a su *hostid*. Sin embargo, gracias al subneteo, podemos dividir esta dirección para que dos redes distintas puedan usarla. Entonces la máscara de red pasa a ser 255.255.255.0, es decir que, a pesar de ser una dirección de clase B, el tercer bytes ahora también se utiliza para identificar a la red.

El ruteo, entonces, es el siguiente: un datagrama llega de internet al router R (cuya dirección puede ser, por ejemplo, 128.10.0.1), el router mira los primeros dos bytes de la

dirección de destino y ve 128.10, así que sabe que ese datagrama está dirigido (si contiene una dirección válida) a alguna de las redes a la que está conectado. Como dicha dirección está utilizando subneteo, se fija en el tercer byte para saber a qué red enviarlo. Por último, si el tercer byte era 1 y el cuarto 1 o 2, entonces transforma la dirección IP en una física, lo encapsula en una trama y realiza una entrega directa a la Red 1. Análogo para la Red 2.

La ventaja del subneteo es que es flexible, es decir, la cantidad de bits que se reservan para el *subnetid* dependen exclusivamente de cada red. Si una empresa tiene una dirección tipo B asignada y necesita sólo 5 subredes con bastantes anfitriones en cada una, entonces puede ser una buena decisión reservar 3 bits para el *subnetid* (podría representar hasta $2^3 = 8$ subredes), y dejar los otros 13 para *hostid*. Por el contrario, si una organización necesita muchas redes, supongamos 1000, con pocos anfitriones, entonces lo más razonable sería reservar al menos 10 bits (para poder tener hasta $2^{10} = 1024$ subredes), y dejar los otros 6 para identificar anfitriones. Es decir, el direccionamiento de subred permite encontrar un balance entre cantidad de subredes y anfitriones, adaptándose a la necesidad de cada organización.

Esta flexibilidad implica que se tenga que modificar el algoritmo de ruteo, ya que un router no tiene forma de saber cuál es la máscara de una red con subneteo. Para eso, lo que se hizo fue agregar a cada registro de las tablas de ruteo, además de la dirección de red y la dirección de salto siguiente, su máscara de subred. Entonces, cuando el algoritmo elige rutas, usa la máscara de subred para extraer bits de la dirección de destino y compararlos con el registro en la tabla. Es decir, agarra la máscara de subred, hace una operación AND con la dirección de destino del datagrama, y si el resultado coincide con el campo *dirección de red* en una entrada de la tabla, rutea el datagrama a la *dirección de salto siguiente* que tenga asociada.

4.4. IPv6

Lo último que vimos de IPv4 está relacionado con la necesidad de direcciones IP debido al aumento de popularidad que tuvo Internet desde sus inicios. Cansados de hacer malabares con las pocas (2^{32} , o sea más de 4 mil millones) direcciones IP que ofrecía IPv4, los diseñadores de TCP/IP decidieron crear la sexta versión de IP, es decir, IPv6 (IPv5 también existió pero fue sólo de prueba, nunca se usó realmente).

Una de las mayores diferencias que tiene IPv6 con respecto a sus predecesores, es la cantidad de direcciones que ofrece, ya que en esta versión cada dirección IP, llamadas **direcciones IPv6**, cuenta con 128 bits. Esto quiere decir que existen 2^{128} direcciones posibles. Este aumento quizás no parece mucho pero representa un aumento exponencial de direcciones.

Otros aportes que hizo IPv6 fue: simplificar los encabezados con un sistema de encabezados de extensión, hacer que la fragmentación y el ensamble de fragmentos se hagan únicamente en el origen y destino, no requiere el uso de NAT (usar direcciones privadas para las redes y públicas para los routers), lo que permite conexiones extremo a extremo, entre otros.

Como hay varios aspectos que ambas versiones de IP comparten, vamos a explicar lo nuevo que aporta IPv6 e ir comparando su funcionamiento con IPv4, por lo que puede llegar a ser necesario tener entendido cómo trabaja IPv4.

4.4.1. Formato de un datagrama

Otro de los grandes cambios que tuvo IPv6 con respecto a IPv4 fue el formato y los campos de su encabezado. En la figura 28 podemos ver una comparación entre una cabecera IPv4 y una IPv6.

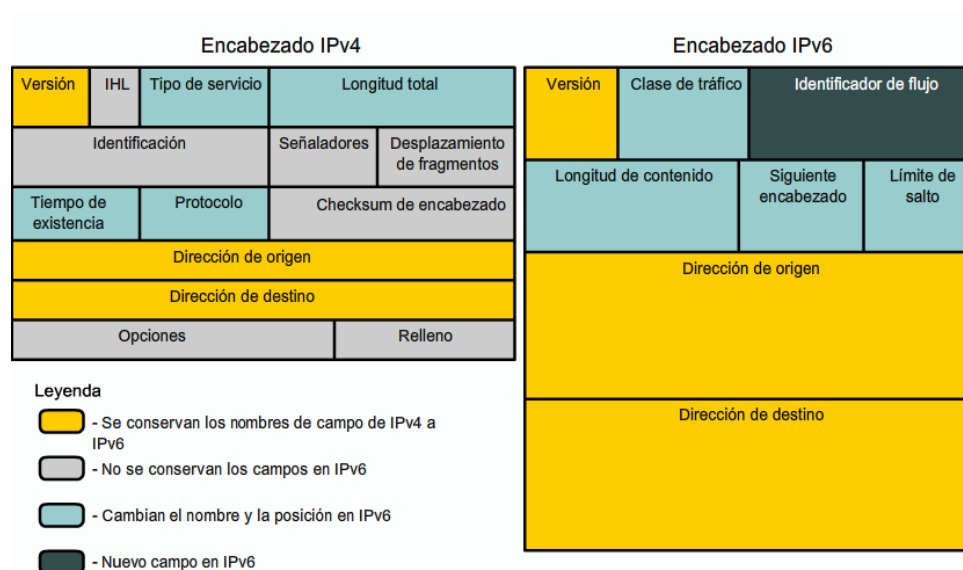


Figura 28: Comparación entre el formato de cabecera de IPv4 y el de IPv6

Notemos que IPv6 presenta menos campos y, además, una longitud fija ($32 * 10 \text{ bits} = 40 \text{ bytes}$) para el encabezado base, ya que no está más el campo OPCIONES que podía variar su tamaño.

Veamos para qué es cada campo de este nuevo encabezado:

- **Versión:** Identifica la versión del protocolo IP utilizado. En este caso su valor es 6.
- **Clase de tráfico:** Identifica y diferencia los paquetes por clases de servicios o prioridad. Este continúa ofreciendo las mismas funcionalidades y definiciones del campo TIPO DE SERVICIO de IPv4.

- **Identificador de flujo:** Identifica y diferencia paquetes del mismo flujo en la capa de red. Este campo permite que el router identifique el tipo de flujo de cada paquete, sin necesidad de verificar su aplicación.
- **Longitud de contenido:** Indica el tamaño, en bytes, solamente de los datos enviados junto con los encabezados de extensión de IPv6. Reemplaza al campo TAMAÑO TOTAL usado en IPv4, que indica el tamaño del encabezado mas el tamaño de los datos transmitidos. En el cálculo del tamaño también se incluyen los encabezados de extensión.
- **Siguiente encabezado:** Identifica el encabezado de extensión que sigue al encabezado de IPv6. El nombre de este campo fue modificado (en IPv4 se llamaba PROTOCOLO) para reflejar la nueva organización de los paquetes IPv6, ya que ahora este campo no solo contiene valores referentes a otros protocolos sino que también indica los valores de los encabezados de extensión.
- **Limite de salto:** Indica el número máximo de routers que el paquete IPv6 puede pasar antes de ser descartado; se decrementa en cada salto. Estandarizó el modo en que se utilizaba el campo Tiempo de Vida (TTL) de IPv4, a pesar de la definición original del campo TTL, diciendo que éste debe indicar, en segundos, el tiempo que el paquete demorará en ser descartado en caso de no llegar a su destino.
- **Dirección de origen:** Indica la dirección IPv6 de origen del paquete. A diferencia de IPv4, este campo ahora ocupa 128 bits.
- **Dirección de destino:** Indica la dirección IPv6 de destino del paquete. Al igual que el campo anterior, ocupa 128 bits y no se modifica en todo el viaje del datagrama.

4.4.2. Cabeceras de extensión

En IPv6 las opciones se tratan por medio de las cabeceras de extensión o encabezados de extensión, que no tienen ni cantidad ni tamaño fijo. Estas cabeceras se encuentran entre el encabezado base y el encabezado de la capa inmediatamente superior. Si en un mismo paquete existen múltiples encabezados de extensión, éstos serán agregados en serie formando una “cadena de encabezados”. En la figura 29 se muestra un ejemplo.

En IPv6, el uso de encabezados de extensión busca aumentar la velocidad de procesamiento en los routers, ya que solo dos son procesados en cada router; los demás sólo son tratados por la máquina destino. Además, se pueden definir y utilizar nuevos encabezados de extensión sin tener que modificar el encabezado base.

Cada encabezado de extensión tiene un campo llamado “*Siguiente Encabezado*” que apunta al próximo encabezado en la cadena. De esta forma, la estructura de los encabezados

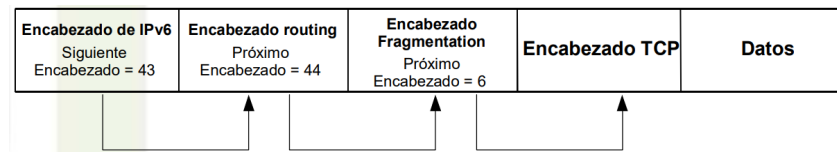


Figura 29: Ejemplo de un datagrama en IPv6 con encabezados de extensión que transporta un segmento TCP. - [Curso IPv6 básico](#).

de extensión es la misma que la de una lista enlazada simple, donde cada nodo apunta al siguiente.

Cada tipo puede aparecer una sola vez (excepto por uno). Además, el orden en el que aparecen es importante para lograr un procesamiento eficiente en los routers intermedios. Las especificaciones de IPv6 definen seis encabezados de extensión (entre paréntesis se aclara cuál es su valor en el campo “*Siguiete Encabezado*”):

- **Hop-by-Hop** (0): Transporta datos que deben ser procesados por todos los nodos a lo largo del camino del paquete. Puede contener dos posibles opciones: *Router Alert* indica que el mensaje debe recibir un tratamiento especial, y *Jumbogram* indica que el paquete tiene tamaño mayor o igual a 64KB. Tiene además dos bits que indican qué hacer si no se reconoce ninguna opción:
 - **00**: ignorar y continuar con el procesamiento
 - **01**: descartar el paquete.
 - **10**: descartar el paquete y enviar un mensaje ICMP *Parameter Problem* al origen.
 - **11**: descartar el paquete y enviar un mensaje ICMP *Parameter Problem* al origen del paquete si el destino no es una dirección multicast.
- **Destination** (60): Este es el único encabezado que puede aparecer dos veces, si aparece antes del encabezado *Routing*, es procesado por un router. Si no, tiene que estar último y es procesado únicamente por el destino. Permite agregar opciones extras para que el destino (o router) utilice en el procesamiento.
- **Routing** (43): Indica una lista de routers por los que el paquete debe pasar antes de llegar a destino. Su funcionamiento se puede ver en el ejemplo de la figura 30.
- **Fragmentation** (44): Utilizado cuando un datagrama viaja fragmentado. Contiene información necesaria para identificar cada fragmento al momento de reensamblar el mensaje original. Además del campo “*Siguiete Encabezado*”, cuenta con otros tres campos:

1. **Desplazamiento del fragmento:** Indica, en unidades de ocho bytes, la posición de los datos transportados por el fragmento actual respecto del inicio del paquete original.
 2. **Bandera M:** Si está encendida, indica que hay más fragmentos. Si está apagada, indica que es el fragmento final.
 3. **Identificación:** Valor único generado por el nodo de origen para identificar el paquete original. Se utiliza para detectar los fragmentos de un mismo paquete.
- **Authentication (51):** Sirve para verificar que los datos del datagrama no hayan sido alterados. En el caso de mensajes fragmentados, se utiliza una vez que ya se reensambló el mensaje.
 - **Encapsulating Security Payload (52):** Garantiza integridad y confidencialidad de los paquetes. Este encabezado puede ir en cualquier parte entre el header base y el de la capa superior. Todos los datos que siguen luego de este encabezado son encriptados.

Algunas observaciones a tener en cuenta. El orden de los encabezados importa porque reduce el tiempo de procesamiento en cada router. Se deben poner en primer lugar los headers que deben ser procesados por todos los nodos, y al final los que son sólo para el destino. Por otro lado, si un paquete fue enviado a una dirección *multicast*, todos los encabezados de extensión serán examinados por todos los nodos del grupo.

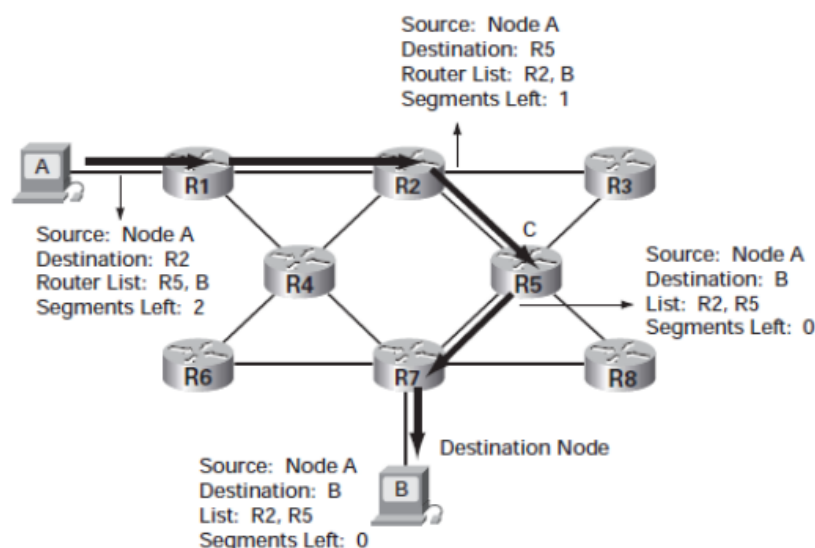


Figura 30: Ejemplo de cómo van cambiando los campos del encabezado Routing a medida que se va ruteando un datagrama IPv6 de un router en otro.

4.4.3. Direccionamiento

Dijimos que las **direcciones IPv6** están formadas por 128 bits. Escribirlas de la misma forma que hacíamos con IPv4 sería muy tedioso. Veamos cómo son y cómo se escriben las direcciones IPv6.

La representación de las direcciones IPv6 divide la dirección en ocho grupos de 16 bits, separados mediante ':', escritos con dígitos hexadecimales. Por ejemplo, una dirección IPv6 podría ser la siguiente:

2001:0DB8:AD1F:25E2:CADE:CAFE:FOCA:84C1

En la representación de las direcciones IPv6 está permitido utilizar tanto caracteres en mayúscula como en minúscula. Además, se pueden aplicar reglas de abreviatura para facilitar la escritura de algunas direcciones muy extensas. Se permite omitir los ceros a la izquierda de cada bloque de 16 bits y también reemplazar una larga secuencia de ceros por "::". Este último reemplazo se permite realizar solo una vez por dirección para evitar ambigüedades.

Otra representación importante es la de los prefijos de red. En las direcciones IPv6 continúa escribiéndose del mismo modo que en IPv4, utilizando la notación CIDR. Esta notación se representa con la forma "*dirección/tamaño del prefijo*", donde "tamaño del prefijo" especifica la cantidad de bits contiguos a la izquierda de la dirección que comprenden el prefijo. En el ejemplo de prefijo de subred presentado a continuación, de los 128 bits de la dirección, 64 bits se utilizan para identificar la subred.

Prefijo: 2001:db8:3003:2::/64

Prefijo global: 2001:db8::/32

ID de la subred: 3003:2

En IPv6, las direcciones se asignan a interfaces, no a nodos. Esto quiere decir que una misma interfaz puede tener múltiples direcciones IPv6 asociadas. Además, en esta versión no existen más las direcciones broadcast, sino que se definieron tres tipos de direcciones nuevas:

- **Unicast:** Identifica una única interfaz. Es decir que un paquete enviado a una dirección unicast, se entrega a una única interfaz. A estas comunicaciones se las llama "uno a uno".
- **Anycast:** Identifica un conjunto de interfaces. Un paquete enviado a una dirección anycast se entrega a la interfaz del conjunto más próxima al origen (de acuerdo con la distancia medida por los protocolos de enrutamiento). Las direcciones anycast se utilizan en comunicaciones "de uno a alguno".

- **Multicast:** También identifica un conjunto de interfaces, pero un paquete enviado a una dirección multicast se entrega a todas las interfaces asociadas a esa dirección. Las direcciones multicast se utilizan en comunicaciones “de uno a muchos (todos)”.

Veamos más en detalle cada uno de los tipos de direcciones.

4.4.4. Unicast

Las direcciones **unicast** se utiliza para comunicaciones entre dos nodos, y su estructura fue definida para permitir agregaciones con prefijos de tamaño flexible.

Existen varios tipos de direcciones unicast IPv6:

- **Global Unicast:** Equivalente a las direcciones IPv4 públicas. Son globalmente ruteables y accesibles desde Internet. Está formada por un prefijo de enrutamiento global (de $n = \{40, 48, 56\}$ bits), una identificación de subred (de $64 - n = \{24, 16, 8\}$ bits) y una identificación de interfaz (de 64 bits). Su rango va desde `2000::` hasta `3fff:ffff:...:ffff`. En la práctica se usa mucho `2001:DB8::`

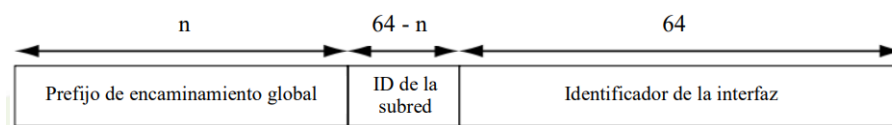


Figura 31: Formato de una dirección IPv6 global unicast. - [IPv6](#).

- **Link Local:** Pueden utilizarse solo en el enlace específico en el cual la interfaz está conectada. Es atribuida automáticamente usando el prefijo `FE80::/64`. Los routers no deben enrutar paquetes cuyo origen o destino es una dirección link-local hacia otros enlaces.

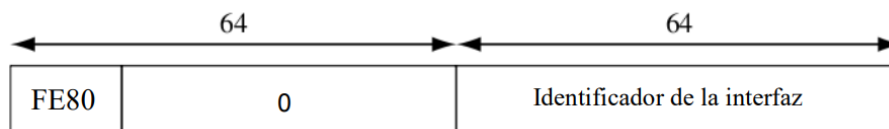


Figura 32: Formato de una dirección IPv6 link local. - [IPv6](#).

- **Unique local (ULA):** Utilizada solamente para comunicaciones locales, generalmente dentro de un mismo enlace o conjunto de enlaces. Una dirección ULA no debe ser ruteable en Internet. Su prefijo es `FC00::/7`. El octavo bit es una bandera (L), si el valor es 1, el prefijo es atribuido localmente. Si el valor es 0, el prefijo debe ser atribuido por una organización central. Luego sigue el identificador de 40

bits usado para crear un prefijo globalmente único, 16 bits más para la subred y los últimos 64 para el identificar las interfaces.

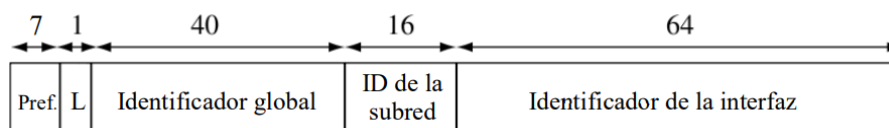


Figura 33: Formato de una dirección IPv6 única local. - IPv6.

Los **identificadores de interfaz (IID)**, utilizados para distinguir las interfaces dentro de un enlace, deben ser únicos dentro del mismo prefijo de subred. Son lo equivalente a los *hostid* de IPv4. El mismo IID se puede usar en múltiples interfaces de un único nodo, pero éstas deben estar asociadas a subredes diferentes.

Si bien se pueden elegir arbitrariamente, un IID puede formarse a partir de la dirección MAC de una interfaz, en el formato **EUI-64**. Un IID basado en el formato EUI-64 se genera de la siguiente manera: Si la interfaz tiene una dirección MAC de 64 bits, solo debe cambiar (pasar de 0 a 1, o viceversa) el séptimo bit más a la izquierda de la dirección MAC. Si la interfaz tiene una MAC de 48 bits, primero se agregan los dígitos **FFFE** entre el tercer y cuarto byte de la MAC, y luego se hace lo mismo que antes.

En la figura 34 se muestra un ejemplo para la dirección MAC **48:1E:C9:21:85:0C**, obteniendo como IID final **4A:1E:C9:FF:FE:21:85:0C**. Para terminar con unicast, veamos

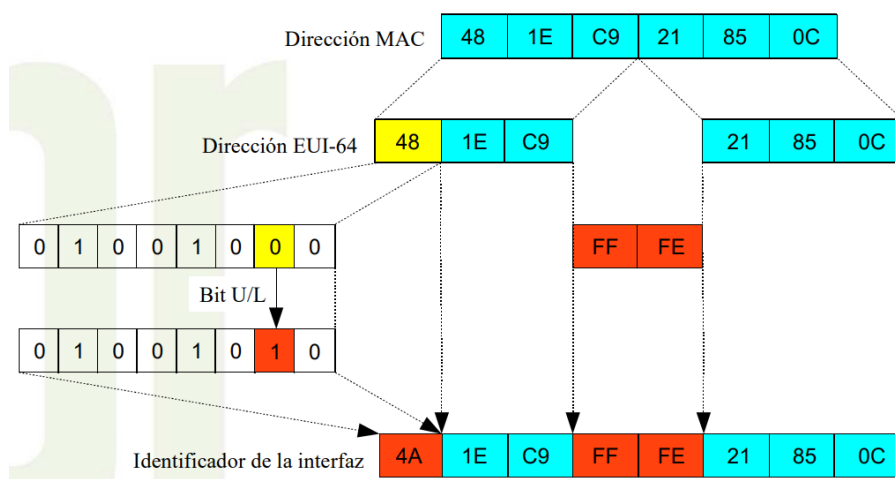


Figura 34: Ejemplo de IID basado en el formato EUI-64 para la dirección MAC **48:1E:C9:21:85:0C**. - IPv6.

las direcciones especiales:

- **Dirección no especificada (*Unspecified*)**: Representada con la dirección **::0**. Nunca debe ser atribuida a la dirección de un nodo ya que representa la ausencia

de dirección. Se usa como dirección de origen en la etapa de inicialización de un datagrama.

- **Loopback:** Representada por `::1`. Equivalente a `127.0.0.1` en IPv4. Se utiliza para representar la propia máquina y es utilizada en pruebas internas. Este tipo de dirección no debe atribuirse a ninguna interfaz física ni ser utilizada como dirección de origen en paquetes IPv6 enviados a otros nodos. Además, un paquete IPv6 con una dirección loopback como destino no puede ser enviado por un router IPv6, y si un paquete recibido en una interfaz tiene una dirección loopback como destino, éste debe ser descartado.

4.4.5. Anycast

Una dirección IPv6 anycast se utiliza para identificar un grupo de interfaces, aunque con la propiedad de que un paquete enviado a una dirección anycast es ruteado solamente a la interfaz del grupo más próxima al origen del paquete. Las direcciones anycast se atribuyen a partir del rango de direcciones unicast y no hay diferencias sintácticas entre las mismas. Por lo tanto, una dirección unicast atribuida a más de una interfaz se transforma en una dirección anycast, debiéndose en este caso configurar, explícitamente, los nodos para que sepan que les ha sido atribuida una dirección anycast. Por otra parte, esta dirección se debe configurar en los routers como una entrada independiente (prefijo /128 - host route).

Este esquema de direccionamiento se puede utilizar para descubrir servicios en la red, como por ejemplo servidores DNS y proxies HTTP. También se puede utilizar para realizar balanceo de carga en situaciones donde múltiples hosts o routers proveen el mismo servicio, para localizar routers que provean acceso a una determinada subred o para localizar los Agentes de Origen en redes con soporte para movilidad IPv6.

Todos los routers deben soportar la dirección **anycast Subnet-Router**. Este tipo de direcciones están formadas por el prefijo de la subred y el IID completado con ceros (por ejemplo: `2001:DB8:CAFE:DAD0::/64`). Un paquete enviado a la dirección Subnet-Router será entregada al router más próximo al origen dentro de la misma subred.

4.4.6. Multicast

Las direcciones **multicast** se utilizan para identificar grupos de interfaces, ya que cada interfaz puede pertenecer a más de un grupo. Los paquetes enviados a esas direcciones se entregan a todas las interfaces que componen el grupo. Su funcionamiento es similar al de broadcast, dado que un único paquete es enviado a varios hosts, diferenciándose solo por el hecho de que en broadcast el paquete se envía a todos los hosts de la red sin excepción,

mientras que en multicast solamente un grupo de hosts reciben el paquete.

Estas direcciones **no** deben ser utilizadas como dirección de origen de un paquete. Derivan del bloque `FF00::/8`, donde el prefijo `FF`, que identifica una dirección multicast, es precedido por cuatro bits, que representan cuatro flags, y un valor de cuatro bits que define el alcance del grupo multicast. Los 112 bits restantes se utilizan para identificar el grupo multicast. En la figura 35 se muestra el formato de una dirección multicast. Las

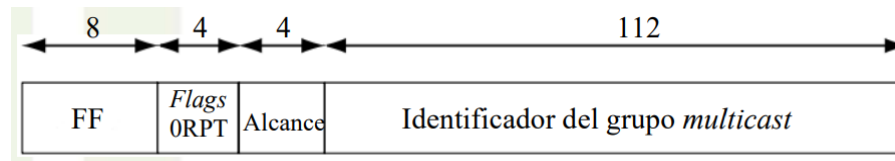


Figura 35: Formato de una dirección IPv6 multicast. - [IPv6](#).

banderas son:

- **0**: Reservado para uso futuro.
- **R**: Indica si lleva (1) o no (0), una “Dirección de encuentro”.
- **P**: Indica si está basada (1) o no (0), en un prefijo de red.
- **T**: Indica si la dirección es permanente (0), es decir, fue asignada por la IANA. O si fue asignada dinámicamente (1).

Los cuatro bits que representan el alcance de la dirección multicast se utilizan para delimitar el área que abarca un grupo multicast. Los valores atribuidos a este campo son los siguientes:

- **1**: Interfaz local (loopback).
- **2**: Enlace.
- **3**: Subred.
- **4**: Admin (configurado).
- **5**: Site.
- **8**: Organización.
- **E**: Internet.
- **0, F**: Reservados.

- 6, 7, 9, A, B, C, D: No distribuidos.

La figura 36 muestra algunas direcciones multicast permanentes.

Dirección	Alcance	Descripción
FF01::1	Interfaz	Todas las interfaces en un nodo (<i>all-nodes</i>)
FF01::2	Interfaz	Todos los routers en un nodo (<i>all-routers</i>)
FF02::1	Enlace	Todos los nodos del enlace (<i>all-nodes</i>)
FF02::2	Enlace	Todos los routers del enlace (<i>all-routers</i>)
FF02::5	Enlace	Routers OSFP
FF02::6	Enlace	Routers OSPF designados
FF02::9	Enlace	Routers RIP
FF02::D	Enlace	Routers PIM
FF02::1:2	Enlace	Agentes DHCP
FF02::1:FFXX:XXXX	Enlace	<i>Solicited-node</i>
FF05::2	Site	Todos los routers en un site
FF05::1:3	Site	Servidores DHCP en un site
FF05::1:4	Site	Agentes DHCP en un site
FF0X::101	Variado	NTP (<i>Network Time Protocol</i>)

Figura 36: Direcciones multicast permanentes. - IPv6.

Veamos, por último, dos casos especiales de direcciones multicast: las multicast ***Solicited Node***, y las multicast **derivadas de unicast**.

Las direcciones **multicast *Solicited Node*** identifican un grupo multicast del cual todos los nodos forman parte una vez que les es asignada una dirección unicast o anycast. Una dirección solicited-node se forma agregando al prefijo FF02::1:FF00:0000/104 los 24 bits más a la derecha del identificador de la interfaz. Para cada dirección unicast o anycast del nodo existe una dirección multicast solicited-node correspondiente. En las redes IPv6, la dirección solicited-node es utilizada por el protocolo de *Descubrimiento de Vecinos* para resolver la dirección MAC de una interfaz. Para ello se envía un mensaje *Neighbor Solicitation* a la dirección solicited-node.

Por otro lado, las direcciones multicast **basadas en direcciones unicast** fueron creadas para reducir el número de protocolos necesarios para la distribución de direcciones multicast. Para formar una de estas direcciones, se deben encender las banderas P y T, entonces el prefijo queda FF30::/12.

Entonces, por ejemplo, una dirección multicast basada en una dirección unicast con prefijo de red 2001:DB8::/32 queda FF3E:2001:DB8::CADE:CAFE, donde CADE:CAFE identifican al grupo dentro de esa red.

A modo de síntesis, las direcciones IPv6 y sus tipos se pueden representar como en la figura 37.

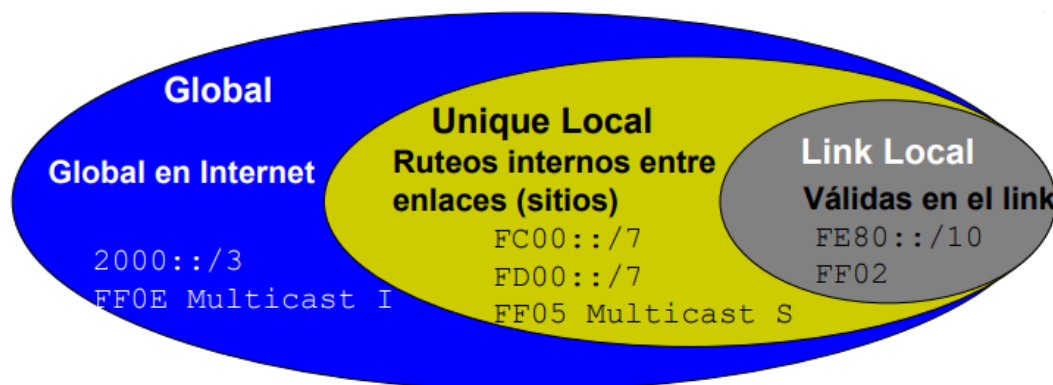


Figura 37: Scope de cada tipo de dirección IPv6. - IPv6.

4.5. ICMP

Hasta el momento vimos que IP proporciona un sistema de entrega de datagramas no confiable y sin conexión. A pesar de ser no confiable, el protocolo hace el mayor esfuerzo por entregar los datagramas, no los descarta porque sí, sin embargo hay circunstancias en las que no se puede entregar. A partir de esto se creó un protocolo que permite reportar errores e informar sobre circunstancias inesperadas. El **Protocolo de Mensajes de Control Internet (ICMP)** permite que routers y anfitriones envíen mensajes de error o de control hacia otros routers o anfitriones; el ICMP proporciona comunicación entre los software IP de dos máquinas distintas.

Tengamos en cuenta que ICMP es un mecanismo de **reporte de errores**, por lo que cuando un datagrama causa un error, sólo se puede reportar su condición a la fuente original, y es la fuente quien debe manejarlo. El protocolo no sabe corregir ni manejar errores, sólo los reporta a la fuente, ni siquiera puede reportarlo a routers intermedios.

4.5.1. Entrega de mensajes ICMP

Los mensajes ICMP requieren dos niveles de encapsulación. Esto es, cada mensaje ICMP (su encabezado y sus datos) viaja en la sección de datos de un datagrama IP, que a su vez, viaja dentro de una trama de red. Veremos en la capa de transporte que los mensajes de niveles más altos también viajan así, sin embargo ICMP no es un protocolo de nivel más alto que IP, sino que forma parte de él. Esta encapsulación se debe a que probablemente tenga que viajar por una red de redes, por lo que sólo con la dirección física no alcanza para que el mensaje llegue.

Los datagramas que llevan mensajes ICMP se rutean igual que el resto, no existe ni una confiabilidad ni una prioridad adicionales. Por lo tanto los mensajes ICMP se pueden perder o descartar. Además, para no congestionar aún más una red, si un datagrama que lleva mensajes ICMP se pierde, no se notifica ni se genera ningún mensaje de error nuevo.

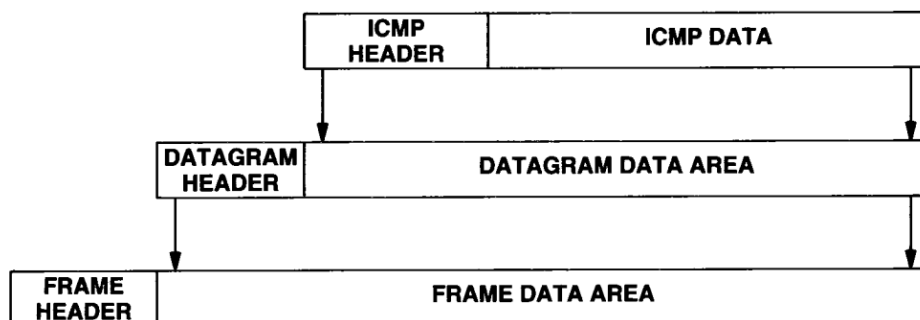


Figura 38: Encapsulado de un mensaje ICMP - Comer

4.5.2. Formato de los mensajes ICMP

Todo mensaje ICMP comienza con los mismos tres campos: un campo TIPO de 8 bits que identifica al mensaje, un campo CÓDIGO de 8 bits que proporciona más información sobre el tipo de mensaje, y un campo SUMA DE VERIFICACIÓN de 16 bits para garantizar la correctitud de los datos. Además, los que reportan errores incluyen el encabezado y los primeros 64 bits de datos del datagrama que causó el problema.

4.5.3. Usos de mensajes ICMP

Además de reportar errores, los mensajes ICMP sirven para verificar el correcto funcionamiento de una red. Para lograrlo se utiliza un mecanismo conocido como **ping**, que consiste en un envío de **solicitud de eco** del origen al destino. Una vez que el destino recibe una solicitud de eco, le responde con una **respuesta de eco**. Este mecanismo sirve para comprobar si un destino es alcanzable y si responde.

Uno de los reportes más comunes en ICMP es el de **destino no accesible**. Cuando un router no puede entregar un datagrama IP, envía uno de estos mensajes a la fuente original y descarta el datagrama. Otros reportes vinculados con la entrega del datagrama y no con el datagrama en sí son el de **ruta de origen** causado cuando un datagrama tiene una opción de ruta de origen con una ruta incorrecta, y el de **necesidad de fragmentación** que ocurre cuando un datagrama tiene activado el bit de “no fragmentar” pero el router necesita fragmentar sí o sí para entregarlo.

También hay mensajes ICMP no relacionados a errores, sino que orientados al control de flujo o de rutas de los datagramas. Ellos son: **disminución de tasa al origen** para reportar el congestionamiento a la fuente. Un mensaje de estos es una solicitud a la fuente para que reduzca la velocidad de transmisión de datagramas. En general, un router envía uno de estos mensajes por cada datagrama que recibe, pero como no hay un mensaje que realice la solicitud opuesta (que se aumente el flujo de datagramas), la fuente reduce la

velocidad de envío hasta que deje de recibir estos mensajes. Otro tipo de mensaje es el de **redireccionar**, que un router envía a un anfitrión para solicitarle que cambie su ruta ya que no es la óptima. Elegir una ruta óptima es importante ya que un datagrama puede no ser entregado por terminación de su tiempo de vida, debido al paso innecesario por muchos routers. Cuando el campo TIEMPO DE VIDA de un datagrama llega a cero, el router lo descarta y envía al origen un mensaje ICMP de **tiempo excedido**.

4.6. ICMPv6

Hasta el momento, estudiamos el protocolo ICMP para IPv4 (ICMPv4). Sin embargo, IPv6 también tiene su versión, llamada ICMPv6 que tiene las mismas funciones que la otra versión (informar características de la red, realizar diagnósticos, e informar errores en el procesamiento de paquetes), aunque no son compatibles. ICMPv6 no sólo agrega nuevas funcionalidades, sino que también (a diferencia de ICMPv4) es encargado de realizar las funciones del protocolo ARP.

4.6.1. Formato de un mensaje ICMPv6

En un paquete IPv6, el ICMPv6 se coloca último en la cadena de encabezados de extensión. El encabezado de todos los mensajes ICMPv6 tienen la misma estructura y está compuesto por cuatro campos, como se muestra en la figura 39:

Tipo	Código	Checksum
Datos		

Figura 39: Formato del encabezado ICMPv6. - [IPv6](#).

El **Tipo** (8 bits) especifica el tipo de mensaje; el **Código** (8 bits) ofrece datos adicionales para determinados tipos de mensajes; la **Suma de verificación** (16 bits) se utiliza para detectar datos corruptos en el encabezado ICMPv6 y en parte del encabezado IPv6; y los **Datos** (tamaño variable) presentan información de diagnóstico y error según el tipo de mensaje.

Los mensajes ICMPv6 se dividen en dos clases: mensajes de **error** y mensajes de **diagnóstico**. Sin embargo, vamos a concentrarnos en los más importantes.

4.6.2. Descubrimiento de vecinos

El protocolo de **Descubrimiento de Vecinos** (*Neighbor Discovery*) de IPv6 es utilizado por los hosts y routers para los siguientes propósitos: determinar la dirección MAC de los nodos de la red, encontrar routers vecinos, determinar prefijos y otros datos de configuración de la red, detectar direcciones duplicadas, determinar la accesibilidad de los routers, redireccionamiento de paquetes y autoconfiguración de direcciones. Para ello, se utilizan cinco mensajes ICMPv6 (entre paréntesis se aclara el código que lo identifica):

- **Router Solicitation (RS)** (133): Utilizado por los hosts para solicitar a los routers mensajes *Router Advertisements* inmediatamente. Normalmente se envía a la dirección multicast FF02::2 (all-routers on link).
- **Router Advertisement (RA)** (134): Se envía periódicamente o en respuesta a un *Router Solicitation* y es utilizado por los routers para anunciar su presencia en un enlace. Los mensajes periódicos se envían a la dirección multicast FF02::1 (all-nodes on link), mientras que los solicitados se envían directamente a la dirección del solicitante.
- **Neighbor Solicitation (NS)** (135): Mensaje multicast enviado por un nodo para determinar la dirección MAC y la accesibilidad de un vecino, y detectar la existencia de direcciones duplicadas. Este mensaje tiene un campo para indicar la dirección de origen del mensaje.
- **Neighbor Advertisement (NA)** (136): Se envía como respuesta a un *Neighbor Solicitation*, pero también se puede enviar para anunciar el cambio de alguna dirección dentro del enlace.
- **Redirect** (137): Utilizado por los routers para informar al host el mejor router para encaminar el paquete a su destino. Este mensaje contiene información como la dirección del router que se considera el mejor salto y la dirección del nodo que está siendo redireccionado.

Este protocolo (descubrimiento de vecinos) ofrece varias funcionalidades nombradas anteriormente, sin embargo, en la materia se estudian tres:

- **Descubrimiento de direcciones en capa de enlace:** Determina la dirección MAC de los vecinos del mismo enlace. Reemplaza al protocolo ARP, utilizado en IPv4. Para ello, utiliza la dirección multicast solicited-node, en lugar de la dirección broadcast. El host envía un mensaje NS informando su dirección MAC y solicita la dirección MAC del vecino. El vecino responde enviando un mensaje NA informando su dirección MAC.

- **Descubrimiento de routers y prefijos:** Localiza routers vecinos dentro del mismo enlace. Determina prefijos, parámetros relacionados con la autoconfiguración de direcciones, hop limit, MTU, etc. En IPv4 esta función es realizada por los mensajes ARP Request. Los routers envían mensajes RA a la dirección multicast all-nodes.
- **Autoconfiguración de direcciones stateless:** Permite atribuir direcciones IPv6 a las interfaces sin necesidad de realizar configuraciones manuales, ni utilizar servidores adicionales. Sólo utiliza una combinación de datos locales, como la dirección MAC de la interfaz o un valor aleatorio para generar el ID, e información recibida de los routers, como múltiples prefijos. En caso de que no haya routers, el host solo genera la dirección link local con prefijo FE80:::. El mecanismo de autoconfiguración de direcciones se ejecuta respetando los siguientes pasos:
 1. Se genera una dirección link-local agregando el prefijo FE80::/64 al IID;
 2. Esta dirección pasa a formar parte de los grupos multicast solicited-node y all-node;
 3. Se realiza la verificación de la unicidad de la dirección link-local generada. Si otro nodo del enlace ya está utilizando la misma dirección, se procede a realizar una configuración manual;
 4. Si la dirección se considera única y válida ésta será automáticamente inicializada para la interfaz;
 5. El host envía un mensaje *Router Solicitation* al grupo multicast all-routers;
 6. Todos los routers del enlace responden con un mensaje *Router Advertisement* informando routers por defecto, MTU del enlace, lista de prefijos de red, etc.

4.6.3. Path MTU Discovery (PMTUD)

Repasemos algunos conceptos previos: **MTU** (*Maximum Transmit Unit*) es el tamaño máximo de paquete que puede atravesar el enlace; y **fragmentación** es una técnica utilizada para enviar paquetes de tamaño mayor que la MTU del enlace. En IPv6, la fragmentación se realiza únicamente en el origen, por lo que se vuelve necesario saber el MTU de la red antes de enviar el paquete. Para eso existe un proceso propio de ICMPv6 llamado **Path MTU Discovery**, PMTUD, (o *Descubrimiento de MTU del Camino* pero suena mejor en ingles). El funcionamiento del PMTUD es el siguiente:

1. Asume que la máxima MTU del camino es igual a la MTU del primer salto.
2. Los paquetes mayores que el soportado por algún router a lo largo del camino se descartan.

3. Se devuelve un mensaje ICMPv6 *packet too big*.
4. Luego de recibir este mensaje el nodo de origen reduce el tamaño de los paquetes de acuerdo con la MTU indicada en el mensaje *packet too big*.
5. El procedimiento finaliza cuando el tamaño del paquete es igual o menor que la menor MTU del camino.
6. Estas iteraciones pueden ocurrir varias veces hasta encontrar la menor MTU.
7. Los paquetes enviados a un grupo multicast utilizan un tamaño igual a la menor PMTU de todo el conjunto de destinos

4.6.4. Calidad de servicio

El concepto de **QoS** (*Quality of Service*) o, en español, Calidad de Servicio, se utiliza en los protocolos cuya función es proporcionar la transmisión de determinados tráficos con prioridad y garantía de calidad. Actualmente existen dos arquitecturas principales: **Differentiated Services** (DiffServ) e **Integrated Services** (IntServ). A rasgos generales, IntServ se basa en la reserva de recursos (RSVP) aunque es complejo de implementar y no es escalable. Por otro lado, DiffServ utiliza el campo Traffic Class del encabezado IPv6. Además, no exige ninguna identificación ni gestión de los flujos y en general es más utilizado en las redes debido a su facilidad de implementación.

Bibliografía usada: Capítulo 5 (ARP), 7 (IPv4), 8 (Ruteo IP) y 9 (ICMP) de [Comer](#) para IP e IPv4. [IPv6.br](#) y [slides de la cátedra](#) para IPv6 e ICMPv6.

5. Capa de Transporte

5.1. Protocolo de Datagrama de Usuario (UDP)

Hasta el momento sabemos que en una red de redes TCP/IP cada datagrama se rutea basándose en la dirección IP de destino. Dado que en un sistema operativo existen procesos concurrentes, y cada uno puede necesitar comunicarse independientemente con otros, se vuelve necesario especificar tanto la dirección IP como el proceso puntual al cual se le envían los datagramas. Sin embargo, esto puede ser confuso ya que se crean y destruyen de forma dinámica. A partir de esto surge el concepto de **puertos de protocolo**, que consiste en puntos abstractos de destino identificados por un número entero positivo. Como estos puertos cuentan con una memoria intermedia, la transmisión de mensajes se vuelve asíncrona, ya que un proceso puede enviarle datos a otro sin que éste esté necesariamente esperándolo. A su vez, el receptor tiene un puerto designado al cual acudir en caso de esperar mensajes, que si aún no recibió, el sistema operativo lo pondrá en espera hasta que lleguen. Una vez que ese puerto recibe datos, el sistema se los envía al proceso y lo despierta para reanudar su ejecución. En resumen:

El Protocolo de Datagrama de Usuario (UDP por sus siglas en inglés) proporciona un servicio de entrega sin conexión y no confiable, utilizando el IP para transportar mensajes entre máquinas. Emplea el IP para enviar mensajes, pero agrega la capacidad para distinguir entre varios destinos dentro de una computadora anfitrión gracias a los puertos de protocolo.

Algunas características de UDP son:

1. Proporciona la misma semántica de entrega de datagramas, sin conexión y no confiable que el IP.
2. No emplea acuses de recibo.
3. No ordena los mensajes entrantes.
4. No proporciona retroalimentación para controlar la velocidad a la que fluye la información.

Un programa de aplicación que utiliza UDP acepta toda la responsabilidad por el manejo de problemas de confiabilidad, incluyendo la pérdida, duplicación y retraso de los mensajes, la entrega fuera de orden y la pérdida de conexión.

Cada mensaje de UDP se conoce como **datagrama de usuario**, el cual consiste en un encabezado (40) y un área de datos. Los cuatro campos del encabezado son: PUERTO

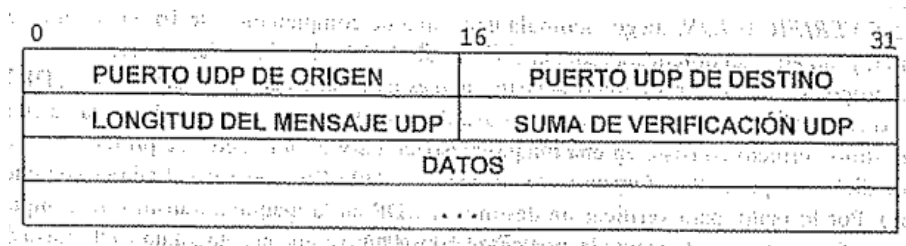


Figura 40: Encabezado UDP. - Comer

UDP DE ORIGEN (es opcional, si se utiliza, especifica el número de puerto al cual se envían las respuestas) y PUERTO UDP DE DESTINO se utilizan para el demultiplexado del datagrama; LONGITUD contiene un conteo de los bytes que ocupa el datagrama incluyendo el encabezado, por lo que el número mínimo es de ocho; y SUMA DE VERIFICACIÓN es opcional y se utiliza para garantizar que los datos llegaron intactos. En realidad la suma de verificación necesita otros datos para computarse, éstos datos se almacenan en un pseudo-encabezado UDP distribuidos como se muestra en 41. Si se prestó la suficiente atención, uno puede observar que el pseudo-encabezado de UDP incluye en uno de sus campos la dirección IP del dispositivo destino. Esto es una clara transgresión a la estratificación y separación en capas, ya que se rompe la abstracción entre la capa de red y la capa de transporte. Sin embargo, esta decisión fue tomada puramente por razones prácticas, ya que es imposible identificar plenamente un programa de aplicación de destino sin especificar la máquina de destino, por lo que lo pasamos por alto.

Estratificar por capas el UDP por encima del IP significa que un mensaje UDP completo

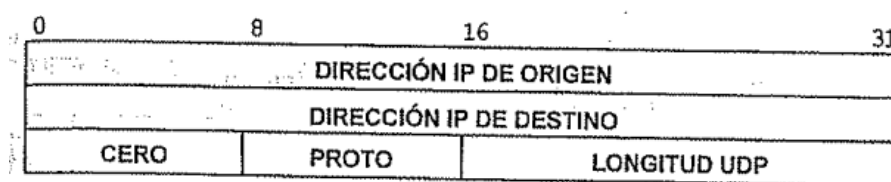


Figura 41: Pseudo-encabezado UDP. - Comer

se encapsula en un datagrama IP, y éste, a su vez, se encapsula en una trama Ethernet, de modo que un datagrama UDP se organiza como se muestra en la figura 42. De este modo, un paquete llega en la capa más baja y comienza a ascender a través de las capas superiores. En cada una se quita un encabezado, de modo que en la capa más alta se encuentran únicamente los datos que se desean enviar. Así, el encabezado exterior corresponde a la capa más baja del protocolo, y el interior a la más alta. El funcionamiento de UDP se puede resumir como:

La capa IP sólo es responsable de transferir datos entre un par de anfitriones

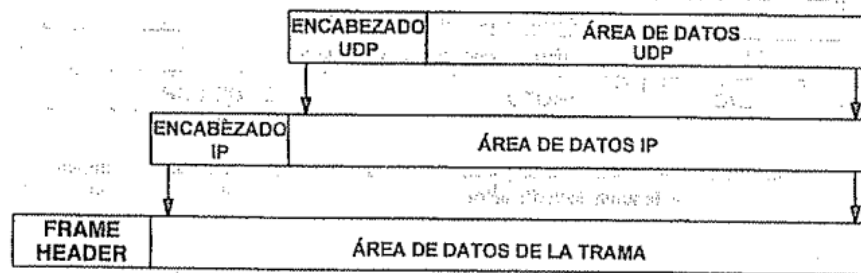


Figura 42: Datagrama UDP encapsulado en un datagrama IP para su transmisión. - Comer

dentro de una red de redes, mientras que la capa UDP solamente es responsable de diferenciar entre varias fuentes o destinos dentro de un anfitrión

Por lo tanto sólo el encabezado IP identifica los anfitriones de origen y destino, y la capa UDP sólo identifica los puertos de origen y destino dentro del anfitrión.

Una vez que un datagrama IP llega a su dirección IP de destino, UDP es el encargado de demultiplexarlo, basándose en el puerto de destino UDP, como se muestra en la figura 43. Al momento de asignar números de puerto podemos pensar en tres enfoques distintos: el

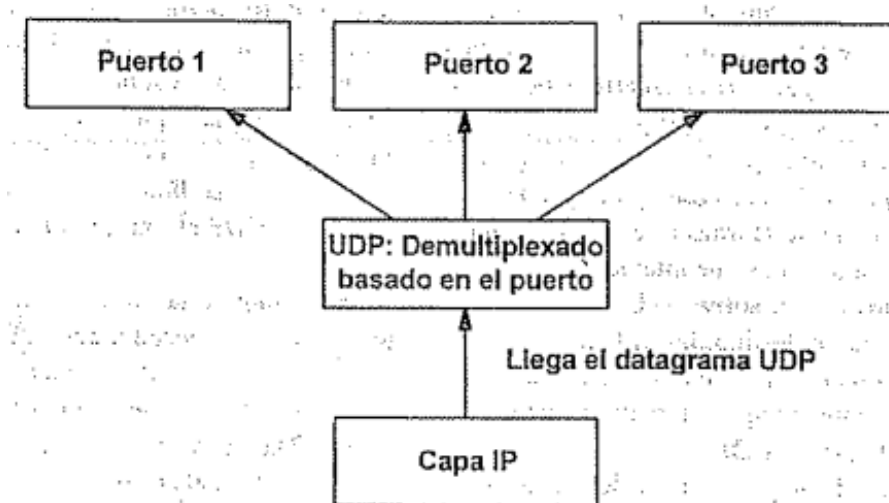


Figura 43: Ejemplo del demultiplexado de una capa sobre el IP. - Comer

enfoque universal, el enfoque dinámico y el enfoque híbrido. El enfoque universal consiste en una autoridad central encargada de asignar números de puerto conforme se necesitan y publicarlos en una lista de los que se llaman *puertos bien conocidos*, de modo que todo el software se diseña de acuerdo con esta lista. El segundo enfoque emplea la transformación dinámica, es decir que los puertos no se conocen de manera global, sino que cuando un programa necesita un puerto, el software de red le asigna uno disponible, por lo tanto es necesario enviar una consulta del puerto que se esté usando antes de enviar información.

El último enfoque es el utilizado por los diseñadores de TCP/IP, en el que se preasigna algunos números de puerto pero deja la mayoría disponibles para sitios locales o programas de aplicación.

5.2. Protocolo de Control de Transmisión (TCP)

Vimos que UDP proporciona una entrega de datagramas no confiable y sin conexión, lo que significa que al momento de comunicarse entre equipos no existe ninguna garantía de que los datos lleguen completos y ordenados, de hecho no hay forma de saber si llegaron siquiera. A partir de esta necesidad de una “confirmación” de que los datos llegaron sanos y salvos surge un nuevo protocolo llamado *Protocolo de Control de Transmisión* (TCP por sus siglas en inglés). El TCP, al igual que UDP, habita en la capa de transporte y sus mensajes también viajan encapsulados en datagramas IP. La diferencia sustancial con UDP es que TCP brinda una entrega de flujo confiable, es decir que los datos se envían de manera que se utiliza al máximo el ancho de banda, y a su vez, el emisor recibe una confirmación de que llegaron correctos, completos y en orden.

5.2.1. Características

Las características principales de TCP son:

1. **Orientación de flujo:** Cuando dos programas de aplicación transfieren grandes volúmenes de datos, pensamos en los datos como un flujo de bytes. El servicio de entrega de flujo en la máquina de destino pasa al receptor exactamente la misma secuencia que le pasa el transmisor en la máquina de origen.
2. **Conexión de circuito virtual:** Este protocolo está orientado a la conexión, es decir que las máquinas que quieren comunicarse, primero deben ponerse de acuerdo en que intercambiarán datos, análogo a una llamada. Es decir que ambos extremos interactúan con sus SO para indicar su necesidad de establecer una transferencia de flujo. Cuando se establece la conexión, se informa que ya está todo preparado y comienza la transferencia. Durante esta transferencia, el software de protocolo en las dos máquinas continúa comunicándose para verificar que los datos se recibían correctamente. Si en algún momento se corta la conexión, ambas máquinas lo detectan.
3. **Transferencia con memoria intermedia:** Los programas envían un flujo de datos a través del circuito virtual pasando repetidamente bytes de datos al software de protocolo. Cuando transfieren datos, cada aplicación utiliza piezas del tamaño que

encuentre adecuado, que pueden ser tan pequeñas como un byte. Para hacer eficiente la transferencia y minimizar el tráfico de red, las implementaciones por lo general recolectan, en una memoria intermedia, datos suficientes de un flujo para llenar un datagrama razonablemente largo antes de transmitirlo. De todas formas existe un mecanismo de empuje (*push*) que las aplicaciones usan para forzar una transferencia sin esperar a que se llene la memoria intermedia.

4. **Flujo no estructurado:** El servicio de flujo TCP/IP no está obligado a formar flujos estructurados de datos. Los programas de aplicación deben entender el contenido del flujo y ponerse de acuerdo sobre su formato antes de iniciar una conexión.
5. **Conexión Full Duplex:** Las conexiones TCP/IP permiten la transferencia concurrente en ambas direcciones. Una conexión full duplex consiste en dos flujos independientes que se mueven en direcciones opuestas, sin ninguna interacción aparente.

5.2.2. Confiabilidad

Dijimos que TCP ofrece un servicio de entrega de flujo confiable, lo que significa que garantiza que los datos enviados lleguen sin pérdida ni duplicación, sin embargo esto no es fácil de lograr. Si vamos al caso, los mensajes en TCP viajan en datagramas IP como con UDP, entonces ¿cómo hace el software del protocolo para brindar esta entrega confiable? La respuesta es con *acuses de recibo positivo de retransmisión*. Esta técnica consiste en que el receptor se comuniqué con el origen enviándole un mensaje de *acuse de recibo* (*ACK*) conforme recibe los datos. El transmisor guarda un registro de cada paquete que envía y arranca un temporizador para esperar un *acuse de recibo* antes de enviar el siguiente paquete. Si dicho temporizador termina antes de recibir el *ACK*, retransmite el paquete. Una representación simplificada se muestra en la figura 44.

En la figura 45 se utiliza el mismo diagrama que en la figura anterior para mostrar qué pasa cuando se pierde un paquete.

El problema final de confiabilidad surge cuando un sistema de entrega de paquetes los duplica ya que ahora no sólo los paquetes, sino también los *acuses de recibo*, se pueden duplicar. A grandes rasgos, lo que se hace es asignar a cada paquete un número de secuencia y los transmisores llevan un registro de los paquetes que enviaron y recibieron *acuse*. De esta manera se puede saber cuando se duplican paquetes o llegan en desorden.

5.2.3. Ventana deslizable

Habíamos dicho que TCP ofrece un servicio de entrega de flujo confiable. Ya explicamos que la confiabilidad la brinda los *acuses de recibo*, veamos ahora cómo se logra la entrega de

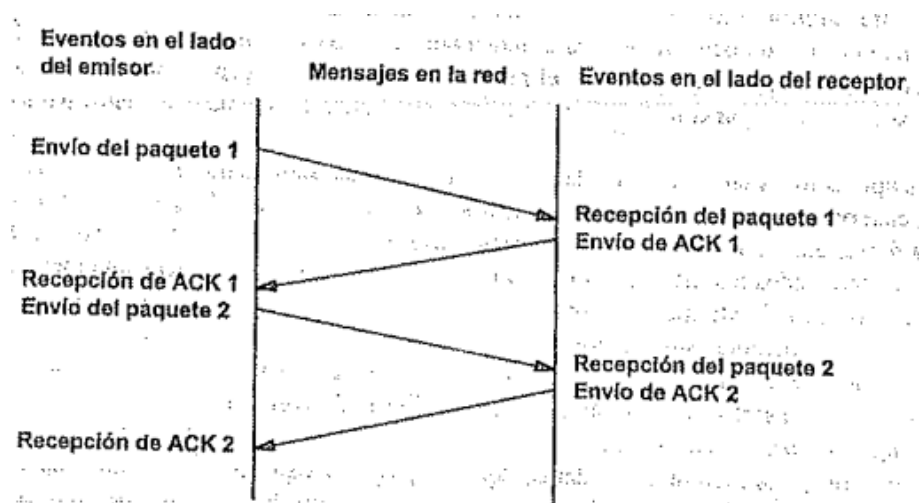


Figura 44: Protocolo con acuses de recibo positivos con retransmisión. La distancia vertical representa el incremento en el tiempo y las líneas que cruzan en diagonal representan la transmisión de paquetes. - Comer.

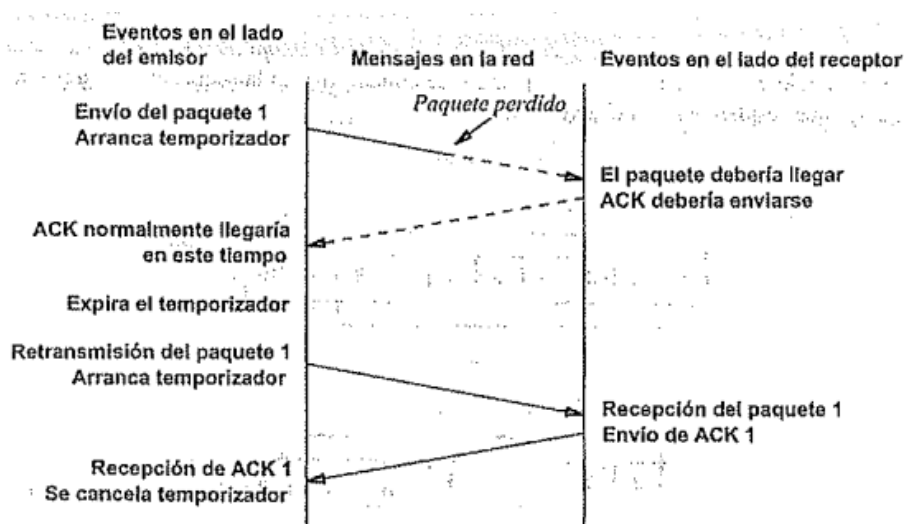


Figura 45: Tiempo excedido y retransmisión que ocurre cuando un paquete se pierde. - Comer.

flujo. Para eso presentamos un concepto nuevo llamado *ventana deslizante*. Para entender el porqué utilizar ventanas deslizables volvamos a la figura 44. Cuando se enviaba un mensaje, el transmisor debía esperar hasta recibir el acuse de recibo para continuar con el envío. Esto generaba que no se aproveche el ancho de banda de la red y que la comunicación sea *relativamente* lenta.

La técnica de la ventana deslizante consiste en delimitar cierta cantidad de paquetes N , estos N paquetes se envían uno atrás del otro sin esperar ningún acuse de recibo. A medida que comienzan a llegar los acuses de recibo, se continúa el envío de los paquetes

subsecuentes. Lo que logra esto es que se aproveche mucho más el ancho de banda, ya que como la comunicación es *full duplex*, se puede estar recibiendo acuses de recibo a la vez que se continúa enviando paquetes. La figura 46 nos muestra una representación de una ventana de tamaño 8, esto quiere decir que se envían 8 paquetes a la vez, entonces cuando llegue el acuse de recibo del primer paquete, la ventana se mueve una posición y se envía el noveno. Veamos que con un tamaño de ventana de 1, un protocolo de ventana

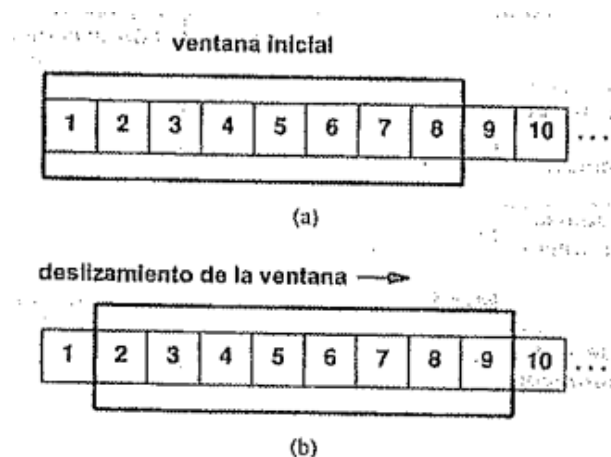


Figura 46: (a) Ventana inicial de tamaño 8, se envían los paquetes del 1 al 8. (b) La ventana se desliza al paquete 9 una vez que se recibió el acuse del paquete 1. - Comer

deslizable sería idéntico a un protocolo simple de acuse de recibo positivo. Al aumentar el tamaño de la ventana, es posible eliminar por completo el tiempo ocioso de la red. La idea principal es que:

Como un protocolo de ventana deslizable bien establecido mantiene la red completamente saturada de paquetes, con él se obtiene una generación de salida substancialmente más alta que con un protocolo simple de acuse de recibo positivo.

Un protocolo de ventana deslizable recuerda qué paquetes tienen acuse de recibo y mantiene un temporizador separado para cada paquete sin acuse. Si se pierde un paquete, el temporizador concluye y se reenvía el paquete. Cuando se desliza la ventana, se mueven hacia atrás todos los paquetes con acuse. En el extremo receptor, el software de protocolo mantiene una ventana análoga que acepta y acuse como recibidos los paquetes a medida que llegan. El paquete con menor número en la ventana es el primer paquete en la secuencia para el que no se ha hecho un acuse de recibo. La figura 47 muestra un ejemplo de una transmisión con una ventana deslizable de tamaño 3.

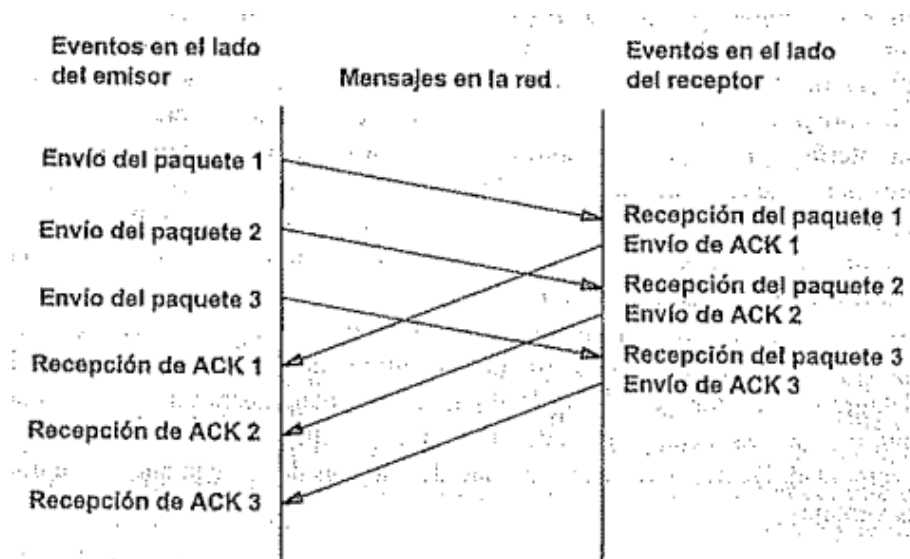


Figura 47: Ejemplo de tres paquetes transmitidos con un protocolo de ventana deslizante.
- Comer.

5.2.4. Protocolo de Control de Transmisión

Ya que entendimos el concepto de acuse de recibo positivo y el de ventana deslizante, estamos listos para examinar el servicio de flujo confiable proporcionado por el grupo de protocolos de TCP/IP de Internet. Primero que nada, tenemos que entender que:

El TCP es un protocolo de comunicación, NO una pieza de software.

Podemos resumir los alcances del protocolo en tres puntos claves. El protocolo especifica:

1. El formato de datos y los acuses de recibo que intercambian dos computadoras para lograr una transferencia confiable, así como los procedimientos necesarios para asegurarse que los datos lleguen de manera correcta.
2. Cómo el software TCP distingue el correcto entre muchos destinos en una misma máquina, y cómo las máquinas en comunicación resuelven errores como la pérdida o duplicación de paquetes.
3. Cómo dos computadoras inician una transferencia de flujo y cómo se ponen de acuerdo cuando se completa.

5.2.5. Puertos, conexiones y puntos extremos

Sabemos que TCP, al igual que UDP, reside en la capa de transporte en el esquema de estratificación por capas de protocolos, y que ambos utilizan el concepto de puerto de

protocolo para lograr que varios programas en una misma máquina se comuniquen con otras. Sin embargo, cada protocolo hace un uso distinto de estos puertos. Cuando vimos UDP (5.1) dijimos que los puertos eran como colas de salida en las que el software de protocolo coloca los datagramas entrantes, por lo que podíamos pensar que el destino final era el puerto en sí, pero en TCP no es así ya que un número de puerto no corresponde a un solo objeto, sino que forma parte de la identificación de una conexión de circuito virtual. Como no se entendió nada vamos a explicarlo de a poco.

Entender que TCP utiliza la noción de conexiones es crucial porque nos ayuda a explicar el uso que hace de los números de puerto.

El TCP utiliza la conexión, no el puerto de protocolo, como su abstracción fundamental. Las conexiones se identifican por medio de un par de puntos extremos.

Entonces surge la pregunta ¿qué son esos “puntos extremos”? Los *puntos extremos* son un par de números enteros (*dirección, puerto*), donde la *dirección* corresponde a la dirección IP de la máquina, y *puerto* al puerto de protocolo usado en esa máquina. Una vez comprendido ese concepto se hace mucho más fácil entender lo que habíamos dicho antes. En TCP una conexión se define por sus dos puntos extremos, es decir, dos pares (*dirección, puerto*). Esto quiere decir que una misma computadora puede tener múltiples conexiones usando sólo un puerto.

5.2.6. Aperturas pasivas y activas

TCP es un protocolo orientado a la conexión, el cual requiere que ambos puntos extremos estén de acuerdo en participar. Se puede hacer una analogía entre la forma en la que dos computadoras establecen una conexión TCP y una llamada telefónica entre dos personas. Como primer paso, el programa de aplicación de un extremo realiza una función de **apertura pasiva**, indicando que aceptará una conexión entrante (análogo a sentarse al lado del teléfono, a la espera de una llamada). En ese momento, el SO asigna un número de puerto TCP a su extremo. El programa de aplicación en el otro extremo debe solicitar una **apertura activa** (agarrar el teléfono y llamar). Los dos módulos de software TCP se comunican para llevar a cabo la conexión (llamada), y una vez establecida, se comienza con la transferencia de datos. Durante este intercambio, los módulos de software TCP en cada extremo se encargan de garantizar la entrega confiable. Cuando veamos el [formato](#) de los mensajes TCP vamos a poder seguir profundizando un poco más este tema.

5.2.7. Segmentos, flujos y números de secuencia

El TCP visualiza el flujo de datos como una secuencia de bytes que divide en segmentos que, por lo general, viajan como un solo datagrama IP.

Es importante entender que el mecanismo TCP de ventana deslizante opera a nivel de byte, no a nivel de segmento ni de paquete. Los bytes del flujo se numeran de manera secuencial, y el transmisor guarda tres apuntadores asociados con cada conexión, los cuales definen una ventana deslizante como se ve en la figura 48. El primero marca el extremo izquierdo de la ventana, separa los bytes enviados que ya recibieron acuse de los que no; el segundo marca el extremo derecho de la ventana, y define el byte más alto en la secuencia que se puede enviar antes de recibir más acuses; el tercero señala la frontera dentro de la ventana entre los bytes que ya se enviaron de los que todavía no. El software envía todos los bytes que estén entre el primer y segundo apuntador casi sin retraso, lo que hace que el tercer apuntador se mueva rápido de izquierda a derecha.

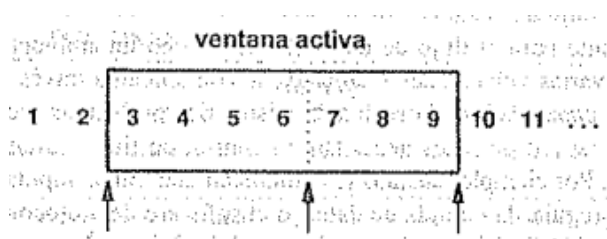


Figura 48: Representación de una ventana deslizante de tamaño 7. Los bytes 1 y 2 ya fueron enviados y acusados; los del 3 al 6 se enviaron pero aún no se recibió su confirmación; los del 7 al 9 aún no se enviaron pero se enviarán sin retardo; y los del 10 en adelante no se pueden enviar hasta no recibir acuse de los ya enviados, es decir, hasta que no se mueva la ventana. - Comer.

Sabemos que el transmisor tiene una ventana como la descrita, y también sabemos que el receptor debe tener una ventana análoga para poder verificar que los datos lleguen completos y reordenarlos en caso que sea necesario. Sin embargo, las conexiones TCP son *full duplex*, lo que significa que el receptor también puede enviarle datos al transmisor y éste debe responderle con acuses de recibo, lo que implica que ambos tengan una ventana más. Por lo tanto, en una conexión TCP, cada extremo tiene dos ventanas deslizantes (sumando un total de cuatro por conexión), una se desliza a lo largo de los datos que envía, y otra se desliza a lo largo de los datos que recibe.

5.2.8. Tamaño variable de ventana y control de flujo

La ventana deslizante que mencionamos hasta ahora es una versión simplificada de la que se utiliza realmente, ya que TCP permite que el tamaño de la ventana varíe para regular

el flujo de datos.

En cada acuse de recibo se informa cuántos bytes se recibieron y cuántos bytes adicionales está preparado a aceptar el receptor. Este segundo valor se lo conoce como *aviso de ventana*. Como los anuncios de ventana acompañan a los acuses de recibo, el transmisor reajusta el tamaño de su ventana al momento en que se mueve hacia adelante.

Lo bueno de usar una ventana de tamaño variable es que ésta proporciona control de flujo y una transferencia confiable, ya que si la memoria intermedia del receptor se llena, se puede enviar un anuncio de ventana más pequeño. En casos extremos, el receptor anuncia un tamaño de ventana igual a cero para tener la transmisión. Una vez que se libere un poco la memoria intermedia, el receptor anuncia un tamaño de ventana mayor a cero para reanudarla.

Otra ventaja que ofrece este tipo de ventanas es que regula la transferencia de datos entre computadoras (y redes) de distintas capacidades y velocidades, dando un control extremo a extremo del flujo para evitar congestionamientos o retardos altos.

A pesar de las ventajas que ofrece utilizar una ventana variable, existe un problema que surge al utilizarlas. Supongamos que en una conexión, el receptor decide leer los datos de entrada de a un byte a la vez. También supongamos que al comenzar la conexión, TCP asigna, por defecto, una ventana de tamaño mayor a 1, y que el emisor genera datos rápidamente. Entonces el emisor transmitirá segmentos con datos para toda la ventana y el receptor le enviará un acuse de recibo diciendo que su ventana está llena.

Una vez que el receptor procese el primer byte de su memoria intermedia, enviará otro acuse de recibo indicando que ahora puede recibir un byte. El emisor entonces, procederá a enviar un datagrama con un sólo byte. Notemos que este comportamiento genera que la transmisión sea sumamente ineficiente, ya que se enviaría de a un byte por vez. Enviar segmentos pequeños ocupa ancho de banda innecesariamente e introduce una sobrecarga computacional, teniendo en cuenta que para enviar cada segmento se debe crear su encabezado y encapsularlo en un datagrama IP, que a su vez tiene otro encabezado.

A este fenómeno se lo conoce como el ***síndrome de la ventana tonta***. En las primeras versiones de TCP significó un gran problema ya que era difícil de tratar. Hoy en día existen formas de prevención:

- **Del lado del receptor:** antes de enviar el anuncio de una ventana actualizada, luego de anunciar una ventana igual a 0, esperar hasta que se obtenga un espacio disponible equivalente a por lo menos el 50 % del tamaño total de la memoria intermedia o igual al segmento de tamaño máximo.
- **Del lado del emisor:** cuando una aplicación de emisión genera datos adicionales para enviarse sobre una conexión por la que se han transmitido datos anteriormente, pero de los cuales no hay un acuse de recibo, debe colocarse los datos nuevos en la

memoria intermedia de salida como se hace normalmente, pero no enviar segmentos adicionales hasta que se reúnan suficientes datos para llenar un segmento de tamaño máximo. Si todavía se está a la espera cuando llega un acuse de recibo, debe enviarse todos los datos que se han acumulado en la memoria intermedia. Aplíquese la regla incluso cuando el usuario solicita la operación de empujar.

5.2.9. Formato del segmento TCP

La unidad de transferencia en TCP se conoce como *segmento*. Debido a que TCP utiliza acuses de recibo incorporados, un acuse que viaja de la máquina A a la máquina B puede viajar en el mismo segmento en el que van los datos de la máquina A a la máquina B, aún cuando el acuse de recibo se refiera a datos enviados de B hacia A. En la figura 49 se muestra el formato del segmento TCP. Cada segmento se divide en encabezado y datos.

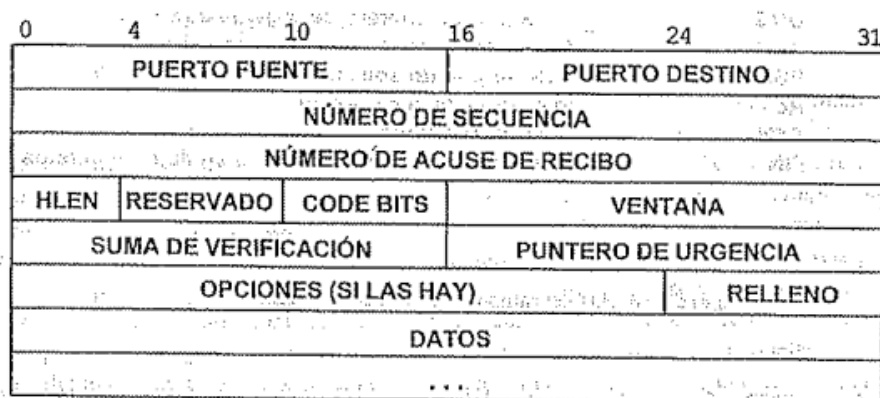


Figura 49: Formato de un segmento TCP con un encabezado TCP seguido de datos. - Comer.

El encabezado transporta la identificación y la información de control. Veamos para que sirve cada campo:

- Los campos PUERTO FUENTE y PUERTO DESTINO guardan los puertos TCP de cada extremo.
- El campo NÚMERO DE SECUENCIA indica la posición de los datos en el flujo del transmisor, y el campo NÚMERO DE ACUSE DE RECIBO indica el número de bytes que la fuente espera recibir después. Es decir que el número de secuencia se refiere al flujo que va en la dirección del mensaje, y el número de acuse de recibo va en la dirección contraria, en la que llegan datos.
- El campo HLEN contiene la longitud del encabezado del segmento, medida en múltiplos de 32 bits. Es necesario porque el campo OPCIONES varía en su longitud.

- El campo CODE BITS indica el propósito y contenido del segmento (acuse de recibo, solicitud para establecer una conexión, datos, etc.). Los posibles valores que puede tomar este campo son URG (datos urgentes), ACK (acuse de recibo), PSH (operación push), RST (inicio de conexión), SYN (sincronizar números de secuencia) y FIN (fin del flujo). Cuando se envían datos considerados *urgentes* (por ejemplo, una interrupción por teclado en un servicio de escritorio remoto), se utiliza el código URG y en el campo PUNTERO DE URGENCIA se especifica la posición dentro del segmento en la que terminan los datos urgentes. Este tipo de datos se los considera *datos fuera de banda*.
- El campo VENTANA especifica cuántos datos está dispuesto a aceptar, es decir, el tamaño de su memoria intermedia. Este campo tiene un número entero sin signo de 18 bits.
- El campo SUMA DE VERIFICACIÓN contiene una suma de números enteros de 16 bits que se utiliza para corroborar la integridad de los datos y del encabezado TCP. Para ello se utiliza un pseudo-encabezado al igual que en UDP, agrega bits en cero hasta que el segmento sea un múltiplo de 16 bits y calcula la suma sobre todo el resultado. Tanto el pseudo-encabezado como el relleno no se cuenta para la suma ni se transmite.
- El campo OPCIONES sirve para dar información extra dependiendo del mensaje que se esté enviando. Uno de sus usos más importantes es especificar el *tamaño máximo de segmento (MSS)* que está dispuesto a recibir. Este tamaño se establece al momento de entablar la conexión, a diferencia del tamaño de ventana que se actualiza dinámicamente. Encontrar un tamaño de segmento apropiado es importante ya que uno muy pequeño implicaría que no se aproveche el ancho de banda de la red, mientras que un tamaño muy grande puede generar que el datagrama IP en el que esté viajando el segmento se fragmente, lo cual lleva a problemas como que un fragmento no se confirme y se tenga que retransmitir todo el segmento.

5.2.10. Acuses de recibo y retransmisión

Para entender mejor cómo funcionan los acuses de recibo, tengamos en cuenta tres puntos que vimos hasta el momento: primero, que el receptor mantiene una ventana deslizante análoga a la del transmisor; segundo³, que un accuse puede viajar en un datagrama junto a otros datos; y tercero, que ambos extremos de la conexión cuentan con una memoria intermedia.

³Francia

En una conversación entre dos computadoras, el receptor recolecta bytes de datos de los segmentos entrantes y reconstruye una copia exacta del flujo enviado utilizando los números de secuencia. Una vez reconstruido el mensaje (o una parte de él), el receptor acusa el prefijo contiguo más largo del flujo que se haya recibido correctamente. Podemos resumir la idea como:

Un accuse de recibo TCP especifica el número de secuencia del siguiente octeto que el receptor espera recibir.

Se dice, entonces, que el esquema TCP de accuse de recibo es *acumulativo*, ya que reporta cuánto se ha acumulado del flujo. El problema de esto es que el receptor del accuse no obtiene información sobre todas las transmisiones exitosas, lo que hace que sea menos eficiente.

Veamos un ejemplo para explicarlo: Supongamos que tenemos una ventana de tamaño 500 comenzando en la posición 101 en el flujo, y que se enviaron todos los datos de la ventana en 4 segmentos (del 101 al 200, del 201 al 300, del 301 al 400 y del 401 al 500). Supongamos también que se pierde el primer segmento pero los otros tres llegan bien. Por cada segmento que llega, el receptor enviará un accuse con el byte 101, que es el siguiente byte que espera recibir. Notemos que no hay forma que el receptor indique que llegó todo menos el primero.

Cuando sucede esto, el transmisor tiene dos alternativas, ambas posiblemente ineficientes:

1. Retransmitir los cuatro segmentos: cuando llegue el primero, el receptor tendrá todos los datos en la ventana, y en el accuse de recibo aparecerá 501. El resto de los segmentos se descartan. Notemos que esta opción es la menos eficiente para este caso.
2. Retransmitir sólo el primer segmento: al hacerlo, deberá esperar su accuse para saber con qué datos continuar la comunicación. Esta opción es la establecida por el estándar actual, que si bien en este caso es la más eficiente, puede no serlo si se perdieron los otros segmentos, ya que vuelve a ser un protocolo simple de accuse de recibo positivo y pierde las ventajas de tener una ventana.

5.2.11. Medición precisa de muestras de viaje redondo

Calcular el tiempo de viaje promedio de los mensajes TCP es necesario para saber si un mensaje enviado se perdió y tiene que ser retransmitido o no. Para eso, TCP originalmente utilizaba un *algoritmo adaptable de retransmisión* con el que calculaba dinámicamente su noción de tiempo de viaje redondo promedio para una conexión dada, basándose en la

diferencia entre la hora de salida y la hora de llegada de un mensaje, conocido como *tiempo ejemplo de viaje redondo* y el promedio que iba llevando en cada momento. Así computaba el tiempo estimado de viaje redondo, o RTT (round trip time).

$$RTT = (\alpha * Old_RTT) + ((1 - \alpha) * Nueva_muestra) \quad (4)$$

La ecuación 4 muestra el cálculo del promedio según una nueva muestra, α está entre 0 y 1. Si α está más cerca del 0, se ponderará más la nueva medición frente al promedio anterior. Si α está más cerca del 1, es al revés.

En base al RTT, computaba un valor de *terminación de tiempo*. Obviamente este valor debía ser mayor o igual al promedio de viaje, sino la mayoría de los mensajes serían retransmitidos aún cuando se hayan enviado exitosamente. Esta computación se realizaba según la siguiente ecuación (5), donde β es mayor a 1. El valor de β cuan rápida es la detección de pérdida de paquetes. Si su valor es cercano a 1, el TCP se vuelve muy ansioso, y cualquier retraso mínimo puede llevar a una retransmisión innecesaria. Se solía usar un β de 2.

$$Temporizador = \beta * RTT \quad (5)$$

Sin embargo hay que tener algo más en cuenta: ¿Que pasa si se retransmite un mensaje por terminación de tiempo y llega su acuse de recibo? Teniendo en cuenta que como es el mismo datagrama, tanto el original como el retransmitido llevan los mismos datos. Este fenómeno se conoce como *ambigüedad de acuse de recibo* ya que no hay forma de saber a cuál corresponde.

Entonces surge la pregunta: ¿debe TCP asumir que el acuse pertenece al primer mensaje o al segundo? Sorprendentemente, ninguna de las dos opciones funciona. Supongamos que los datagramas viajan en una red de redes que pierde muchos datagramas. Si asocia los acuses de recibo a la transmisión original, el RTT aumentaría sin medida ya que tomaría como un viaje a lo que en realidad son (probablemente) dos o más. Al tener un RTT tan grande, TCP esperaría datagramas ya perdidos durante tiempos demasiado grandes. Por otro lado, asociar los acuses con la última retransmisión implica que el RTT será cada vez más chico, lo que genera que espere poco los acuses y retransmita innecesariamente, malgastando ancho de banda.

La solución que se encontró para este fenómeno fue no tener en cuenta los mensajes retransmitidos para la actualización del RTT, de esta manera no hay ambigüedad en los acuses, ya que sólo se consideran los acuses de los segmentos que se transmitieron una sola vez. Esta idea es conocida como *algoritmo de Karn*. Para que funcione correctamente, este algoritmo necesita que el transmisor combina las terminaciones de tiempo de transmisión con una estrategia de *anulación del temporizador*. Esta técnica computa una terminación de tiempo inicial por medio de una fórmula. Sin embargo, si se termina el tiempo y se pro-

voca una retransmisión, el TCP aumenta el valor de terminación de tiempo. Resumiendo, el algoritmo de Karn funciona de la siguiente manera:

Cuando se compute la estimación de viaje redondo (RTT), ignorar los ejemplos que correspondan a los segmentos retransmitidos, pero utilizar una estrategia de anulación, y mantener el valor de terminación de tiempo de un paquete retransmitido para los paquetes subsecuentes, hasta que se obtenga un ejemplo válido.

5.2.12. Respuesta al congestionamiento

El congestionamiento es una condición de retraso severo causada por una sobrecarga de datagramas en uno o más puntos de conmutación (por ejemplo, en routers). Si no se trata adecuadamente, el comportamiento de TCP descrito hasta ahora sólo empeoraría la situación en caso de uno ya que los datagramas tardarían más en llegar, lo que generaría la retransmisión y, por consiguiente, un mayor uso del ancho de banda. Para evitarlo, existen dos técnicas complementarias: el *arranque lento* y la *disminución multiplicativa*. Por otro lado, además del tamaño de ventana del receptor que el transmisor debe recordar, existe una segunda ventana llamada *ventana de congestionamiento*, que implica un segundo límite. Ahora sí podemos explicar las dos estrategias de prevención:

- **Prevención por Disminución Multiplicativa:** Cuando se pierda un segmento, reducir a la mitad la ventana de congestionamiento. Para los segmentos que permanezcan en la ventana permitida, anular exponencialmente el temporizador para la retransmisión.
- **Recuperación de arranque lento (aditiva) :** siempre que se arranque el tráfico en una nueva conexión o se aumente el tráfico después de un periodo de congestionamiento, activar la ventana de congestionamiento con el tamaño de un solo segmento, y aumentarla un segmento cada vez que llegue un acuse de recibo.

Para evitar el aumento demasiado rápido del tamaño de la ventana, hay una restricción más. Una vez que la ventana de congestionamiento llega a la mitad de su tamaño original, el TCP entra en una fase de *prevención de congestionamiento* y hace más lenta la velocidad de incremento. Durante esta fase, aumenta el tamaño de la ventana por 1 solamente si, para todos los segmentos en la ventana, se tiene acuses de recibo.

5.2.13. Establecimiento de una conexión TCP

Ahora que ya vimos los campos de la cabecera de un segmento TCP, podemos profundizar un poco más en cómo se establece y cierra una conexión TCP.

Para establecer una conexión TCP, se realiza un saludo en tres etapas, representado en la figura 50.

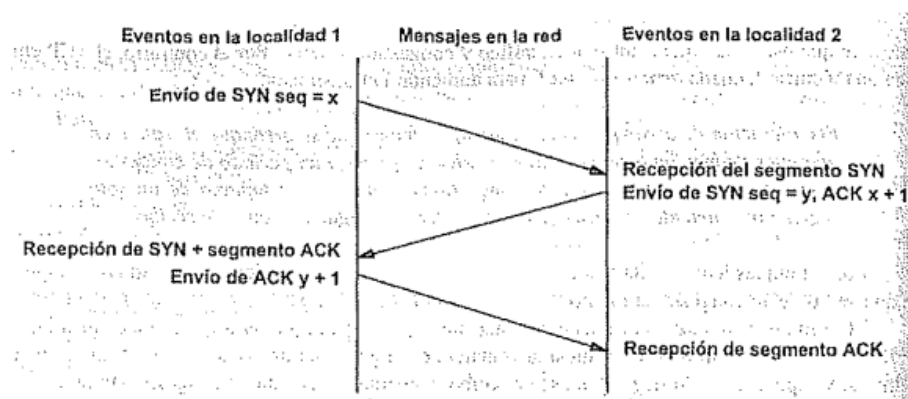


Figura 50: Secuencia de mensajes de saludo de tres etapas en TCP. - Comer.

El primer segmento se identifica gracias a que tiene activo el campo SYN. El segundo mensaje tiene tanto el SYN (porque sigue en el proceso de sincronización) como el ACK (porque es un acuse de recibo). El último mensaje tiene sólo el campo ACK activado, ya que es un acuse de recibo que indica que ambas partes están de acuerdo con la conexión. Por lo general, el software TCP de una máquina espera conexiones de forma pasiva todo el tiempo. Además, el saludo está diseñado de tal forma que si ambos extremos intentan establecer la conexión con el otro al mismo tiempo, igual lo logran. Son necesarios los tres pasos ya que, como TCP está basado en un sistema confiable, se necesita que ambos lados reciban una confirmación de que la conexión está establecida correctamente.

Además de establecer la conexión, el saludo de tres pasos sirve para acordar un número de secuencia inicial. El número de secuencia se genera aleatoriamente y sirve para identificar la conexión. En la figura anterior se ve que extremo de la localidad 1 le envía su número de secuencia ($seq = x$). El otro extremo, siguiendo la idea de enviar en el acuse de recibo el próximo byte que se espera recibir, le responde un ACK con $x + 1$, y además le envía su propio número de secuencia inicial ($seq = y$). Por último, el primer participante le envía un acuse de recibo con ACK $y + 1$ para indicar que su mensaje fue recibido con éxito.

5.2.14. Terminación de una conexión TCP

Para terminar una conexión TCP sin errores, se emplea una versión modificada del saludo de tres etapas. Como las conexiones TCP son *full duplex*, deben cerrarse ambos lados de

la conexión. Uno de los extremos, cuando no tiene más datos para transmitir, le envía un mensaje al destino con el bit FIN encendido (en lugar de SYN) para indicar que no hay mas datos por enviar, junto con un número de secuencia. Al hacer eso, el origen cierra la conexión en una de la direcciones, y el otro, envía un acuse de recibo, incrementando en uno el número de secuencia recibido. Luego de eso, el que recibió el mensaje FIN sigue transmitiendo todos los datos que desee. Una vez que haya transmitido todos, se realiza el mismo procedimiento para poder cerrar la otra dirección de la conexión. Una vez que se hayan cerrado ambos lados de la conexión, el software TCP de cada extremo borra los registros de la conexión.

En la figura 51 se muestra este procedimiento.

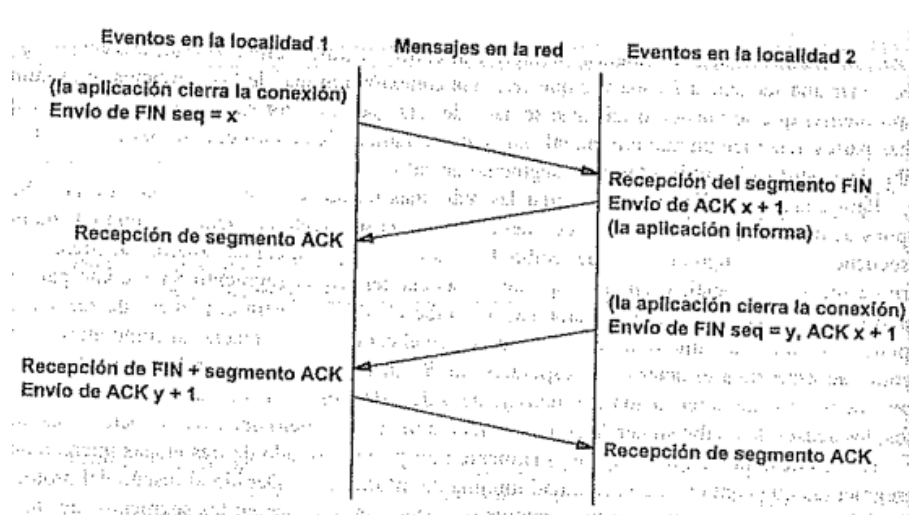


Figura 51: Versión modificada del saludo en tres pasos para lograr el cierre de conexiones TCP. (Disculpen la calidad) - Comer.

6. Capa de Aplicación

¡Llegamos! Después de una larga caminata a través de distintas capas del modelo OSI (y TCP/IP) (La tierra media), llegamos por fin a la última capa que veremos, la capa de aplicación (Mordor). ¡A no desesperar! Todavía hay esperanzas de terminar de entender la materia y sobrevivirla (tirar el Anillo en el Monte del Destino).

De entre todos los protocolos de alto nivel que ofrece esta capa, nosotros nos concentraremos en el protocolo de resolución de nombre de dominio, DNS, y en el protocolo de firewall.

6.1. Domain Name Server (DNS)

6.1.1. Servidores de Nombre (NS)

Nosotros sabemos por lo que leímos en la unidad 4 que dos máquinas en una red se comunican por medio de su dirección IP. Hoy en día existen millones de máquinas y páginas web en Internet, por lo que si quisiésemos conectarnos con alguna de ellas deberíamos saber las direcciones IP de todas, algo que es imposible. Para solucionar esto, entran a la ecuación los **servidores de nombre** (NS, *Name Server*), cuyo trabajo es traducir (o mapear) el nombre de un recurso a su dirección IP (esto se conoce como *Forward Mapping*) y traducir de una IP física al nombre de un recurso (*Reverse Mapping*). Cuando escribimos el nombre de una página web en internet (www.example.com, por ejemplo) el sistema de resolución de nombre se encarga de conseguir su dirección IP (x.x.x.x).

Es fácil ver como los servidores de nombre han tomado un protagonismo clave en las comunicaciones en Internet, por lo que se creó el concepto de servidores primarios y secundarios. Estos últimos responderán en los casos que el servidor primario falle y no pueda responder.

6.1.2. Ahora sí, DNS

El problema que surge en la solución anterior es la falta de **escalabilidad**. Cada vez hay más y más usuarios y páginas web, esto genera que cada vez haya más nombres en nuestros servidores de nombre, causando 3 problemas en particular:

1. Como hay más nombres, encontrar una entrada particular en una base de datos de nombre se vuelve tedioso, teniendo que recorrer millones de nombres hasta encontrar el buscado. Tenemos que conseguir alguna manera de indexar u organizar las cosas.
2. Si TODOS los hosts acceden a los mismos servidores de nombre puede causar una carga muy alta en los servidores. Habría que lograr dividir la carga entre varios.

3. Muchas veces pasa que se vuelve difícil manejar los nombres, ya que pueden haber muchos usuarios accediendo e intentando modificar los registros al mismo tiempo. Capaz, logrando separar la administración de estos registros de nombre podemos solucionar esto.

Con todos estos problemas en mente, se creó el **DNS** (*Domain Name Server*), que vendría a ser una implementación específica del concepto de servidores de nombre optimizado para prevalecer a las condiciones del internet. Como lo dice su nombre, los DNS usan un sistema de **dominios**, donde cada dominio es responsable por todos los registros que habitan en él. A su vez, los dominios están divididos en distintas jerarquías, formando una especie de estructura de árbol. En un nombre, los dominios se separan con un punto ('.'). En la raíz de ese árbol se encuentra el dominio **root** (Este se representa como un punto ('.')) silencioso, todas las direcciones lo tienen). El tipo de dominio que lo sigue son los **TLDs** (*Top Level Domains*), que originalmente se dividían en dos clases, los **gTLDs** (*Generic Top Level Domains*) donde viven dominios como *.com*, *.net*, *.edu* y los **ccTLDs** (*Country Code Top Level Domains*) que engloban a los dominios dedicados a países, utilizando una secuencia de dos letras para identificarlos. Ejemplos de estos pueden ser *.ar*, *.us*, *.uk*.

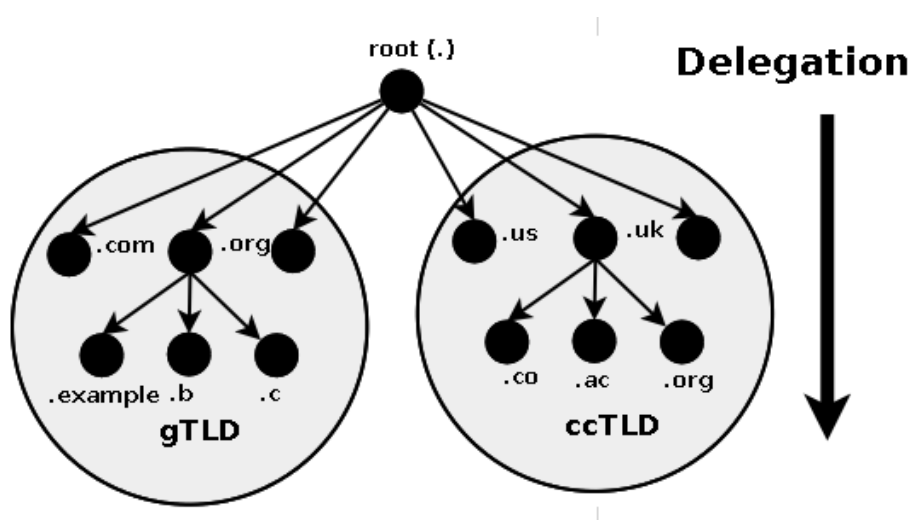


Figura 52: Estructuras de dominio y delegación - [DNS for rocket scientists](#)

Cuando hablamos de un **Nombre de Dominio** hablamos de una combinación de un nombre y un TLD. Se escriben de izquierda a derecha y el orden de jerarquía asciende hacia la derecha. De este modo, el nombre de dominio *example.com* está compuesto por el nombre *example* y el gTLD *.com*. El nombre se suele decir que es un **SLD** (*Second Level Domain*) ya que esta en el segundo nivel de la jerarquía.

6.1.3. Autoridad y Delegación

Como dijimos, DNS venía a solucionar algunos problemas de las primeras implementaciones de NS. Para solucionar los posibles errores que se producen a la hora de cambiar los registros de nombre de un servidor, se implementa el concepto de autoridad y delegación del control. Cada dominio tiene autoridad sobre todos los nombres de dominio que contiene. Por ejemplo, el dominio de Argentina (**.ar**) tiene autoridad sobre el nombre de dominio **ros.ar** (**ros** hace referencia a la ciudad de Rosario), y puede administrarlo. Ahora, imagínense si el DNS de Argentina tiene que encargarse de todos los nombres de dominio de cada ciudad de cada provincia, un quilombo. Para eso existe la delegación. El dominio de Argentina puede delegar su autoridad hacia dominios de niveles más bajos. De este modo, el dominio de Rosario tendrá autoridad sobre todos los nombres de dominios que vivan en él. Por ejemplo, el nombre de dominio **lcc.ros.ar** será administrado por el DNS de Rosario.

Si leemos un nombre de dominio de derecha a izquierda podemos ir llevando una traza de las distintas delegaciones. Cada unidad de delegación se suele conocer con el nombre de **zona**.

El dominio delegado tiene autoridad completa sobre todo lo que esta a su izquierda. Siguiendo el ejemplo, el nombre de dominio **www.lcc.ros.ar** es controlado por el dominio **lcc.ros.ar**. Como convención, **www** se conoce como el nombre de host de la página web (ya que refiere a *World Wide Web*) pero no es obligatorio, fácilmente el nombre puede haber sido **juan.lcc.ros.ar**. Todas las computadoras o servicios que son direccionales (es decir, tienen un URL) via internet tienen una parte de nombre de host. Algunos ejemplos son:

- **www.lcc.ros.ar** - El servicio web de lcc.
- **ftp.lcc.ros.ar** - El servidor de protocolo de transferencia de archivos de lcc.
- **pc12.lcc.ros.ar** - Una PC en particular dentro de la red.

Ahora veamos el siguiente nombre: **www.jcc.lcc.ros.ar**. Es claro que este nombre refiere a una pagina web (lo deducimos por el **www**), pero , ¿que pasa con la parte de **jcc**? Esta parte la creó el dueño del dominio **lcc.ros.ar** y tienen autoridad sobre ella (Si no delegó el control a la zona **jcc**). En este caso **jcc** es un **subdominio**.

En conclusión, un dueño de un dominio puede delegar de la manera que mejor le parezca a cualquier cosa a su izquierda en el nombre de dominio.

6.1.4. Zonas y archivos de zonas

Para que un servidor DNS funcione correctamente, se requieren tres componentes:



Figura 53: Meme de DNS para descontracturar

1. Datos que describan el dominio en sí.
2. Uno o mas programas de servidores de nombre.
3. Un programa o librería de resolución de direcciones.

Un servidor puede manejar uno o más dominios. Dentro de cada servidor se encuentra un archivo que contiene datos en forma de **registros de recursos** que describen los dominios o subdominios que el servidor maneje. Estos archivos se conocen como **archivos de zonas**. Un archivo de zonas se compone de los siguientes tipos de datos:

1. Datos que indiquen propiedades generales de la zona (registros **SOA**).
2. Datos autoritarios para todos los nodos o hosts dentro de la zona. Es decir, proveer información sobre las direcciones de los hosts dentro de la zona (registro **A** para IPv4 y **AAAA** para IPv6).
3. Información que describa información global de la zona (registros **MX** para servidores de mail y **NS** para servidores de nombre).
4. Si existiese alguna delegación de subdominio se aclara aquí el servidor de nombre delegado (registro **NS**).
5. Si hubo una delegación de un subdominio, se agregan **glue records** para indicarle al servidor de nombre como llegar al subdominio delegado.

Como dijimos anteriormente, se utilizan registros de recursos para describir los dominios del servidor. Existen una gran variedad de registros, que representan distintas características, nosotros nos concentraremos en un puñado:

- **SOA** (*State Of Authority*): Define el nombre de la zona, provee un mail de contacto (del administrador) y setea distintos temporizadores relacionados con las actualizaciones del archivo. Siempre se ubica en el comienzo del archivo de zona.
- **A**: Se utiliza para indicar direcciones IPv4 de hosts.
- **AAAA**: Se utiliza para indicar direcciones IPv6 de hosts.
- **NS**: Indica el servidor de nombre autoritario para el dominio (declarado en el registro SOA) o un subdominio especificado.
- **MX**: Registro utilizado para servidores de mail. Se agrega un valor de prioridad y un nombre de host para el servicio de mail.
- **CNAME** (*Canonical Name*): Se utiliza para asignar alias a los registros y lograr redirecciones.
- **PTR**: Utilizado para direcciones IP (Tanto IPv4 como IPv6). Se utiliza para mapeos reversos (ir de IP a nombre de dominio).

Además de valerse de registros, el archivo de zonas utiliza **directivas** para funcionar. Estas se indican con un \$ al principio y las que utilizaremos serán:

- **\$INCLUDE**: Incluye el archivo definido en la directiva.
- **\$DEFINE**: Define un nombre base utilizado en sustituciones de nombre.
- **\$TTL** (*Time To Live*): Define el tiempo de vida por defecto de los registros de recursos.

Como último detalle vamos a hablar de los **glue records**. Estos registros no son registros per se, sino que son técnicas que podemos implementar para resolver un problema en particular. Digamos que yo tengo mi dominio `juan.com`, y el servidor DNS que lo resuelve es `ns1.juan.com`. Ahora, introducimos un subdomino `hola.juan.com` que delegamos al DNS `ns.hola.juan.com`. Si pensamos en lo que acabamos de leer, cuando yo le pregunte a `ns1.juan.com` sobre la dirección de `hola.juan.com`, este me referirá al servidor `ns.hola.juan.com`. Ahora, ¿Cómo hacemos para llegar a `ns.hola.juan.com`? Claramente no es una dirección IP, por lo que debemos hacer es intentar resolver la dirección de `ns.hola.juan.com`. Cuando resolvamos vamos a llegar al punto que tenemos que preguntarle a `ns1.juan.com` si sabe la dirección de `hola.juan.com` (¿suena conocida esta pregunta?). El DNS no la sabe, pero nos refiere a ... `ns.hola.juan.com`, y volvemos a empezar. Este problema lo conocemos como una **dependencia circular**. ¿Como lo

arreglamos? Podemos incluir en el archivo de zonas del dominio superior (`juan.com` en este caso) información extra sobre la ubicación del servidor a referir (`ns.hola.juan.com`). Esto lo logramos con un registro NS y un registro A (o AAAA). El ejemplo se vería de la siguiente manera:

```

1 $ORIGIN juan.com
2 ...
3 @      IN   NS    ns1.juan.com
4
5 hola    IN   NS    ns.hola.juan.com // delegacion del subdominio | glue
6 ns.hola IN   A     192.168.235.10  // direccion del servidor  | record
7

```

Listing 1: Glue record

De esta manera rompemos el ciclo ya que brindamos una dirección adicional para el servidor. Este glue record podría leerse como: “*el subdominio `hola` de `juan.com` está delegado al servidor `ns.hola.juan.com`, y el servidor `ns.hola.juan.com` está en la dirección IP `192.168.235.10`”.*

6.1.5. Ejemplo de archivo de Zona

Ahora, veamos qué aspecto tiene un archivo de zonas.

```

1 // ARCHIVO DE ZONA DE ejemplo.com
2
3 $TTL 2d // 172800 segundos es el TTL por defecto.
4 $ORIGIN ejemplo.com. // Nombre base, reemplaza los @. Los espacios
5 // vacios se reemplazaran por lo que esta
6 // inmediatamente arriba.
7
8 @      IN      SOA    ns1.ejemplo.com. hostmaster.ejemplo.com. (
9                          12h // ref = refresh
10                         15m // ret = update retry
11                         3w  // ex = expiry
12                         3h  // min = minimum
13                         )
14      IN      NS      ns1.ejemplo.com. // Define el servidor de
15                                          // nombre autoritario
16                                          // para ejemplo.com
17
18      IN      MX      10 mail.ejemplo.net. // Host de mail principal
19                                          // Prioridad mayor (10)
20
21      IN      MX      20 mail2.ejemplo.net // Host de mail auxiliar
22                                          // Prioridad menor (20)
23
24      IN      A        192.168.254.3 // Asigna la direccion
25                                          // IPv4 del host
26                                          // juan.ejemplo.com
27
28      IN      CNAME    juan // Asigna el alias
29                          // juan a
30                          // www.ejemplo.com
31

```

```
32 ns2      IN      A      192.168.125.25    // Asigna la direccion
33                                     // IPv4 del servidor
34                                     // ns2.ejemplo.com
35
36 mateo    IN      NS      ns2.ejemplo.com.    // Define el servidor de
37                                     // nombre autoritario
38                                     // para el subdominio
39                                     // mateo.ejemplo.com
40 // Estas ultimas dos instrucciones forman un glue record
```

Listing 2: Ejemplo de archivo de zona

6.1.6. Consultas

La principal tarea (y el propósito) de un servidor DNS es poder responder a las consultas que realiza un host (u otro servidor DNS) para resolver direcciones IP de nombres de dominio. Un servidor DNS puede en teoría recibir una consulta de cualquier dominio, hasta de dominios en los que el no sea autoritario (no pueda responder directamente). En la mayoría de los casos, un DNS puede recibir un nombre de dominio del cual no tenga información (no esté en su archivo de zonas local). Debido a estas posibilidades, se definieron tres tipos de consultas:

1. **Iterativas:** Las consultas iterativas son aquellas en las que el DNS puede responder dos cosas: La dirección buscada o una **referencia** a otro DNS. Esta última respuesta sirve para poder redirigir una consulta hacia otro servidor (que, por ejemplo, sea el autoritario del dominio delegado que buscamos). Todos los servidores DNS deben aceptar consultas iterativas.
2. **Rekursivas:** Las consultas recursivas aseguran que la respuesta completa (dirección buscada) se consigue o se produce un error, no hay otro resultado posible. Esto se logra utilizando distintas consultas iterativas y refiriéndonos a los servidores que estas consultas nos puedan referir.
3. **Inversas:** Las consultas inversas justamente hacen el trabajo inverso. Dada una dirección IP nos devuelven el nombre del dominio. Usualmente se utiliza para temas de seguridad y autenticación.

6.1.7. Ejemplo de una consulta recursiva

¿Qué estamos viendo en la figura 54?

Supongamos que el usuario escribe la dirección `www.ejemplo.com` en el buscador (browser). El buscador primero envía la consulta a un solucionador de direcciones local (stub-resolver), que revisa si tiene la dirección en su caché. Si no es así, el solucionador pregun-

Recursive and Iterative Queries

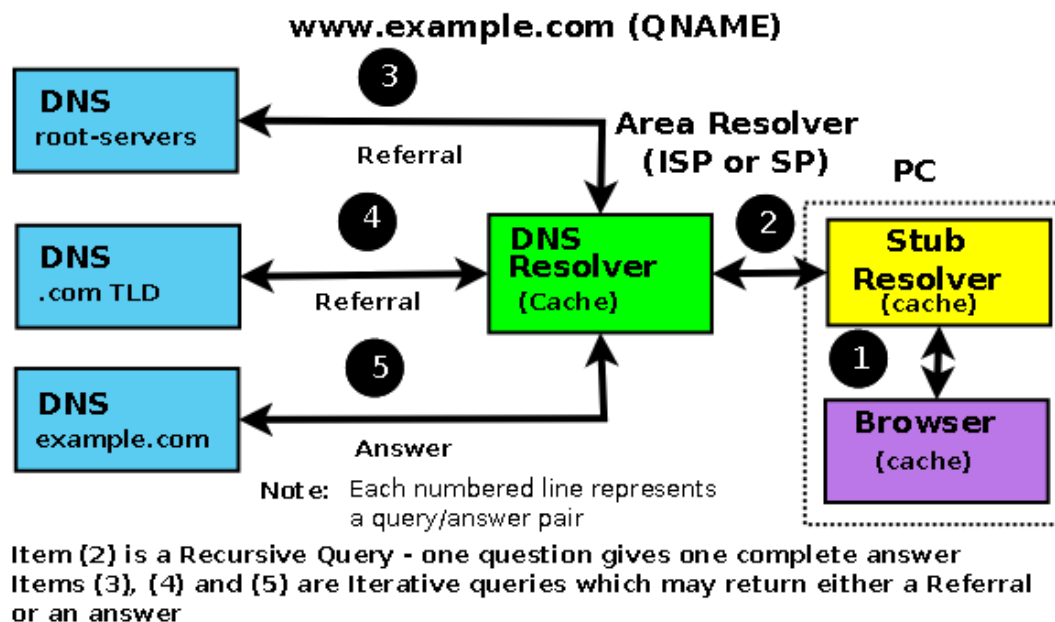


Figura 54: Diagrama de una consulta recursiva - [DNS for rocket scientists](#)

ta mediante una consulta (probablemente recursiva) al DNS solucionador (DNS resolver) “¿Cuál es la dirección IP de `www.ejemplo.com`”. Si este lo tiene almacenado en su caché lo devuelve inmediatamente. Si no, comienza a resolver el nombre. Como se resuelve de derecha a izquierda, el primer dominio que encuentra es el **root** (ya que el nombre es `www.ejemplo.com.`), con el ‘.’ al final. De este modo, envía una consulta (iterativa) al servidor root. Este DNS no sabe nada del nombre de dominio, pero sí conoce el gTLD `.com`, por lo que devuelve la referencia al DNS encargado de resolver ese dominio. El DNS solucionador, entonces, le pregunta al servidor DNS de `.com` (mediante una consulta iterativa). Este tampoco sabe nada de `www.`, pero conoce a `ejemplo.com`, por lo que responde con la referencia al DNS que resuelve ese dominio. Una vez mas, el DNS solucionador pregunta al nuevo DNS referido si conoce la dirección de `www.ejemplo.com`. A diferencia del resto, este sí sabe qué es, porque tiene, en su archivo de zonas local, un registro A que vincula al nombre de dominio `www.ejemplo.com` con la dirección IPv4 `x.x.x.x`. Esta respuesta sí es la esperada, por lo que el DNS solucionador reenvía esta dirección al solucionador de direcciones local, que lo guarda en su caché y se la envía al buscador.

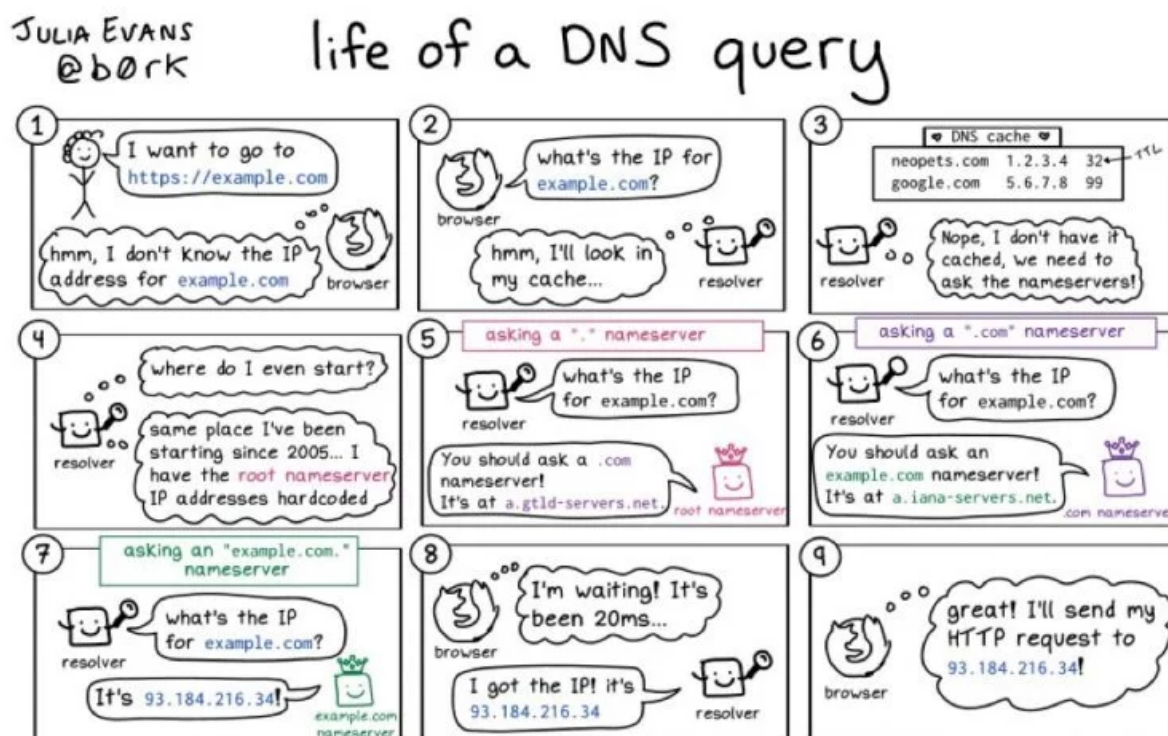


Figura 55: Otro meme de DNS para desconstruir, este es un poco mas interesante.

6.1.8. Mapeo Inverso

Una consulta normal de DNS sería: “¿Cuál es la dirección IP del nombre de dominio `www.juan.com`?”, en cuyo caso el servidor DNS nos respondería: “Es `x.x.x.x`”. Ahora, hay casos en los que sabemos la IP pero necesitamos conseguir el nombre de dominio. Normalmente este tipo de consultas las realiza un sistema de seguridad, para poder mantener el rastro de algún *hacker* o un *spammer*. Resolver estas consultas es lo que se conoce como **mapeo inverso**. Para lograrlo mediante consultas iterativas y recursivas comunes, los diseñadores de DNS definieron un nombre de dominio especial conocido como `IN.ADDR.ARPA` para direcciones IPv4 y `IP6.ARPA` para direcciones IPv6.

¿Cómo hace el servidor para resolver el nombre de dominio de una dirección IP?

Tenemos que analizar dos casos, según el tipo de dirección a resolver. Si estamos intentando de resolver una dirección IPv4 entonces se plantea lo siguiente. Una dirección IPv4 se ve mas o menos así:

192.168.27.12

Ahora, si seguimos la idea de como resolver un nombre de dominio, deberíamos comenzar por el dominio con más jerarquía, es decir el de la derecha de todo. Sin embargo, en la dirección, eso representaría el host. De este modo, se opta por invertir el orden de jerarquía

y hacerlo crecer hacia la izquierda. Además, se omite la dirección del host. La dirección, entonces, queda de la siguiente manera:

192.168.27

Donde el 192 sería el “dominio” con mayor jerarquía. Ahora, en la consultas comunes, el DNS lee de derecha a izquierda, pero en esta dirección, la jerarquía va de izquierda a derecha, lo que complica el trabajo. Esto se soluciona invirtiendo la dirección y agregando un dominio especial de mayor jerarquía llamado `in-addr.arpa`, que permite al DNS comprender que se quiere hacer una consulta inversa. La dirección entonces queda así:

7.168.192.in-addr.arpa

Cabe aclarar que este dominio especial debe tener su propio archivo de zonas especial para poder reconstruir el nombre de domino. Este archivo se ve de la siguiente manera:

```

1 $ORIGIN 27.168.192.IN-ADDR.ARPA
2 ...
3 12 IN PTR www.juan.com // se utiliza el registro
4                          // PTR
5 15 IN PTR hola.juan.com // La direccion
6                          // 192.168.27.15 corresponde
7                          // al nombre de dominio
8                          // hola.juan.com

```

Listing 3: Archivo de zona para mapeo inverso

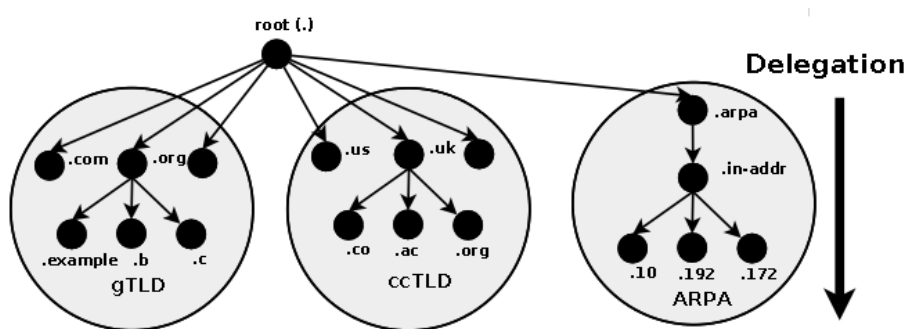


Figura 56: Estructura DNS de mapeo inverso con IN-ADDR.ARPA - [DNS for rocket scientists](#)

En cuanto a direcciones IPv6, el procedimiento es similar, se invierte la dirección y se coloca todo bajo un dominio en particular, en este caso es `ip6.arpa`. La diferencia con las direcciones en IPv4 es que la unidad de delegación es un *nibble*, es decir, 4 bits en vez de 8 bits (un byte), como lo es en las direcciones IPv4. De este modo, todos los caracteres hexadecimales que componen una dirección IPv6 común deberán ser divididas en dominios con distintas jerarquías. Entonces, si tenemos la siguiente dirección IPv6:

2001:0db8:0000::/48

Obtendremos la siguiente dirección ordenada de manera jerárquica y dividida en los dominios establecidos:

0.0.0.0.8.b.d.0.1.0.0.2.IP6.ARPA

Un ejemplo de un archivo de zona para mapeo inverso de direcciones IPv6 puede ser:

[illegible]

Listing 4: Archivo de zona para mapeo inverso IPv6

No es difícil notar que las direcciones usadas en los registros PTR son increíblemente largas. Esto se puede resolver quitando dominios comunes de las direcciones en los registros PTR y agregándolas a la directiva \$ORIGIN.

6.1.9. Tipos de servidores DNS

Podemos clasificar a los servidores DNS por la relación que tienen por sobre las zonas de las cuales tienen autoridad. Esta clasificación se puede dividir en dos grandes tipos, los servidores **maestros** (Master) y los servidores **esclavos** (Slave).

Un servidor maestro define uno o más archivos de zonas para las cuales este servidor será autoritario. Es decir, la zona en cuestión fue delegada (mediante un registro NS) a este DNS. Un servidor maestro puede notificar cambios en la zona a otros servidores esclavos. Esto se realiza mediante mensajes **NOTIFY**, que aseguran que los cambios se realicen en los servidores esclavos.

Los servidores esclavos consiguen su información de zona mediante una **operación de transferencia de zona**, que típicamente viene de un servidor maestro. El servidor esclavo responderá como autoritario por las zonas en las cuales fue definido como esclavo.

Es imposible diferenciar si el resultado de una consulta provino de un servidor DNS maestro o un servidor esclavo.

Como no entendí nada (y eso que soy el que escribió esta unidad y tengo la bibliografía abierta acá al lado del Overleaf) voy a resumir esta subsección con una breve conclusión:

“Los servidores Maestros y Esclavos solamente se diferencian en el hecho de que los primeros obtienen su información de zona de un archivo almacenado localmente (El archivo de zona). Mientras que los segundos dependen de otro servidor que les provea de esta información”

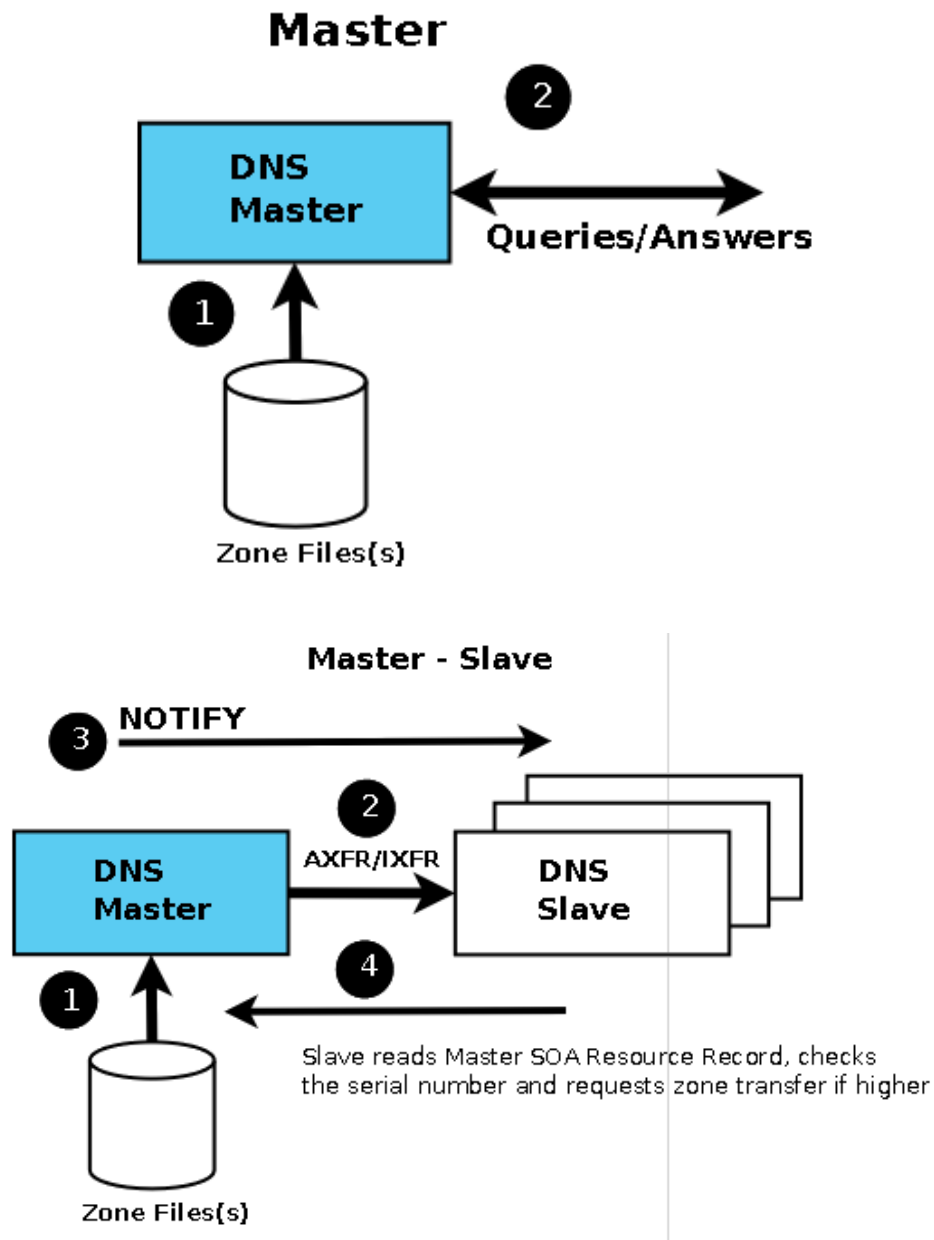


Figura 57: Diagramas de servidores DNS Maestro y Esclavo - [DNS for rocket scientists](#).

6.1.10. Archivos **named.conf**

BIND es el tipo mas común de servidores DNS especialmente en dispositivos que utilizan UNIX. Un sistema BIND consiste de las siguientes partes:

1. Un archivo **named.conf** que describe la funcionalidad del sistema BIND.
2. Según la configuración, uno o mas archivos de zonas que describan los dominios manejados.
3. Dependiendo de la configuración se pueden necesitar uno o mas archivos de zonas que describan al localhost o al servidor de nombre de root.

Aunque ya está spoileado en el nombre de la subsubsección, lo que nos importa de este sistema BIND (más allá de los archivos de zona, que ya hablamos) son los archivos **named.conf**. Estos archivos son los encargados de controlar el comportamiento y las funcionalidades de BIND. Los archivos **named.conf** administran las zonas sobre las cuales el servidor es maestro y esclavo, permiten que solo algunas zonas sean visibles para ciertos usuarios, y especifican el archivo de zona al que se debe corresponder para cada zona en particular, entre otras cosas.

Un archivo **named.conf** esta compuesto de **cláusulas** que agrupan **sentencias** que controlan la funcionalidad y seguridad del servidor.

Para poder definir una zona, utilizaremos la cláusula **zone**. Dentro de la cláusula nosotros podremos especificar propiedades del servidor sobre la zona. La primer propiedad que nos puede interesar es el tipo que tendrá el servidor frente a esta zona. Esto lo logramos con la sentencia **type**, que puede estar acompañados de las palabras **master** si se quiere definir un servidor maestro o **slave** si se quiere hacer uno esclavo. En el último caso, debemos aclarar quién o quienes serán los maestros. Para esto, utilizamos la cláusula **masters**, dentro de la que especificaremos las direcciones IP de cada uno de los servidores maestros. También, es esencial indicar en esta cláusula (**zone**) el archivo de zona que utilizaremos. Esto se logra con la sentencia **file** seguida del path al archivo deseado. Por convención, en la cátedra utilizamos el siguiente path: “**etc/bind/db.nombredelazona**”.

También contamos con la habilidad de modificar la forma en la que distintas máquinas observan el orden de jerarquía de nombres del DNS. Esto lo logramos con la cláusula **view**. Por ejemplo, podemos revelar los host solamente a los usuarios internos y esconderlos a los externos, o podemos ofrecer los mismos hosts a usuarios internos y externos pero proporcionar registros adicionales o distintos a los hosts internos. Casi siempre, esta cláusula viene acompañada de la cláusula **match-clients** que controla quién puede ver la vista. Estas vistas se procesan de manera secuencial, por lo que la primera vista (de

arriba a abajo) debe ser la mas restrictiva.

Existen muchas mas cláusulas y sentencias útiles, nosotros nombraremos sólo un puñado:

- `allow-notify {lista_de_direcciones}`: Aplica sólo a servidores esclavos. Permite modificar quién puede mandarle un NOTIFY y actualizar el archivo de zona. Por defecto estos valores son las direcciones IP de los maestros.
- `allow-update {lista_de_direcciones}`: Define una lista de direcciones IP de hosts que pueden subir actualizaciones dinámicas a las zonas de los maestros. Esta cláusula permite el **DDNS** (*Dynamic DNS*).
- `allow-query {lista_de_direcciones}`: Utiliza una lista de direcciones IP para determinar qué hosts pueden realizar consultas sobre esa zona a ese servidor.
- `allow-transfer {lista_de_direcciones}`: Define la lista de direcciones IP que tendrán permitido tranferir (copiar) información de zona desde un servidor (maestro o esclavo).

6.1.11. Ejemplo de archivo named.conf

¿Como se ve un archivo named.conf? De la siguiente manera:

```
1 zone "." {
2     type hint;
3     file "etc/bind/db.root";
4 };
5
6 zone "localhost" {
7     type master;
8     file "etc/bind/db.localhost";
9 };
10
11 view "Internal" {
12     match-clients {192.168.170.20; 192.168.170.30;};
13     zone "ejemplo.com" {
14         allow-query {any;};
15         allow-update {none;};
16         allow-transfer {any;};
17         type master;
18         file "etc/bind/db.ejemplo.com";
19     }
20 };
21
22 view "External" {
23     match-clients {any;};
24     zone "lcc.com" {
25         allow-notify {any;};
26         type slave;
27         masters {192.168.190.45;};
28         file "etc/bind/db.lcc.com";
29     }
30 }
```

Listing 5: Ejemplo de archivo named.conf

6.2. Firewall

Fue largo DNS, ¿No?

Descansemos en esta **fogata**.

Mientras tanto, y manteniendo la temática de fuego, hablemos sobre **firewall**.

Hoy en día, el Internet nos da acceso a infinidad de recursos y nos mantiene conectados con todo el mundo casi instantáneamente. Sin embargo, estar conectado también significa ser vulnerables a ataques de seguridad, donde gente maliciosa puede conseguir acceso a nuestro sistema y conseguir información privada. Este problema se puede solucionar (o intentar) evitando que personas sin autorización puedan llegar a los hosts mediante la red, y para lograr esto entran en juego los firewall (o **cortafuegos**).

6.2.1. ¿Qué es un firewall?

Un firewall en sí es un host de una red que contiene una serie de reglas (definidas de una manera específica) que deciden si un paquete entra a la red y es enviado al destino deseado o es descartado. Usualmente el host es una especie de cuello de botella entre dos redes (la mayoría de las veces será entre una red privada y el internet) de manera de que todos los paquetes que se transmitan entre redes deban pasar si o si por el firewall.

Existen varias manera de implementar cortafuegos, nosotros veremos dos: La primer manera requiere más de un host para formar una especie de red perímetro, o **zona desmilitarizada** (DMZ, *Desmilitarized Zone*). Dos hosts actúan como filtros para permitir pasar solamente a un cierto tipo de tráfico, y entre estos filtros, residen servidores de red como los dedicados a hostear páginas web o email.

La segunda manera implica tener un único servidor que no solo hostee las aplicaciones como el mail y páginas web, sino que también contenga el firewall. Este método es claramente más barato y fácil de manejar, pero si alguno de los servidores de aplicación es atacado, la seguridad de toda la red se puede comprometer.

Un firewall puede hacer varias cosas con un paquete que pase por él. La acción mas básica es de filtrar paquetes (Filtering). Es decir, si un paquete cumple con ciertos requisitos, como venir desde una IP no autorizada, simplemente bloquea el paso al paquete, y puede decidir si tirarlo (**DROP**, sin avisarle al emisor), o denegarlo (**REJECT**, avisar al emisor). Además, un cortafuegos puede modificar (Mangle) un paquete a medida que se mueve y puede cambiar la dirección IP (NAT, *Network Address Translation*) a la cual se dirige o de la que viene.

6.2.2. Iptables

Iptables es la interfaz de comandos de Linux provista para configurar firewalls. La arquitectura de iptables trata de agrupar **reglas de procesamiento de paquetes de red** en **tablas** divididas por función (filtrado, modificación y nateo). Cada tabla está compuesta por **cadenas de reglas**. Cada regla a su vez consiste de **coincidencias** (*matches*), que determinarán si dicha regla es aplicada al paquete y **objetivos** (*targets*), que determinarán qué le pasará al paquete.

Iptables define cinco “puntos de gancho” (*hook points*) en la manera en la cual el kernel procesa los paquetes. Estos puntos tienen cadenas anexadas que permiten agregar secuencias de reglas a cada punto. Estos ganchos son:

- **FORWARD**: Permite procesar paquetes que estén entrando por una interfaz y saliendo hacia otra.
- **INPUT**: Permite procesar paquetes justo antes de que sean entregados a un proceso local. En otras, palabras, paquetes recién llegados.
- **OUTPUT**: Permite procesar paquetes justo luego de que hayan sido generados por un proceso local.
- **POSTROUTING**: Permite procesar paquetes justo antes que salgan a una interfaz de red. Es usado en tablas de nateo.
- **PREROUTING**: Permite procesar paquetes a medida que llegan desde una interfaz de red. Es usado en tablas de nateo.

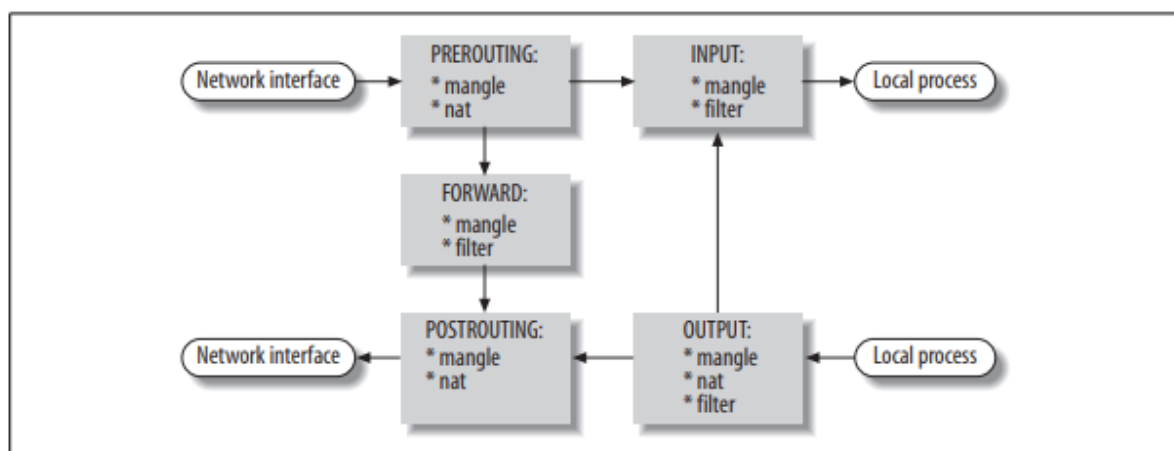


Figura 58: Movimiento de un paquete mediante puntos de gancho - [Linux Network Administrators Guide](#)

En la figura 58 vemos el camino que toma un paquete dentro del sistema. Las cajas representan cadenas de iptables, y dentro de cada caja hay una lista de las tablas que tienen dicha cadena. Un paquete que pase por el sistema será afectado por diferentes cadenas según el tipo de proceso al que se le está sometiendo. Por ejemplo, un paquete que va a ser filtrado podrá atravesar el sistema solamente por las cadenas **INPUT**, **FORWARD** y **OUTPUT**. Otra característica de iptables es que implementa un “cortafuego de flujo” (*stateful firewall*). Este tipo de firewall considera a los paquetes entre dos máquinas como una única conexión de un mismo flujo de datos. Lo importante es que la conexión cambia de **estado** a lo largo de su vida. Esto permite generar conjuntos de reglas mucho más descriptivos, haciendo a este tipo de cortafuegos mucho más preciso. Iptables define cuatro posibles estados para una conexión, estos son:

- **NEW**: Este estado representa una conexión nueva. Básicamente, un paquete podría tener este estado si es el primero que vemos llegar de alguna conexión.
- **ESTABLISHED**: Representa una conexión previamente establecida, es decir, ya se ha visto tráfico de paquetes en ambas direcciones entre las máquinas.
- **RELATED**: Una conexión se considera “relacionada” si está ligada a una conexión previamente establecida. Por esto, para que una conexión esté relacionada, debe existir una conexión establecida que genere una conexión externa a la principal. Esta nueva conexión se dice que está relacionada.
- **INVALID**: Implica que el paquete no puede ser identificado o que no tiene ningún estado. Normalmente es buena idea eliminar todo aquello que se encuentre en ese estado.

Para incluir el estado de un paquete en una regla lo hacemos mediante:

```
-m state --state ESTADO
```

6.2.3. Tablas

Como comentamos, iptables cuenta con tres tablas: **filter**, **mangle** y **nat**. Cada una de ellas esta preconfigurada con cadenas que corresponden a uno o mas hook points. Veamoslas más en detalle:

- **Tabla filter**: Utiliza políticas para determinar el tipo de tráfico que deja pasar a la red. A menos que se refiera a otra tabla explícitamente, iptables tomará esta tabla por defecto para operar sobre las cadenas definidas. Las cadenas predefinidas son: **FORWARD**, **INPUT** y **OUTPUT**.

- **Tabla mangle:** Se utiliza para alterar el paquete de alguna manera específica. Sus cadenas predefinidas son: PREROUTING, POSTROUTING, FORWARD, INPUT y OUTPUT.
- **Tabla nat:** Se utiliza para redirigir paquetes, alterando la dirección de destino o de salida. Sus cadenas preconfiguradas son: PREROUTING, POSTROUTING y FORWARD.

6.2.4. Cadenas

Como comentamos, los paquetes viajan a través de cadenas, pasando por cada una de las reglas de la cadena en orden. Si un paquete no coincide con una regla, pasa a la siguiente. Puede pasar que un paquete no coincida con ninguna regla de la cadena, en dicho caso, se utiliza la **política** por defecto de la misma.

6.2.5. Reglas

Una regla de iptables consiste en uno o más **criterios de coincidencia** para determinar qué paquetes de red se verán afectados y el objetivo (*target*, destino del paquete) que tendrá la regla sobre dichos paquetes. Todos los criterios de coincidencia de una regla deben cumplirse para que un paquete sea afectado por ella. Tanto los criterios como el objetivo son campos opcionales dentro de una regla. Esto implica que si no hay criterio especificado, todos los paquetes coincidirán. Por otro lado, si no hay objetivo, los paquetes no se verán afectados de ninguna manera.

6.2.6. Criterios de comparación (matches)

Existen una gran variedad de criterios de coincidencia disponibles con iptables. Estos criterios suelen ser bastante específicos y están agrupados según categorías. Por ejemplo, existen criterios de coincidencia reservados para paquetes enviados con protocolos específicos, como TCP o UDP. De todos modos, existen ciertos criterios genéricos que sirven para cualquier protocolo, algunos de estos son:

- **-p, --protocol:**
Acompañado de un protocolo, este criterio se utiliza para verificar si el paquete pertenece alguno de los protocolos TCP, UDP o ICMP. Por ejemplo: `iptables -A INPUT -p tcp`
- **-s, --s, --source:**
Verifica la dirección IP de origen del paquete. Se puede invertir la comparación con el operador '!'. Ejemplo: `iptables -A INPUT -s 192.168.230.20`

- **-d, --dst, --destination:**
Sirve para comparar la dirección IP de destino de los paquetes. Tiene la misma sintaxis que **--source**. Ejemplo: `iptables -A INPUT -d 192.168.1.1`
- **-i, --in-interface:**
Se utiliza para reconocer a través de qué interfaz proviene un paquete entrante. Sólo es válida en cadenas **INPUT**, **FORWARD** y **PREROUTING**. Por ejemplo: `iptables -A INPUT -i eth0`
- **-o, --out-interface:**
Al contrario de **--in-interface**, este criterio se emplea con los paquetes que están a punto de abandonar la interfaz de salida. Sólo está disponible en las cadenas **OUTPUT**, **FORWARD** y **POSTROUTING**. Ejemplo: `iptables -A FORWARD -o eth0`
- **--sport, --source-port:**
Esta comparación no es tan genérica y sirve sólo para paquetes enviados mediante protocolo TCP o UDP. Sirve para filtrar paquetes basándose en su puerto de origen. Este puede tener el nombre del servicio (Ya que referirá a un puerto bien conocido) o directamente el número del puerto. Por ejemplo: `iptables -A INPUT -p tcp --sport 22` (puede utilizarse también con `-p udp`).
- **-dport, --destination-port:**
Funciona del mismo modo que **--sport** solo que filtra los paquetes según su puerto de destino. Un ejemplo: `iptables -A INPUT -p udp --dport 80`
- **-m multiport:**
Especifica una serie de puertos para buscar coincidencia. Su utilidad es meramente para ahorrarnos trabajo. Un ejemplo: `iptables -A INPUT -p tcp -m multiport --sport 22,53,80,110`

6.2.7. Objetivos (Targets)

Los objetivos son usados para especificar qué acción tomar en el momento que un paquete coincide con una regla. También lo usamos para especificar políticas de las cadenas. Existen varios objetivos, nosotros nos concentraremos en un puñado:

- **ACCEPT:**
El paquete es aceptado y fluye hasta la siguiente etapa de procesamiento. Deja de atravesar la cadena actual.

- **DROP:**

Deja de procesar el paquete por completo. No sigue analizando el paquete con otras reglas, cadenas o tablas. No avisa al emisor que el paquete no fue aceptado.
- **REJECT:**

Del mismo modo que **DROP**, el paquete no es aceptado y el procesamiento cesa. Se envía una notificación al emisor mediante un mensaje de error. Sólo es válido en cadenas **INPUT**, **FORWARD** y **OUTPUT**.
- **SNAT:**

Este objetivo se emplea para hacer traducciones de direcciones de origen de paquetes IP. Se reemplaza la cabecera del paquete por otra con la dirección actualizada. Es decir, cambia la dirección de origen de todos los paquetes que salgan de la red local por la IP de origen de nuestra conexión a internet. Además, este objetivo también setea una operación inversa, logrando que paquetes entrantes lleguen a su host objetivo. Es válido solamente dentro de la tabla nat, de ahí su nombre **SNAT**, *Source Network Address Translation* y dentro de la cadena **POSTROUTING**.
- **DNAT:**

De manera parecida a **SNAT**, **DNAT** (*Destination Network Address Translation*) se utiliza para cambiar la dirección IP de un paquete. En este caso se reemplaza la dirección de destino y se redirigen al host adecuada. Es útil cuando estamos dentro de una red privada cuyas direcciones no son ruteables. Si un paquete viene dirigido a esta red, el firewall cambiará su dirección de destino (la IP de nuestra conexión a internet) por su IP dentro de la red local. Es válido dentro de la tabla nat y en las cadenas **PREROUTING** y **OUTPUT**.
- **MASQUERADE:**

El objetivo **MASQUERADE** es prácticamente el mismo que **SNAT**, pero no se requiere especificar la dirección IP que se va a usar para reemplazar. Esto es porque **MASQUERADE** se creó para trabajar con direcciones IP dinámicas. Al enmascarar una conexión establece la dirección IP utilizada por una tarjeta de red específica, de modo que la dirección IP de origen se obtiene directamente desde esta tarjeta específica. Se utiliza solamente en la cadena **POSTROUTING** de la tabla nat.

6.2.8. Comandos

Esta subsubsección es para presentar algunos comandos que utilizaremos:

- **-A, --append:** Permite agregar una regla al final de la cadena. Por defecto esta regla será la última en el conjunto de reglas y lógicamente se comprobará última.

- `-P, --policy`: Sirve para establecer una nueva politica u objetivo por defecto de la cadena.
- `-j, --jump`: Nos permite enviar un paquete coincidente a algun objetivo establecido en la regla.
- `--to-source`: Es complementario del objetivo `SNAT` y permite especificar que dirección debe utilizar el paquete.

6.2.9. Ejemplo de firewall

```

1  # DECLARACION VARIABLES
2  I = /sbin/iptables // ubicacion de iptables
3  LAN = 10.0.1.0/24
4  SVR = 10.0.2.0/24
5  INET = 200.3.1.2
6  DNS = 181.16.1.19
7  ILAN = eth0 // interfaz de LAN
8  ISVR = eth1 // interfaz de servidores
9  IINET = eth2
10
11 # ELIMINAMOS PAQUETES INVALIDOS
12 $I -A INPUT -m state --state INVALID -j DROP
13 $I -A INPUT -m state --state RELATED ESTABLISHED -j ACCEPT
14
15 # ESTABLECEMOS POLITICAS POR DEFECTO
16 $I -P INPUT -j DROP
17 $I -P FORWARD -j DROP
18
19 # EVITAMOS QUE LAN SE COMUNIQUE CON LOS SERVIDORES
20 $I -A FORWARD -s $LAN -i $ILAN -d $SVR -j REJECT
21
22 # PERMITIMOS QUE INTERNET ENVIE AL DNS
23 $I -A FORWARD -s $INET -d $DNS -p tcp -dport 53 -j ACCEPT // el puerto
24 $I -A FORWARD -s $INET -d $DNS -p ucp -dport 53 -j ACCEPT
25
26 # NATEO
27 $I -t nat -A POSTROUTING -s $LAN -o $IINET -j SNAT --to-source 200.3.1.2

```

Listing 6: ejemplo de un archivo de firewall

Bibliografía usada: [DNS for rocky scientists](#), capítulos 7 y 9 de [Linux Network Administrators Guide](#).

7. Preguntas Frecuentes

Para cerrar con broche de oro, nos pareció buena idea juntar todas las preguntas (o por lo menos las más relevantes) que suelen preguntar en los finales de teoría, e intentar responderlas de la forma mas concisa posible, como para hacer un repaso general de la materia sin detenernos a explicar cosas que ya están en el apunte. También recomendamos intentar responderlas antes de ver la respuesta, a modo de auto examen. Aclaramos que las preguntas no están divididas por capítulo, pero sí van siguiendo el orden en el que se dan los temas.

Para evitar spoilers, vamos a dejar las preguntas por un lado y las respuestas por otro. Empecemos con las preguntas.

7.1. Preguntas

1. Dibujar y explicar principales diferencias entre modelo OSI y TCP/IP
[Respuesta](#)
2. ¿Se puede aumentar la velocidad de un canal que opera en el límite de Nyquist aumentando los niveles de transmisión? ¿Y lo mismo en Shannon? [Respuesta](#)
3. Describa las funciones de un puente, hub, switch, indicando en qué capa del modelo OSI opera cada uno [Respuesta](#)
4. ¿En que casos utilizaría un puente, un repetidor o un switch? Justifique.
[Respuesta](#)
 - a) Interconectar dos redes de diferentes tecnologías.
 - b) Mejorar el rendimiento de una red saturada.
 - c) Interconectar cinco redes con mucho tráfico.
5. ¿Qué pasa si se quiere conectar dos LANS mediante dos puentes/bridges en paralelo? [Respuesta](#)
6. Diferencias entre par trenzado y fibra óptica. [Respuesta](#)
7. Dé 2 diferencias entre ipv4 y ipv6 (que no sea el tamaño de las direcciones). [Respuesta](#)
8. ¿Cómo se hace broadcast en IPv6? [Respuesta](#)

9. Diferencias entre fragmentación en IPv4 y fragmentación en IPv6. [Respuesta](#)
10. Métodos y técnicas que utiliza un host desde que se inserta a una LAN hasta tener información suficiente para conectarse a Internet en IPv6. [Respuesta](#)
11. ¿Qué pasa si un router no puede transferir un paquete que llegó con el bit DF (no fragmentar) activado? [Respuesta](#)
12. Diferencias entre reensamblar en un router intermedio y reensamblar en el destino. [Respuesta](#)
13. Modificación del algoritmo de cálculo de timeout para retransmisión, cuando se pierde un paquete. [Respuesta](#)
14. Explique el algoritmo por el cual un host decide si debe hacer una entrega directa de un datagrama o si debe pasarlo a un router. [Respuesta](#)
15. ¿Puede ser una dirección de difusión un número par? En caso afirmativo de un ejemplo. [Respuesta](#)
16. ¿Es posible que 2 máquinas mantengan una conexión TCP con un servidor y estas conexiones estén en un mismo puerto? Explique [Respuesta](#)
17. Ventana tonta. ¿Qué es? ¿Cuándo sucede? ¿Cómo se soluciona? [Respuesta](#)
18. Explicar control de flujo en TCP (Ventana deslizante). [Respuesta](#)
19. ¿En qué situación se manda un frame con ACK y SYN activados? ¿Qué pasa si se pierde el frame? [Respuesta](#)
20. Datos fuera de banda (URG). [Respuesta](#)
21. Respuestas de TCP al congestionamiento [Respuesta](#)
22. ¿Qué es un Glue record? [Respuesta](#)

7.2. Respuestas

1. [Pregunta.](#)

Las principales diferencias entre OSI y TCP/IP son la cantidad de capas (7 en uno, 4 en otro), y que OSI soporta envíos tanto con conexión como sin conexión en capa de red, mientras que TCP/IP lo hace recién en capa de transporte (por TCP).

OSI	TCP/IP
Aplicación	Aplicación
Presentación	
Sesión	
Transporte	Transporte
Red	Red
Enlace	Interfaz de red
Física	

2. [Pregunta.](#)

En Nyquist si se puede ya que aumentando los niveles aumenta el valor logarítmico de la fórmula. En Shannon no, ya que aumentando los niveles de transmisión aumenta el ruido e influye en la relación S/N.

3. [Pregunta.](#)

Un puente es un dispositivo de capa 2, que procesa y rutea datos a nivel capa 2, entre 2 segmentos de red. Un switch es un bridge multipuerto de capa 2. Y un hub es un dispositivo de capa 1, es un repetidor con múltiples puertos, uno por cada línea de entrada.

4. [Pregunta.](#)

- a) Bridge. Hay que conectar sólo 2 redes.
- b) Switch. Se puede distribuir la red en redes virtuales para reducir el tráfico.
- c) Switch. Con un bridge no alcanza porque son más de 2.

5. [Pregunta.](#)

Se genera un bucle en el que los paquetes se retransmiten hacia ambas LAN indefinidamente, saturando la red y dejándola inutilizable.

6. [Pregunta.](#)

Ancho de banda (fibra), costo (par trenzado), distancia efectiva (fibra), resistencia al ruido (fibra), facilidad de instalación (par trenzado).

7. [Pregunta.](#)
IPv6 tiene cabeceras de tamaño fijo, y por sus cabeceras de extensión, su procesamiento en los routers es más eficiente. Las direcciones en IPv6 son asignadas a interfaces, y no a nodos. Extra: proceso de fragmentación.
8. [Pregunta.](#)
El broadcast en IPv6 se logra mediante las direcciones multicast (de uno a muchos). A diferencia de IPv4, puede elegirse el alcance del broadcast.
9. [Pregunta.](#)
IPv6 fragmenta en el origen, sabiendo el MTU de la red (PMTUD). IPv4 fragmenta en routers a medida que se necesita.
10. [Pregunta.](#)
Utiliza el proceso de autoconfiguración de direcciones stateless. Resumiendo, crea una dirección de IP link-local y verifica que sea válida. Luego, utiliza Router Solicitation y almacena la información recibida por los mensajes de respuesta Router Advertisement.
11. [Pregunta.](#)
Responde al origen con un mensaje ICMP de necesidad de fragmentación y descarta el paquete.
12. [Pregunta.](#)
Reensamblar en un router intermedio requiere un mayor trabajo computacional de los routers pero aprovecha mejor el ancho de banda. Reensamblar en el destino implica mantener los fragmentos, incluso en redes de MTU mucho mayor, pero quita trabajo a los routers.
13. [Pregunta.](#)
Algoritmo de Karn. Anulación de temporizador para los segmentos retransmitidos.
14. [Pregunta.](#)
Se fija el prefijo de red de la dirección de destino. Si coincide con el prefijo de su propia dirección, hace una entrega directa. Si no, lo pasa un router para que use su tabla de ruteo.
15. [Pregunta.](#)
En IPv4 no porque todas las direcciones de broadcast tienen su hostid con unos, en particular, el último bit vale uno, por lo que la dirección es impar. En IPv6, en cambio, las direcciones de broadcast son las multicast, que pueden ser par, como por ejemplo all routers, que es FF02::2.

16. [Pregunta.](#)

Sí. Las conexiones TCP están definidas por sus puntos extremos. Como son dos máquinas con direcciones distintas, por más que usen el mismo puerto, sus conexiones con el servidor serán diferentes (incluso si el servidor usa el mismo puerto en ambos casos).

17. [Pregunta.](#)

Problema en TCP del mal aprovechamiento del ancho de banda, en donde una ventana envía segmentos con muy pocos datos. Sucede cuando el receptor no puede procesar datos a la misma velocidad que el emisor los envía. Para solucionarlo, el emisor espera a que se llene su memoria intermedia antes de enviar el segmento, el receptor espera a que se libere su memoria intermedia hasta la mitad para enviar un acuse de ventana mayor.

18. [Pregunta.](#)

Utiliza una ventana deslizante de tamaño variable, lo que le permite a ambos extremos, controlar la cantidad de bytes enviados y ajustar la velocidad en función de sus capacidades de procesamiento.

19. [Pregunta.](#)

Este paquete se envía en el segundo paso del handshake cuando se establece una conexión TCP. Si se pierde ese frame, como el emisor no recibió confirmación, reenvía el mensaje SYN original.

20. [Pregunta.](#)

Esta técnica consiste en enviar datos cuando aún no se completó un segmento. Esto permite romper con el pipeline y enviar mensajes de carácter urgente.

21. [Pregunta.](#)

Se utiliza arranque lento, que por cada ACK extiende la ventana en 1 hasta la mitad, luego espera a todos los ACK de la misma ventana para seguir extendiendo. También Disminución Multiplicativa que por cada segmento perdido reduce a la mitad la ventana y anula los temporizadores de los segmentos restantes.

22. [Pregunta.](#)

Un Glue record es una técnica utilizada en DNS al delegar subdominios cuyos servidores son subdominios también.

8. Glosario

- **Access Point:** Es la estación central de los sistemas inalámbricos distribuidos. Conecta una celda del sistema con otras celdas (o el internet) mediante enlaces físicos (como el Ethernet).
- **Acuse de recibo positivo con retransmisión (TCP):** Técnica utilizada en una conexión entre computadoras para lograr la confiabilidad en la transmisión. Cuando el receptor recibe un mensaje del emisor, le responde con un acuse de recibo para confirmarle que su mensaje llegó correctamente.
- **Ancho de banda:**
 1. (Analógico): El rango de frecuencias en el cual se transmite sin una atenuación considerable.
 2. (Digital): Tasa de datos máxima que se puede enviar por cierto canal de transmisión. Se mide en bits/segundo.
- **Archivo de zona:** Archivo local de un servidor DNS que describe la zona al que se refiere. Los datos se representan mediante registros y brindan información de propiedades generales del archivo, resolución de direcciones, etc.
- **Banda base:** Señal cuyo ancho de banda va desde cero hasta una frecuencia máxima.
- **Bridge (Puente):** Dispositivo de capa de enlace. Es un puente entre dos segmentos de red. Tiene dos puertos de entrada y salida, permitiendo procesar y rutear a nivel de capa de enlace las tramas transmitidas.
- **Byte/Bit bandera:** Sistema de reconocimiento de tramas basado en agregar bytes (o bits) que delimiten los inicios y finales de cada trama.
- **Byte/Bit de escape:** Utilizado en los sistemas de relleno de bytes/bits. Se utilizan para marcar si el byte/bit bandera entre encuentra en los datos de la trama. Se eliminan cuando el receptor lee la trama.
- **Conexión (TCP):** Abstracción utilizada por el software TCP para identificar un flujo de datos. Se identifica mediante un par de puntos extremos, uno referido al emisor y el otro referido al receptor.
- **CSMA/CA:** *Carrier Sense Multiple Access with Collision Avoidance*, protocolo utilizado para bajar la probabilidad de colisiones entre tramas enviadas por diferentes estaciones en una red inalámbrica de difusión.

- **Datagrama (IP):** Unidad básica de transferencia de mensajes del protocolo IP. Se compone por un encabezado y una sección de datos.
- **Datagrama del usuario (UDP):** No confundir con el datagrama de la capa de red. Es la unidad básica de transmisión entre máquinas del protocolo UDP.
- **DCF:** *Distributed Coordination Function*, mecanismo de acceso básico del estándar CSMA/CA. Soporta accesos al medio tanto de manera aleatoria o utilizando RTS/CTS. No es libre de contienda.
- **DMZ:** *Demilitarized Zone*, diseño conceptual de red, en la cual los servidores públicos viven en un segmento separado de la red.
- **DNS:** *Domain Name Server*, implementación del concepto de servidor de nombre. Agrega la idea de dominios con distinta jerarquía, resolviendo los problemas existentes en el modelo de servidores de nombre.
- **Dominio:** Técnicamente se podría considerar como un nodo en la estructura jerárquica de un servidor DNS. Cada dominio tiene un subdominio y un nombre de dominio asociado. Se suele utilizar como forma corta de referirse a un nombre de dominio.
- **DSSS:** *Direct Sequence Spread Spectrum*, método de ensanchamiento de espectro mediante la utilización de secuencias de bits que modulan la señal y proveen una mejor resistencia al ruido. Se suele utilizar la secuencia de Baker y a los bits de esta secuencia se los conoce como chips.
- **Extremo a Extremo (Capas):** Una capa es de Extremo a Extremo cuando se comunica directamente con la capa hermana en la máquina destino.
- **FHSS:** *Frequency-Hopping Spread Spectrum*, técnica de ensanchamiento de espectro. Esto se consigue transmitiendo la señal por una frecuencia en particular durante un período arbitrario (dwell time). Cuando este período expira, se “salta” a otra frecuencia y se sigue transmitiendo.
- **Firewall (Cortafuego):** Host especial de una red que hace de cuello de botella para la comunicación interredes. Se encarga de la seguridad de la red filtrando, modificando o redirigiendo los paquetes entrantes y salientes de la red con las redes externas.
- **Full-dúplex:**
 1. (Enlace): Enlace que se puede utilizar en ambas direcciones al mismo tiempo.

2. (Conexión): Consiste en dos flujos independientes que se mueven en direcciones opuestas, sin ninguna interacción aparente.
- **Glue record:** Técnica utilizada para romper con el problema de dependencia funcional en servidores DNS.
 - **Half-dúplex (Enlace):** Enlace que permite la comunicación en cualquier dirección, pero no al mismo tiempo, solamente una a la vez,
 - **Hub:** Dispositivo de Capa física. Contiene varios puertos de entrada y salida. Se encarga de retransmitir una señal que llegue por alguno de sus puertos a todos los demás puertos .
 - **ICMP:**
 - **IEEE 802.11:** Protocolo estándar para la transmisión de mensajes en una red inalámbrica. En la actualidad se utiliza la versión 802.11ac.
 - **IP:**
 - (TCP/IP): *Internet Protocol*, el protocolo oficial utilizado en la capa de interred del modelo TCP/IP.
 - **Iptables:** Interfaz de comandos Linux para la creación y configuración de firewalls.
 - **LLC:** *Link Logical Control*, es una subcapa de la capa de enlace. Proporciona mecanismos y protocolos para la delimitación de tramas, control de errores y el control de flujo. Es la subcapa superior.
 - **MAC**
 1. (Capa de enlace): *Medium Access Control*, es una subcapa de la capa de enlace. Controla el acceso al medio por parte de los hosts en las redes de difusión. Es la subcapa inferior.
 2. (Dirección mac, capa de red): Refiere a la dirección física de un dispositivo. En particular, es la dirección de la tarjeta de red del dispositivo.
 - **MIMO:** *Multiple Input/Multiple Output*, es una técnica que aprovecha las múltiples antenas de los access point para lograr una comunicación mediante canales paralelos e independientes que permiten enviar y recibir multiples tramas a la vez. Se mejora la velocidad de transmisión y la resistencia al ruido mediante BeamForming y multiplexado espacial.

- **Modelo OSI:** *Open System Interconnection*, es un modelo de referencia para la conexión de sistemas abiertos. Sus protocolos no se utilizan pero su modelo es ampliamente aceptado. Cuenta con siete capas: Física, Enlace, Red, Transporte, Sesión, Presentación y Aplicación.
- **Modelo TCP/IP:** El modelo de referencia TCP/IP sirve para modelar redes y soportar conexiones entre varias redes. Cuenta con 4 capas: Enlace, Interred, Transporte y Aplicación.
- **Name Server (NS):** Concepto de servidor encargado de resolver nombres de dominio a direcciones IP.
- **Named.conf:** Archivo de configuración almacenado en un servidor DNS. Tiene información sobre: El servidor en sí, su relación con las zonas que contiene y cómo se relacionan los hosts con el servidor.
- **NAT:** *Network Address Translation*, método traducción de direcciones IP de paquetes entrantes o salientes a la red. Iptables tiene una tabla dedicada a nat.
- **NAV:** *Network Assignment Vector*, vector que contiene información sobre el tiempo que una estación debe esperar para poder volver a intentar transmitir al medio. Usualmente este tiempo está expresado en microsegundos y el tiempo depende de la información a transferir.
- **Nombre de Dominio:** Nombre arbitrario que define la zona que fue delegada del dominio root. Un nombre de dominio tiene dueño registrado y es responsable y autoritario de la información de su DNS.
- **Pasa-banda:** Señal que ocupa un rango de frecuencias altas, como el de las transmisiones inalámbricas.
- **PCF:** *Punctual Coordination Function*, función de coordinación de acceso al medio libre de contienda. Se basa en algoritmos deterministas para decidir.
- **Protocolo (Entre capas):** Serie de reglas y convenciones que rigen la manera en la que dos máquinas se comunican dentro de la misma capa.
- **Puertos de protocolo:** Puntos abstractos de destino identificados por un número entero positivo.
- **Punto a Punto:**

1. (Red): Una red es del tipo Punto a Punto cuando sus máquinas están conectadas de a pares. Un mensaje puede pasar por varias máquinas antes de llegar a su destino.
 2. (Capa): Decimos que una capa es Punto a Punto cuando sus protocolos trabajan entre una máquina y sus dispositivos vecinos.
- **Punto extremo (TCP)**: Consiste en un par (*dirección, puerto*), donde *dirección* corresponde a la dirección IP y *puerto* corresponde al puerto TCP. Sirven para identificar una conexión TCP.
 - **Repetidor**: Dispositivo de capa física. Se utiliza para amplificar señal que viajan por un medio físico. Distintos medios pueden requerir repetidores a distintas distancias (depende de la atenuación del medio).
 - **Router**: Dispositivo de capa de red. Es utilizado para encontrar el camino (rutear) de un paquete que viaja de un anfitrión al otro mediante la red de redes.
 - **RTS/CTS**: Técnica utilizada por el protocolo CSMA/CA para evitar posibles colisiones en redes inalámbricas de difusión. Utiliza tramas de control RTS y CTS para crear un sistema en el cual las estaciones se anuncian al access point para enviar al medio y este les permite (o no) utilizarlo.
 - **RTT**: *Round Time Trip*, tiempo estimado de viaje redondo. En una conexión TCP representa el tiempo promedio desde que se envía un segmento hasta que se recibe su acuse de recibo. Es necesario calcularlo para determinar cuánto tiempo esperar un acuse antes de retransmitir un segmento.
 - **Segmento (TCP)**: Es la unidad básica de transmisión de mensajes del protocolo TCP. Estos llevan datos, acuses de recibos, establecen y cierran conexiones y anuncian tamaños de ventanas, entre otras cosas.
 - **Servicio (Entre capas)**: Conjunto de primitivas (operaciones) que una capa proporciona a la capa que esta por encima de ella.
 - **Servidor Esclavo**: Servidor DNS que depende de alguno otro servidor (Maestro) para conseguir y actualizar la información sobre alguna zona en particular. Esta se obtiene mediante la operación de transferencia de zonas y se actualiza mediante la operación NOTIFY.
 - **Servidor Maestro**: Servidor DNS que obtiene la información sobre alguna zona en particular directamente desde un archivo de zonas almacenado localmente.

- **Simplex (Enlace)** Enlace que solo permite el tráfico en una única dirección.
- **SNR:** *Signal-to-Noise Ratio*, es la relación entre la potencia de una señal y la potencia del ruido en un canal. Este coeficiente se calcula como S/N .
- **Subdominio:** Nombre arbitrario que es colocado por el dueño de un nombre de dominio. Un nombre se subdominio incluye completamente al nombre del dominio al que pertenece. Por ejemplo: `jcc.lcc.com` es un subdominio de `lcc.com`.
- **Subred:**
 1. (Hardware de Red): Colección de enrutadores y líneas de comunicación que transmiten paquetes desde el host de origen hasta el host de destino.
 - 2.
- **Switch:** Dispositivo de capa de enlace. Es utilizado en las topologías Punto a Punto. Cuenta con varios puertos a los cuales se pueden conectar distintas máquinas. El switch transmite paquetes entre las computadoras conectadas a el y utiliza la dirección en cada paquete para determinar a qué computadora se lo debe enviar. Se dice que es un bridge multipuerto.
- **TCP:** *Transmission Control Protocol*, Protocolo de transporte orientado a la conexión y confiable, es de Extremo a Extremo. Pertenece a la capa de transporte del modelo TCP/IP.
- **TLD:** *Top Level Domain*, son los dominios con mayor jerarquía después de root. Se dividen en varios grupos, dos de ellos son los gTLD (*General TLD*), y los ccTLD (Country Code TLD).
- **Trama de Control:** Tramas especiales utilizadas por la capa de enlace para comunicar información especial entre el emisor y el receptor. Ejemplos de estas tramas son las tramas RTS/CTS y ACK.
- **Trama de Datos:** Colección de bytes que transmite la capa de enlace. Se puede decir que es la unidad en la que con la que trata la capa de enlace.
- **Trama de Gestión:** Tramas que solo transmiten información de administración. Estas tramas no son transmitidas a capas superiores
- **UDP:** *User Datagram Protocol*, protocolo de transporte con entrega sin conexión no confiable, que utiliza el IP para transportar los mensajes y permite distinguir entre varios destinos dentro de una computadora anfitrión gracias a los puertos de protocolo. Pertenece a la capa de transporte del modelo TCP/IP.

- **Ventana deslizable:** Técnica utilizada en TCP para enviar datos sin la necesidad de esperar a que llegue un acuse de recibo para enviar el dato siguiente. Esta técnica ofrece control de flujo y aprovecha mejor el ancho de banda de una red que un sistema de acuse de recibo simple.
- **VLAN:** Red LAN virtual, usualmente se utiliza para dividir lógicamente una red LAN en distintas redes “virtuales”.
- **Zona** (Nombre de zona): La parte de un dominio que está configurada en un servidor DNS e incluye completamente al nombre de dominio del cual es servidor es autoritario.

9. Bibliografía

1. Redes de Computadoras - Andrew Tanenbaum - 5ta edición.
2. Redes Globales de Información con Internet y TCP/IP - Douglas Comer - 4ta edición.
3. Apunte Redes Inalámbricas de la cátedra - F.E. Spetale, P. Bulacio, J. Kohan.
4. Apunte IPv4 de la cátedra - F.E. Spetale, P. Bulacio, J. Kohan.
5. Curso IPv6 básico - IPv6.br
6. Slides IPv6 de la cátedra - F.E. Spetale, P. Bulacio, J. Kohan.
7. DNS for rocket scientists ([zytrax](#)).
8. Tutorial Iptables.
9. Linux Network Administrator Guide - 3ra edición.